Các Main Flow cần xem xét cho AITasker

Nhóm 1 - Onboarding & Account Setup (Nền tảng cho mọi thứ)

- **MF-1: Client (CEO) Registration & Subscription** - tự đăng ký, nạp ví, mua Client Pro để mở khóa elicitation engine
- **MF-2: Expert Registration, Profile & Verification** - đăng ký, tạo profile, claim seams (Tier 1 → LLM extraction Tier 2 → Scenario Tier 3), link ngân hàng qua Bank Hub
- **MF-3: Tech Team Account Creation via Handoff Link** - không tự đăng ký được; CEO tạo signed link → TECH_TEAM nhận quyền qua link đó

Nhóm 2 - Path A: Project-Based Flow (Flow chính, phức tạp nhất)

- **MF-4: AI Elicitation Engine (Project Intake)** - CEO nhập raw intent → 4-stage guided conversation (Scenario A: có Tech Team / Scenario B: không có) → Footprint (Archetype + Tier + Seam map) → Auto-publish quality gate → Spec PUBLISHED hoặc RETURNED_TO_CLIENT
- **MF-5: AI Matching & Shortlisting** - engine tính composite score (seam alignment 40%, domain 25%, archetype-tier 20%, engagement model 10%, stack 5%) → hiển thị shortlist với nhãn Strong/Qualified/Conditional
- **MF-6: Bid & Connection Flow (Non-linear)** - Expert xem Artifact A → spec clarification (Surface A) → submit 3-component bid → TECH_TEAM review → revision loop (Surface B) → price negotiation (Surface C) → CEO override (Surface D) → SELECTED → Connection request → CONNECTED
- **MF-7: Milestone Management & Escrow Payment** - CEO tạo milestone + acceptance criteria + DoD items → CEO fund (VietQR QR scan → SePay IPN → FUNDED) → Expert tạo sprint plan → weekly sprint updates → DoD checklist → submit deliverable → TECH_TEAM/CEO sign-off → APPROVED → chi hộ auto-transfer → RELEASED
- **MF-8: Dispute Resolution (3 Layers)** - DISPUTED → Layer 1: LLM auto-resolve (≥0.80 confidence) → Layer 2: mutual agreement → Layer 3: 50/50 split

---

Nhóm 3 - Path B: Service Marketplace Flow

- **MF-9: Expert Service Publishing** - Expert (Pro) dùng AI Service Generator → tạo service listing → publish lên marketplace
- **MF-10: Client Buys Service / Tech Discovery** - browse marketplace → chọn service → SERVICE_PURCHASE (single VA payment, no bid, no elicitation) hoặc TECH_DISCOVERY session

---

Nhóm 4 - Financial Flows

- **MF-11: Wallet Top-Up** - User scan per-user VietQR → SePay IPN → wallet credited (internal ledger)
- **MF-12: Expert Withdrawal** - Expert request → ledger debit → SePay chi hộ API → tự động chuyển khoản → credit IPN → COMPLETED (zero admin action)
- **MF-13: Subscription Purchase from Wallet** - deduct from available_balance → unlock gated features

---

Nhóm 5 - Advanced/Edge Flows

- **MF-14: Add-On Phase Protocol (Scope Evolution)** - Sprint status = BLOCKER/SCOPE_EVOLUTION → F8 auto-trigger → Expert proposes add-on → TECH_TEAM approves technical justification → CEO approves budget/timeline → new milestone appended
- **MF-15: Dual-Role Account Switch** - user có cả CLIENT_CEO + EXPERT roles → switch active_role → JWT reissued → dashboard thay đổi; self-exclusion rule vẫn apply
- **MF-16: Expert Verification Auto-Upgrade (Tier 4)** - platform tự evaluate sau mỗi engagement close, không cần admin action

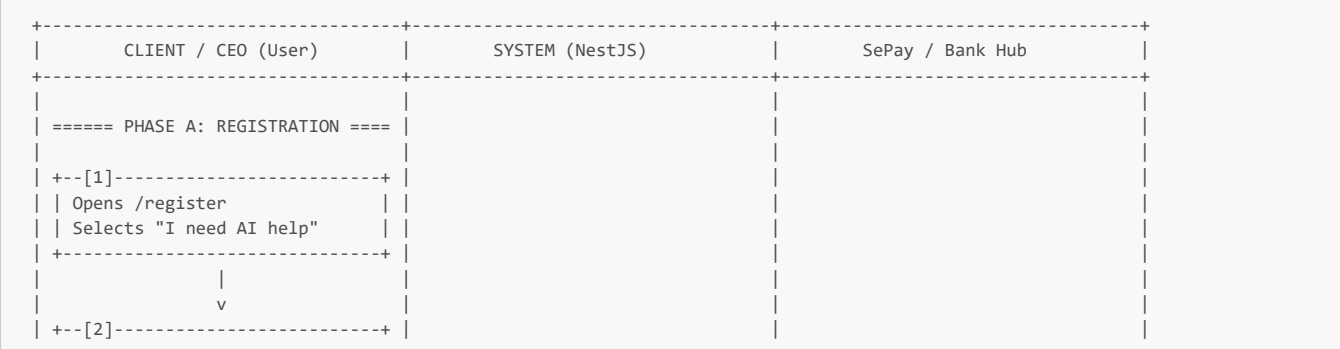
---

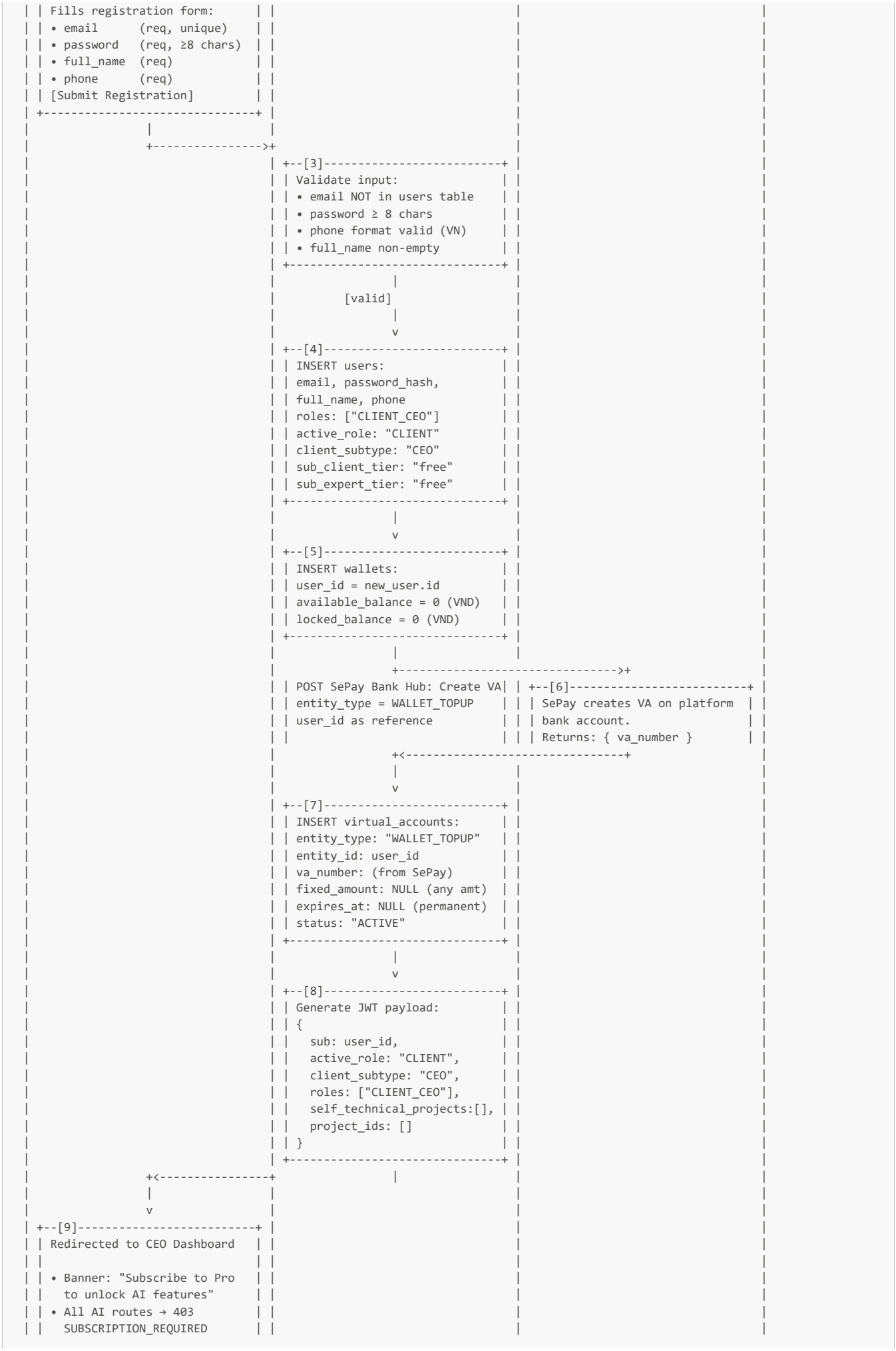
Nhóm 6 - Admin Flows

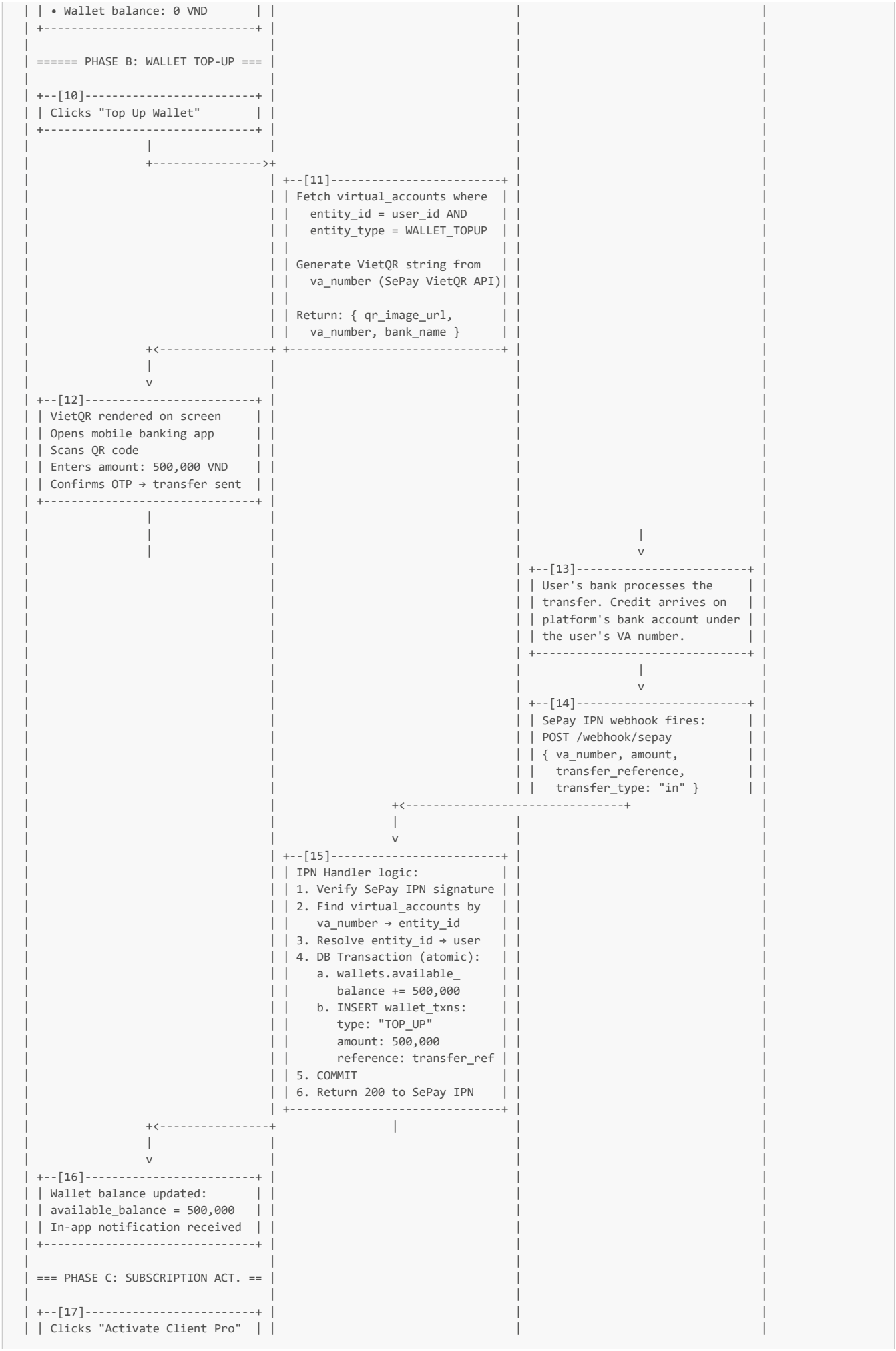
- **MF-17: Platform Integrity Monitor & Analytics** - đọc `platform_decisions` log, xem auto-upgrade/return history, bid conflict override log, analytics dashboard
- **MF-18: Account Management & Fraud Detection** - suspend/reactivate users, bank account deduplication check (cùng tài khoản ngân hàng trên 2 expert → auto-block)

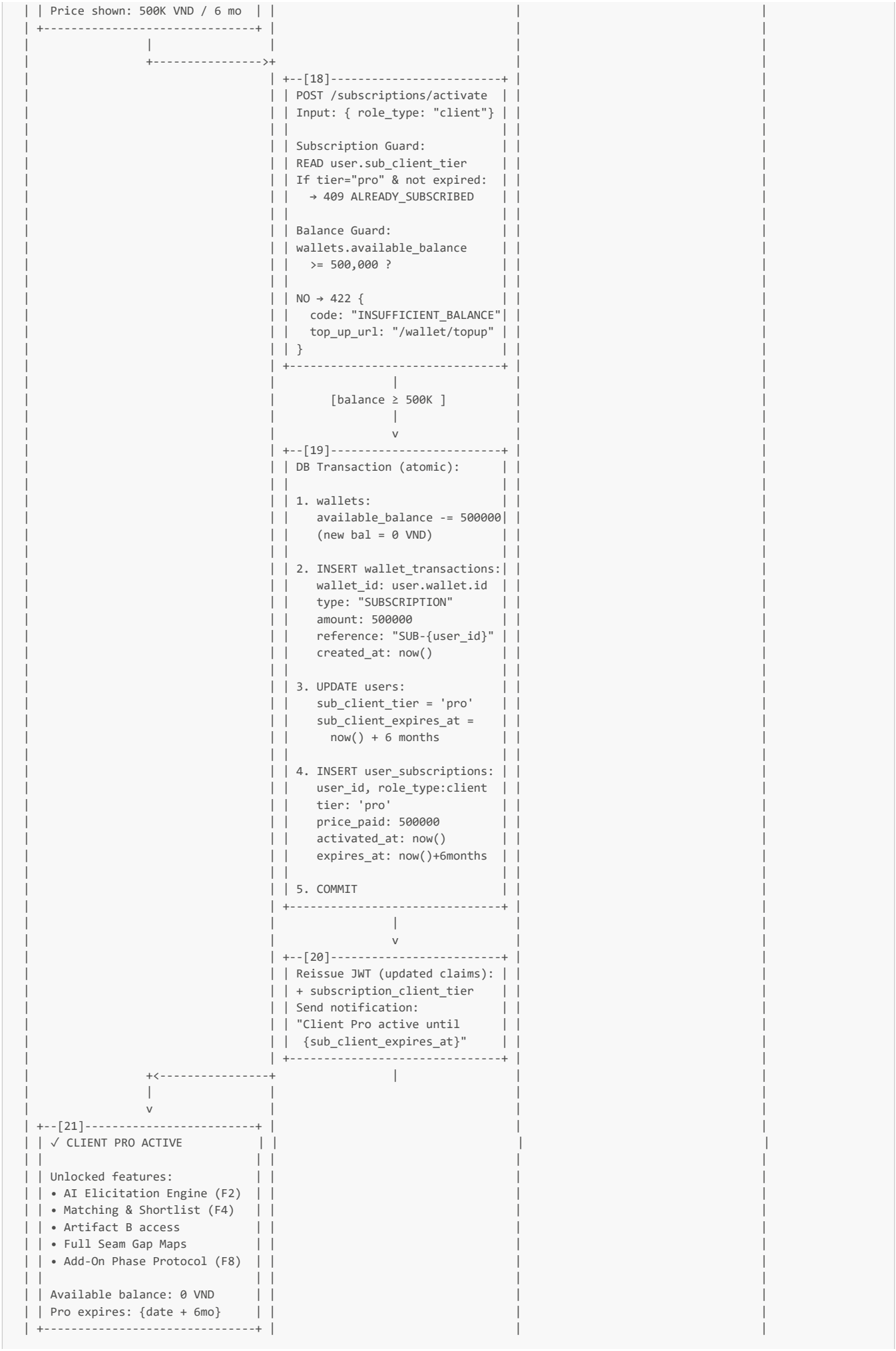
MF-1: Client (CEO) Registration & Subscription

Detailed Swimlane Flow




Fills registration form:
• email (req, unique)
• password (req, ≥8 chars)
• full_name (req)
• phone (req)
[Submit Registration]
[3]
Validate input:
• email NOT in users table
• password ≥ 8 chars
• phone format valid (VN)
• full_name non-empty
[valid]
[4]
INSERT users:
email, password_hash,
full_name, phone
roles: ["CLIENT_CEO"]
active_role: "CLIENT"
client_subtype: "CEO"
sub_client_tier: "free"
sub_expert_tier: "free"
[5]
INSERT wallets:
user_id = new_user.id
available_balance = 0 (VND)
locked_balance = 0 (VND)
POST SePay Bank Hub: Create VA
entity_type = WALLET_TOPUP
user_id as reference
[6]
SePay creates VA on platform
bank account.
Returns: { va_number }
[7]
INSERT virtual_accounts:
entity_type: "WALLET_TOPUP"
entity_id: user_id
va_number: (from SePay)
fixed_amount: NULL (any amt)
expires_at: NULL (permanent)
status: "ACTIVE"
[8]
Generate JWT payload:
{
sub: user_id,
active_role: "CLIENT",
client_subtype: "CEO",
roles: ["CLIENT_CEO"],
self_technical_projects:[],
project_ids: []
}
[9]
Redirected to CEO Dashboard
• Banner: "Subscribe to Pro to unlock AI features"
• All AI routes → 403 SUBSCRIPTION_REQUIRED


• Wallet balance: 0 VND
====== PHASE B: WALLET TOP-UP ===
[10]
Clicks "Top Up Wallet"
[11]
Fetch virtual_accounts where
entity_id = user_id AND
entity_type = WALLET_TOPUP
Generate VietQR string from
va_number (SePay VietQR API)
Return: { qr_image_url,
va_number, bank_name }
[12]
VietQR rendered on screen
Opens mobile banking app
Scans QR code
Enters amount: 500,000 VND
Confirms OTP → transfer sent
[13]
User's bank processes the
transfer. Credit arrives on
platform's bank account under
the user's VA number.
[14]
SePay IPN webhook fires:
POST /webhook/sepay
{ va_number, amount,
transfer_reference,
transfer_type: "in" }
[15]
IPN Handler logic:
1. Verify SePay IPN signature
2. Find virtual_accounts by
va_number → entity_id
3. Resolve entity_id → user
4. DB Transaction (atomic):
a. wallets.available_
balance += 500,000
b. INSERT wallet_txns:
type: "TOP_UP"
amount: 500,000
reference: transfer_ref
5. COMMIT
6. Return 200 to SePay IPN
[16]
Wallet balance updated:
available_balance = 500,000
In-app notification received
=== PHASE C: SUBSCRIPTION ACT. ==
[17]
Clicks "Activate Client Pro"


Price shown: 500K VND / 6 mo
[18]
POST /subscriptions/activate
Input: { role_type: "client"}
Subscription Guard:
READ user.sub_client_tier
If tier="pro" & not expired:
→ 409 ALREADY_SUBSCRIBED
Balance Guard:
wallets.available_balance
>= 500,000 ?
NO → 422 {
code: "INSUFFICIENT_BALANCE"
top_up_url: "/wallet/topup"
}
[balance ≥ 500K]
[19]
DB Transaction (atomic):
1. wallets:
available_balance -= 500000
(new bal = 0 VND)
2. INSERT wallet_transactions:
wallet_id: user.wallet.id
type: "SUBSCRIPTION"
amount: 500000
reference: "SUB-{user_id}"
created_at: now()
3. UPDATE users:
sub_client_tier = 'pro'
sub_client_expires_at =
now() + 6 months
4. INSERT user_subscriptions:
user_id, role_type:client
tier: 'pro'
price_paid: 500000
activated_at: now()
expires_at: now()+6months
5. COMMIT
[20]
Reissue JWT (updated claims):
+ subscription_client_tier
Send notification:
"Client Pro active until
{sub_client_expires_at}"
[21]
✓ CLIENT PRO ACTIVE
Unlocked features:
• AI Elicitation Engine (F2)
• Matching & Shortlist (F4)
• Artifact B access
• Full Seam Gap Maps
• Add-On Phase Protocol (F8)
Available balance: 0 VND
Pro expires: {date + 6mo}

```
|                                  |                                  |                                  |
| === NEXT: Top-up again to fund ===|                                  |                                  |
| === first project milestone    ===|                                  |                                  |
|                                  |                                  |                                  |
+----------------------------------+----------------------------------+----------------------------------+
```

Step-by-Step Narration

PHASE A - REGISTRATION

[1] - Opens Registration Page, Selects "I need AI help"

- **Actor:** Client / CEO (unauthenticated user)
- **Input:** User navigates to `/register`. Clicks the "I need AI help" path selector.
- **Logic:** The path selector determines which `roles` array and `active_role` will be assigned on account creation. Selecting "I need AI help" sets the registration context to `CLIENT_CEO`. The alternative path ("I provide AI services") would set `EXPERT`. This choice is **irreversible for this session** - it determines the primary role.
- **Output:** The registration form renders with CLIENT_CEO-specific fields and copy.
- **State Change:** None yet - UI-only step.

[2] - Fills Registration Form

- **Actor:** Client / CEO
- **Input (form fields):**

Field	Type	Constraints	Purpose
`email`	string, required	Valid email format; must be unique in `users` table	Login identifier + communication
`password`	string, required	≥ 8 characters	Authentication credential (bcrypt hashed before storage)
`full_name`	string, required	Non-empty	Display name on profile, notifications, contracts
`phone`	string, required	Valid VN phone format (10–11 digits)	SePay VA reference; future 2FA

- **Logic:** Client-side validation runs on blur (email format, password length, phone pattern). The `[Submit Registration]` button is disabled until all required fields pass client validation.
- **Output:** A `POST /auth/register` request with payload `{ email, password, full_name, phone, registration_path: "CLIENT_CEO" }`.
- **State Change:** None yet - request in flight.

[3] - Validate Input (Server-Side)

- **Actor:** System (NestJS - AuthController + AuthService)
- **Input:** The registration payload from [2].
- **Logic:**
  1. **Email uniqueness check:** `SELECT 1 FROM users WHERE email = ?` - if any row exists, return `409 CONFLICT { code: "EMAIL_EXISTS" }`.
  2. **Password constraint:** verify `length ≥ 8`; if fail, return `400 BAD_REQUEST { code: "PASSWORD_TOO_SHORT" }`.
  3. **Phone format:** regex `^(0[3|5|7|8|9])+([0-9]{8})$`; if fail, return `400 BAD_REQUEST { code: "INVALID_PHONE" }`.
  4. **Full name non-empty:** trimmed length > 0.
- **Output:** If all validations pass → proceed to [4]. If any fail → HTTP 4xx error response with specific code.
- **State Change:** None - read-only validation.

[4] - Insert User Record

- **Actor:** System (NestJS - AuthService)
- **Input:** Validated registration payload.
- **Logic:** Hash password with bcrypt (salt rounds = 10). Construct the `users` INSERT with **role fields determined by registration path**:

Field	Value Set	Rationale
`email`	from input	Login identifier
`password_hash`	bcrypt hash	Never store plaintext
`full_name`	from input	Display name
`phone`	from input	Contact + VA reference
`roles`	`["CLIENT_CEO"]`	JSONB array - single role at creation; can add EXPERT later via Account Settings
`active_role`	`"CLIENT"`	Current JWT context

Field	Value Set	Rationale
`client_subtype`	`"CEO"`	Distinguishes from TECH_TEAM within CLIENT role
`subscription_client_tier`	`"free"`	Default - all AI features gated
`subscription_expert_tier`	`"free"`	Default - not yet an expert
`sub_client_expires_at`	`NULL`	No subscription yet
`sub_expert_expires_at`	`NULL`	No subscription yet
`sepay_bank_account_xid`	`NULL`	Not applicable for CEO (only experts link bank accounts for chi hộ)

- **Output:** `user_id` (auto-generated UUID or auto-increment) for use in [5] and [7].
- **State Change:** `users` table gains one row. User now exists in the system with CLIENT_CEO role, free tier.
- **Tables Affected:** `users`

---

[5] - Insert Wallet Record

- **Actor:** System (NestJS - WalletService, called from AuthService within same transaction)
- **Input:** `user_id` from [4].
- **Logic:** Create the user's internal ledger wallet. Every user gets exactly one wallet, regardless of how many roles they hold. The wallet is the **sole source of truth** for all financial operations - subscriptions, escrow, withdrawals all reference this single row.

Field	Value Set	Rationale
`user_id`	from [4]	FK to users
`available_balance`	`0` (BIGINT, VND)	Spendable balance - can be used for subscriptions, milestone funding
`locked_balance`	`0` (BIGINT, VND)	Escrow-locked - funds committed to funded milestones; not spendable

- **Output:** `wallet_id` for reference.
- **State Change:** `wallets` table gains one row. User now has a financial identity on the platform.
- **Tables Affected:** `wallets`

---

[6] - Create WALLET_TOPUP Virtual Account via SePay Bank Hub

- **Actor:** SePay (External API)
- **Input:** System sends `POST` to SePay Bank Hub API with request to create a virtual account for wallet top-up.
- **Logic:** SePay creates a sub-account (VA) under the platform's master bank account. This VA is uniquely identified and will appear on the user's VietQR. Any bank transfer credited to this VA is automatically attributed to this user.
- **Output:** `{ va_number: "XXXXXX..." }` - the virtual account number that will be embedded in the user's VietQR.
- **State Change:** SePay-side: new VA created and linked to platform's bank account.

---

[7] - Store Virtual Account Record

- **Actor:** System (NestJS)
- **Input:** `va_number` from SePay [6], `user_id` from [4].
- **Logic:** Persist the VA mapping so that when a SePay IPN arrives with this `va_number`, the system can resolve it back to the user's wallet.

Field	Value Set	Rationale
`entity_type`	`"WALLET_TOPUP"`	Distinguishes from MILESTONE, SERVICE, SUBSCRIPTION VAs
`entity_id`	`user_id`	Maps VA → user for IPN resolution
`va_number`	from SePay [6]	The actual VA number used in VietQR
`fixed_amount`	`NULL`	No fixed amount - user can top up any value
`expires_at`	`NULL`	Permanent VA - never expires; user can top up anytime
`status`	`"ACTIVE"`	Ready to receive transfers

- **Output:** VA record persisted.
- **State Change:** `virtual_accounts` table gains one row. User now has a permanent payment channel.
- **Tables Affected:** `virtual_accounts`

---

[8] - Generate JWT

- **Actor:** System (NestJS - AuthService)
- **Input:** `user_id`, `active_role`, `client_subtype`, `roles` from [4].
- **Logic:** Construct JWT payload per the F1 specification:

```
{
  "sub": "user_id",
  "active_role": "CLIENT",
  "client_subtype": "CEO",
  "roles": ["CLIENT_CEO"],
  "self_technical_projects": [],
  "project_ids": []
}
```

Key design decisions in the JWT:

- `active_role = "CLIENT"` (not "CLIENT_CEO") - the role switcher and guards use `active_role` + `client_subtype` as separate fields.
- `self_technical_projects` is empty at registration - populated later if CEO completes elicitation as self-technical.
- `project_ids` is empty - populated as CEO creates projects.
- The JWT is signed with the server's secret and set as an HTTP-only cookie.
- **Output:** JWT token string. Set in `Set-Cookie` header.
- **State Change:** None - token generation is stateless.

---

[9] - CEO Dashboard Loads (Free Tier)

- **Actor:** Client / CEO (now authenticated)
- **Input:** JWT from [8]. Frontend reads JWT claims to determine dashboard layout.
- **Logic:** The CEO Dashboard renders with the following gated state:
  - **Subscription banner** visible: "*Subscribe to Client Pro to unlock AI features*" with CTA button linking to subscription page.
  - **All FastAPI-calling routes return 403:** The subscription guard middleware checks `user.subscription_client_tier`. Since it is `"free"`, any request to `/elicitation/*`, `/matching/*`, `/artifact-b/*`, `/add-on/*` returns `403 { code: "SUBSCRIPTION_REQUIRED", feature: "elicitation_engine", upgrade_url: "/subscribe" }`.
  - **Wallet balance displays 0 VND:** The dashboard reads `GET /wallet` which returns `{ available_balance: 0, locked_balance: 0 }`.
  - **Marketplace browsing works:** Per Section 0.9, browsing the expert marketplace (Path B) and buying SERVICE_PURCHASE are available on free tier - no subscription required.
- **Output:** Dashboard UI rendered in free-tier mode.
- **State Change:** None - read-only display of current state.

PHASE B - WALLET TOP-UP

---

[10] - Clicks "Top Up Wallet"

- **Actor:** Client / CEO
- **Input:** User clicks the "Top Up Wallet" button from either the subscription page or the wallet section of the dashboard.
- **Logic:** Frontend prepares to request the top-up QR by calling the wallet top-up endpoint.
- **Output:** `GET /wallet/topup` request sent.
- **State Change:** None.

---

[11] - Retrieve VA and Generate VietQR

- **Actor:** System (NestJS - WalletController)
- **Input:** JWT identifies the user.
- **Logic:**
  1. Query `virtual_accounts` where `entity_id = user_id AND entity_type = 'WALLET_TOPUP' AND status = 'ACTIVE'`.
  2. Retrieve `va_number`.
  3. Generate VietQR string using SePay's VietQR API (or a VietQR generation library). The QR encodes: bank account number = platform's master account, sub-account = `va_number`, amount = user-specified or open (no fixed amount for WALLET_TOPUP), memo = `"AITasker TopUp {user_id}"`.
  4. Return the QR image URL (or base64) plus the VA details for display.
- **Output:**

```
{
  "qr_image_url": "https://img.vietqr.io/...",
  "va_number": "XXXXXX",
  "bank_name": "MB Bank",
  "account_holder": "AITasker Platform",
```

```
  "instructions": "Scan with any banking app. Any amount will be credited to your wallet."
}
```

- **State Change:** None - read-only retrieval and QR generation.

---

[12] - Scans VietQR, Transfers Funds

- **Actor:** Client / CEO (via their personal banking app)
- **Input:** The VietQR displayed on screen. User opens their mobile banking app (MB Bank, Vietcombank, Techcombank, etc.), scans the QR, reviews the pre-filled transfer details, enters the desired amount (e.g., 500,000 VND), and confirms with OTP.
- **Logic:** This step occurs entirely **outside the platform** - the user's banking app communicates with their bank's servers. The platform has no visibility into this step until the SePay IPN arrives.
- **Output:** A bank transfer instruction from the user's personal account to the platform's master bank account, with the user's VA number as the sub-account identifier.
- **State Change:** None on the platform side.

---

[13] - Bank Processes Transfer

- **Actor:** SePay / Banking infrastructure
- **Input:** The bank transfer instruction from [12].
- **Logic:** The interbank system processes the transfer. The credit arrives on the platform's bank account. SePay's monitoring detects the credit on the specific VA number and prepares the IPN webhook.
- **Output:** Credit confirmed on VA. SePay prepares IPN payload.
- **State Change:** External (SePay-side) - credit recorded on VA.

---

[14] - SePay IPN Webhook Fires

- **Actor:** SePay (automated webhook)
- **Input:** SePay sends `POST /webhook/sepay` to the platform's NestJS server.
- **Logic:** SePay triggers this webhook automatically when it detects a credit on any VA under the platform's master account.
- **Output (IPN payload):**

```
{
  "va_number": "XXXXXX",
  "amount": 500000,
  "transfer_reference": "FT25000XXXXX",
  "transfer_type": "in",
  "description": "AITasker TopUp user_id",
  "timestamp": "2026-05-15T10:30:00Z"
}
```

- **State Change:** None yet - payload received, awaiting processing.

---

[15] - IPN Handler: Validate, Resolve, Credit Wallet

- **Actor:** System (NestJS - SePayWebhookController + WalletService)
- **Input:** IPN payload from [14].
- **Logic (6 sub-steps):**

  Step 15a - Verify IPN signature:

  - Validate the webhook's HMAC/signature using the platform's SePay API key.
  - If invalid → log warning, return `200 OK` (to prevent SePay retries) but take no financial action.

  Step 15b - Resolve VA → User:

  - `SELECT * FROM virtual_accounts WHERE va_number = ? AND status = 'ACTIVE'`
  - From the result: `entity_type = 'WALLET_TOPUP'`, `entity_id = user_id`.
  - This resolves the incoming payment to a specific user's wallet.

  Step 15c - Idempotency check:

  - `SELECT 1 FROM wallet_transactions WHERE reference_id = ?` using `transfer_reference` from the IPN.
  - If a row exists → this is a duplicate IPN (SePay may retry). Return `200 OK`, take no action.
  - This prevents double-crediting the wallet.

  Step 15d - Credit wallet (DB transaction):

```
BEGIN;
UPDATE wallets
  SET available_balance = available_balance + 500000
  WHERE user_id = ?;

INSERT INTO wallet_transactions
  (wallet_id, amount, transaction_type, reference_id, created_at)
  VALUES (?, 500000, 'TOP_UP', 'FT25000XXXXX', now());
COMMIT;
```

This is **atomic** - if any part fails, the entire transaction rolls back. The wallet is never credited without a corresponding ledger entry, and vice versa.

Step 15e - Return 200 OK to SePay:

- SePay expects a 200 response to confirm the IPN was processed. Failure to respond may trigger SePay retries.

Step 15f - Push notification:

- Trigger in-app notification: "*Your wallet has been credited with 500,000 VND.*"
- If the user has the dashboard open, the wallet balance updates via WebSocket or polling.

- **Output:** HTTP 200 to SePay. In-app notification to user.
- **State Change:**
  - `wallets.available_balance`: `0` → `500,000`
  - `wallet_transactions`: new row with `type = "TOP_UP"`, `amount = 500000`
- **Tables Affected:** `wallets` (UPDATE), `wallet_transactions` (INSERT)
- **Ledger entry:** `[TOP_UP] available_balance += 500,000` - pure internal ledger, no external API call.

---

[16] - Wallet Balance Updated on Dashboard

- **Actor:** Client / CEO
- **Input:** In-app notification from [15f]. Dashboard refreshes wallet display.
- **Logic:** Frontend either receives a WebSocket push or polls `GET /wallet` and updates the displayed balance.
- **Output:** Dashboard now shows `Available Balance: 500,000 VND`. The "Top Up Wallet" flow is complete. User can now proceed to subscription activation.
- **State Change:** None - UI update only.

PHASE C - SUBSCRIPTION ACTIVATION

---

[17] - Clicks "Activate Client Pro"

- **Actor:** Client / CEO
- **Input:** User clicks the "Activate Client Pro" button (shown on the subscription page or the persistent banner). The price is displayed: **500,000 VND / 6 months**.
- **Logic:** Frontend prepares the subscription activation request.
- **Output:** `POST /subscriptions/activate` with body `{ role_type: "client" }`.
- **State Change:** None.

---

[18] - Subscription Guard + Balance Guard

- **Actor:** System (NestJS - SubscriptionController + SubscriptionGuard + WalletGuard)
- **Input:** JWT (identifies user) + `{ role_type: "client" }` from request body.
- **Logic (two sequential guards):**

  Guard 1 - Already subscribed?

  - Read `user.subscription_client_tier` and `user.sub_client_expires_at`.
  - If `tier = "pro"` AND `expires_at > now()` → return `409 CONFLICT { code: "ALREADY_SUBSCRIBED", expires_at: "..." }`.
  - This prevents double-charging.

  Guard 2 - Sufficient balance?

  - Read `wallet.available_balance` for this user.
  - Compare against `subscription_price["client"] = 500,000 VND`.
  - If `available_balance < 500,000` → return `422 UNPROCESSABLE_ENTITY`:

```
```

```
{
  "code": "INSUFFICIENT_BALANCE",
  "current_balance": 300000,
  "required_amount": 500000,
  "shortfall": 200000,
  "top_up_url": "/wallet/topup"
}
```

    - The user is directed back to Phase B to top up the shortfall.

- **Output (happy path):** Guards pass → proceed to [19].
- **Output (error paths):** HTTP 409 or 422 with actionable error codes.
- **State Change:** None - read-only guard checks.

---

[19] - Atomic Subscription Activation (DB Transaction)

- **Actor:** System (NestJS - SubscriptionService)
- **Input:** User confirmed, balance sufficient.
- **Logic:** This is the **core financial transaction** of the subscription flow. It must be atomic - all 4 steps succeed or all roll back. No partial state is acceptable (e.g., wallet debited but subscription not activated).

```
BEGIN TRANSACTION;

-- Step 1: Debit wallet
UPDATE wallets
  SET available_balance = available_balance - 500000
  WHERE user_id = ? AND available_balance >= 500000;
-- (Row-level lock + condition ensures no race condition)

-- Step 2: Write ledger entry
INSERT INTO wallet_transactions
  (wallet_id, amount, transaction_type, reference_id, created_at)
  VALUES (?, 500000, 'SUBSCRIPTION', 'SUB-{user_id}', now());

-- Step 3: Update user subscription fields
UPDATE users
  SET subscription_client_tier = 'pro',
      sub_client_expires_at = now() + interval '6 months'
  WHERE id = ?;

-- Step 4: Write audit record
INSERT INTO user_subscriptions
  (user_id, role_type, tier, price_paid, activated_at, expires_at)
  VALUES (?, 'client', 'pro', 500000, now(), now() + interval '6 months');

COMMIT;
```

Why each step exists:

- **Step 1** moves money out of available balance (the user can no longer spend this 500K on anything else).
- **Step 2** creates the immutable audit trail - every VND movement has a ledger row.
- **Step 3** flips the subscription gate - subsequent API calls to LLM routes will now pass the guard.
- **Step 4** creates a separate audit record with full activation metadata (price, dates) for the `user_subscriptions` table.

Ledger impact summary:

- `[SUBSCRIPTION] available_balance -= 500,000` - pure internal ledger, no external API call, no SePay involved. The money was already in the wallet from Phase B; this is an internal reclassification.

Post-transaction state:

Table	Field	Before	After
`wallets`	`available_balance`	500,000	0
`wallets`	`locked_balance`	0	0
`users`	`subscription_client_tier`	`"free"`	`"pro"`
`users`	`sub_client_expires_at`	`NULL`	`now() + 6 months`

Table	Field	Before	After
`wallet_transactions`	new row	-	`type=SUBSCRIPTION, amount=500000`
`user_subscriptions`	new row	-	`tier=pro, price_paid=500000`

- **Output:** Transaction committed successfully.
- **State Change:** User is now Client Pro. Subscription gate opens for all AI features.
- **Tables Affected:** `wallets` (UPDATE), `wallet_transactions` (INSERT), `users` (UPDATE), `user_subscriptions` (INSERT)
- **External API calls:** None. This is a pure internal ledger operation.

---

[20] - Reissue JWT + Notify User

- **Actor:** System (NestJS - AuthService + NotificationService)
- **Input:** Updated user record from [19].
- **Logic:**
    1. **JWT reissue:** The existing JWT is invalidated and a new JWT is generated with updated claims (subscription status reflected). This ensures that subsequent requests don't need a DB lookup to verify subscription status - the guard can check JWT claims directly, or optionally still validate against DB for security.
    2. **Notification:** Push in-app notification and (optionally) email:
        - "*Your Client Pro subscription is now active. Expires: {sub_client_expires_at formatted as DD/MM/YYYY}. You now have access to the AI Elicitation Engine, Expert Matching, and all Pro features.*"
    3. **7-day expiry warning schedule:** The system schedules a notification to be sent 7 days before `sub_client_expires_at`. When expiry hits, the subscription guard will downgrade the user back to `"free"` (active engagements are grandfathered - in-progress work continues, but new AI features are blocked).
- **Output:** New JWT in `Set-Cookie` header. Notification delivered.
- **State Change:** JWT claims updated. Notification record created.

---

[21] - Client Pro Active - Dashboard Unlocked

- **Actor:** Client / CEO
- **Input:** New JWT from [20]. Dashboard refreshes.
- **Logic:** The CEO Dashboard re-renders with all Pro features visible and accessible:

Feature	Status	Route
AI Elicitation Engine (F2)	Unlocked	`/projects/new` (starts Stage 1)
Matching Engine & Shortlist (F4)	Unlocked	`/projects/{id}/shortlist`
Artifact B Access	Unlocked	Visible post-connection
Full Seam Gap Maps	Unlocked	On match cards
Add-On Phase Protocol (F8)	Unlocked	Triggered by SCOPE_EVOLUTION sprint
Expert Marketplace Browsing	Was already free	`/marketplace`

The subscription banner disappears. The dashboard shows:

- **Available Balance: 0 VND** - the 500K was consumed by the subscription.
- **Pro Badge** with expiry date in the navigation bar.
- **CTA: "Start Your First AI Project"** - links to the Elicitation Engine (MF-4).
- **Output:** Full CEO Dashboard in Pro mode.
- **State Change:** None - UI reflects the committed state from [19].

Summary: End-to-End State Trace for MF-1

Step	`users` row	`wallets` row	`wallet_transactions`	`virtual_accounts`	`user_subscriptions`
[4] Registration	roles=["CLIENT_CEO"], active_role="CLIENT", client_subtype="CEO", sub_client_tier="free"	-	-	-	-

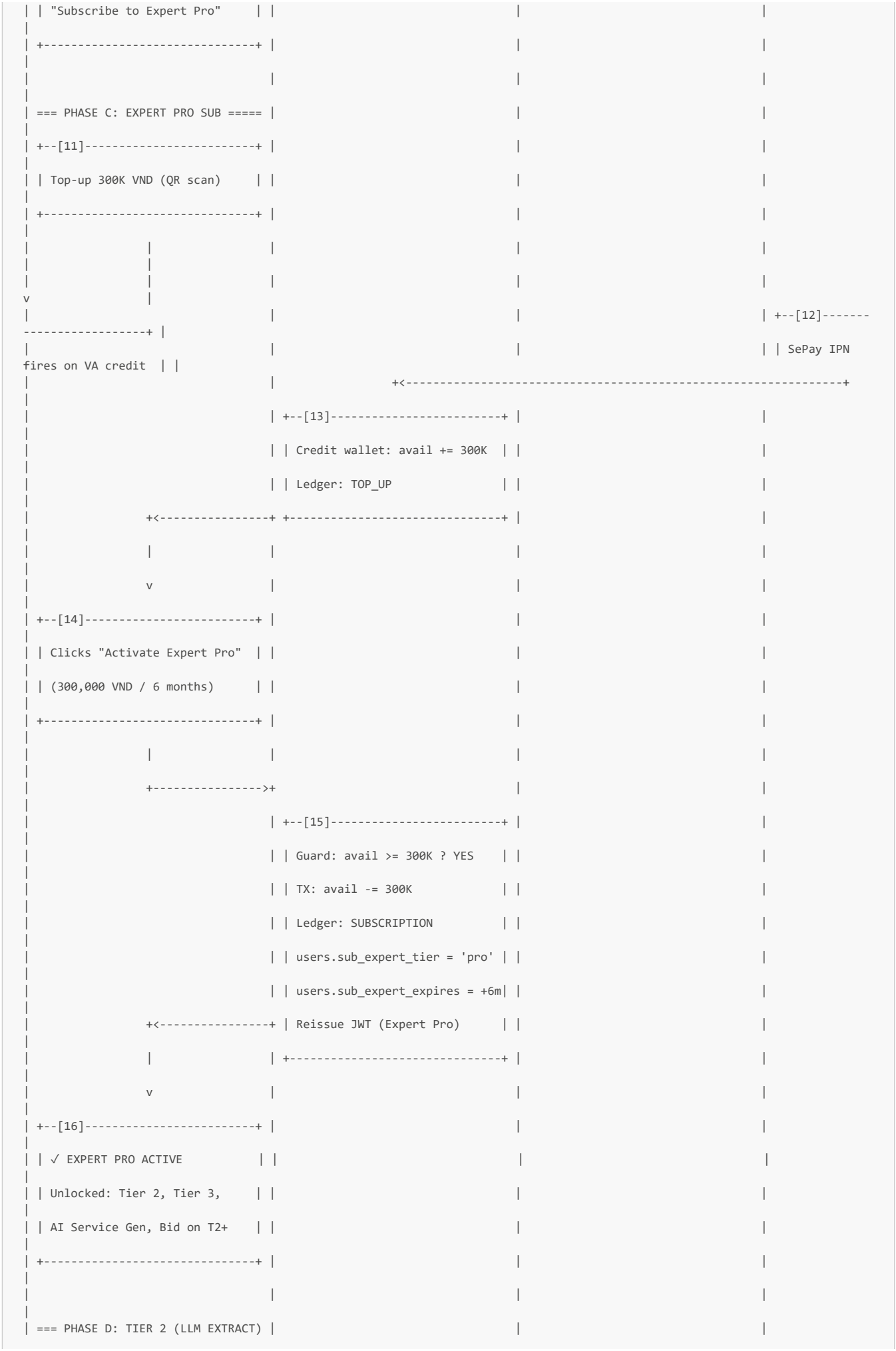
Step	users row	wallets row	wallet_transactions	virtual_accounts	user_subscriptions
[5] Wallet created	-	available=0, locked=0	-	-	-
[7] VA stored	-	-	-	entity_type=WALLET_TOPUP, va_number=XXX, status=ACTIVE	-
[15] Top-up credited	-	available=500,000, locked=0	type=TOP_UP, amount=500K	-	-
[19] Subscription activated	sub_client_tier="pro", sub_client_expires_at=now+6mo	available=0, locked=0	type=SUBSCRIPTION, amount=500K	-	tier=pro, price=500K, expires=now+6mo

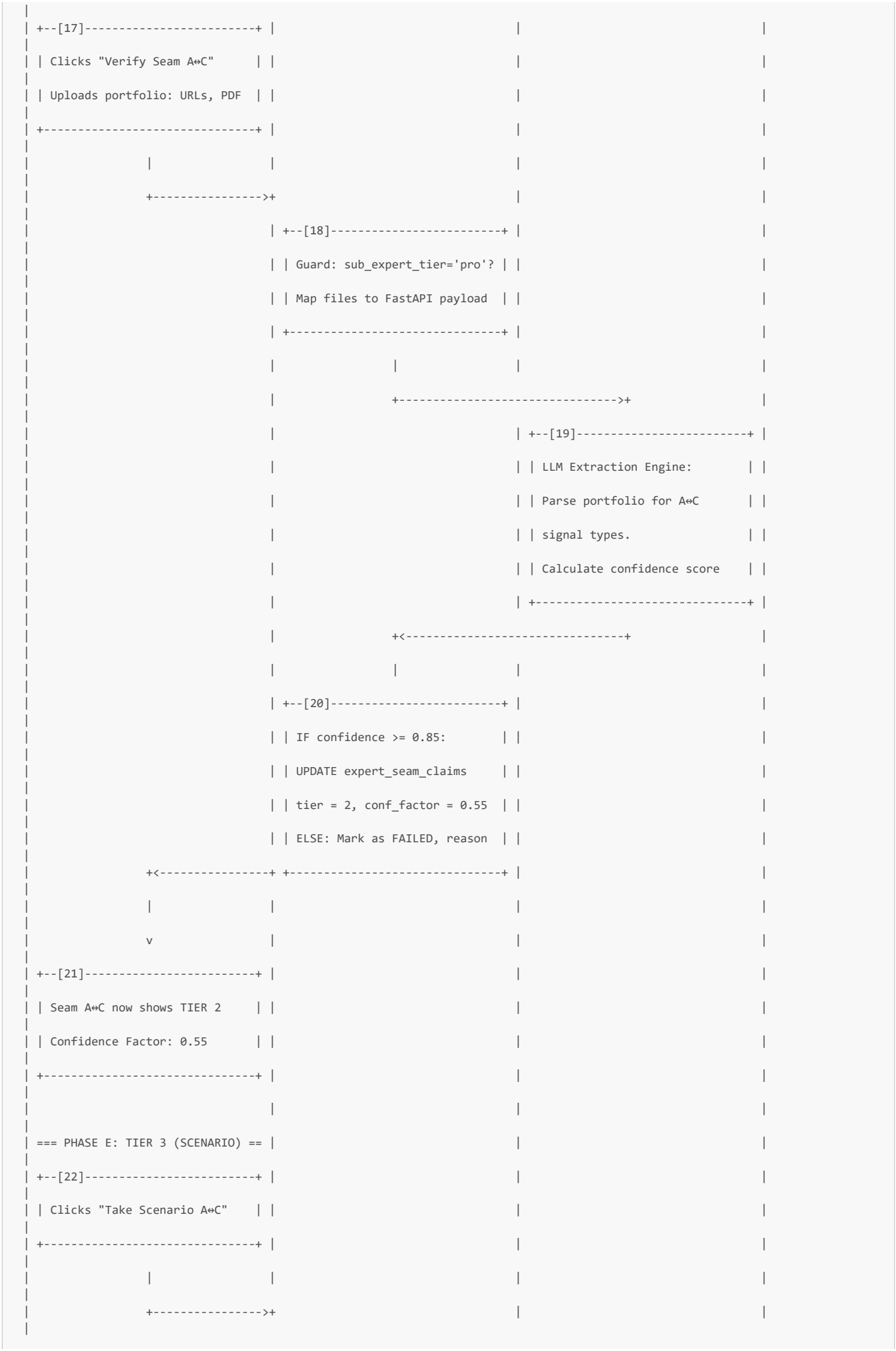
MF-1 is now complete. The CEO holds an active Client Pro subscription and can initiate MF-4 (AI Elicitation Engine). The wallet is at 0 VND - the CEO will need to top up again before funding their first milestone (MF-7, Step AWAITING_PAYMENT → FUNDED).

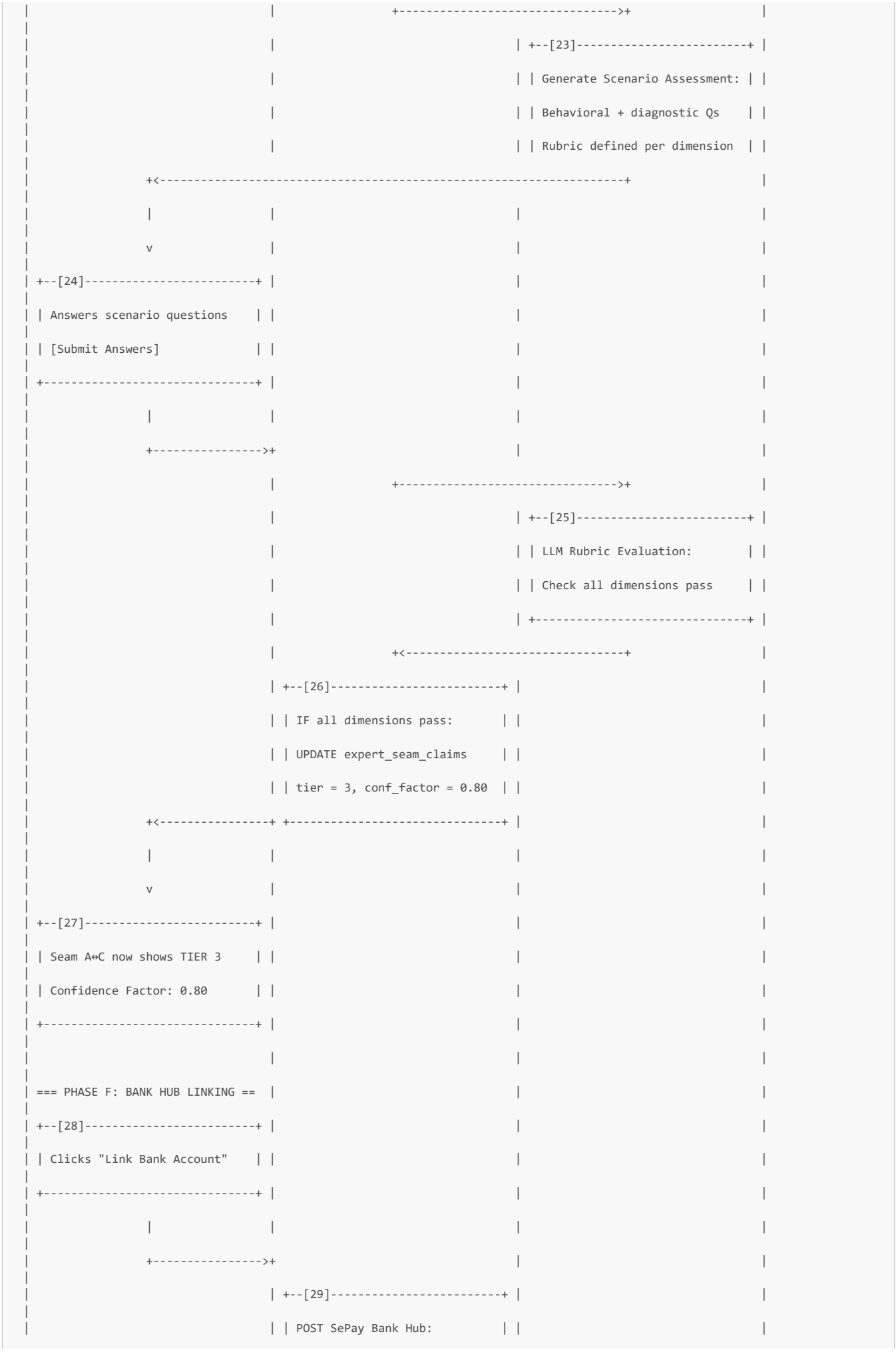
MF-2: Expert Registration, Profile & Verification

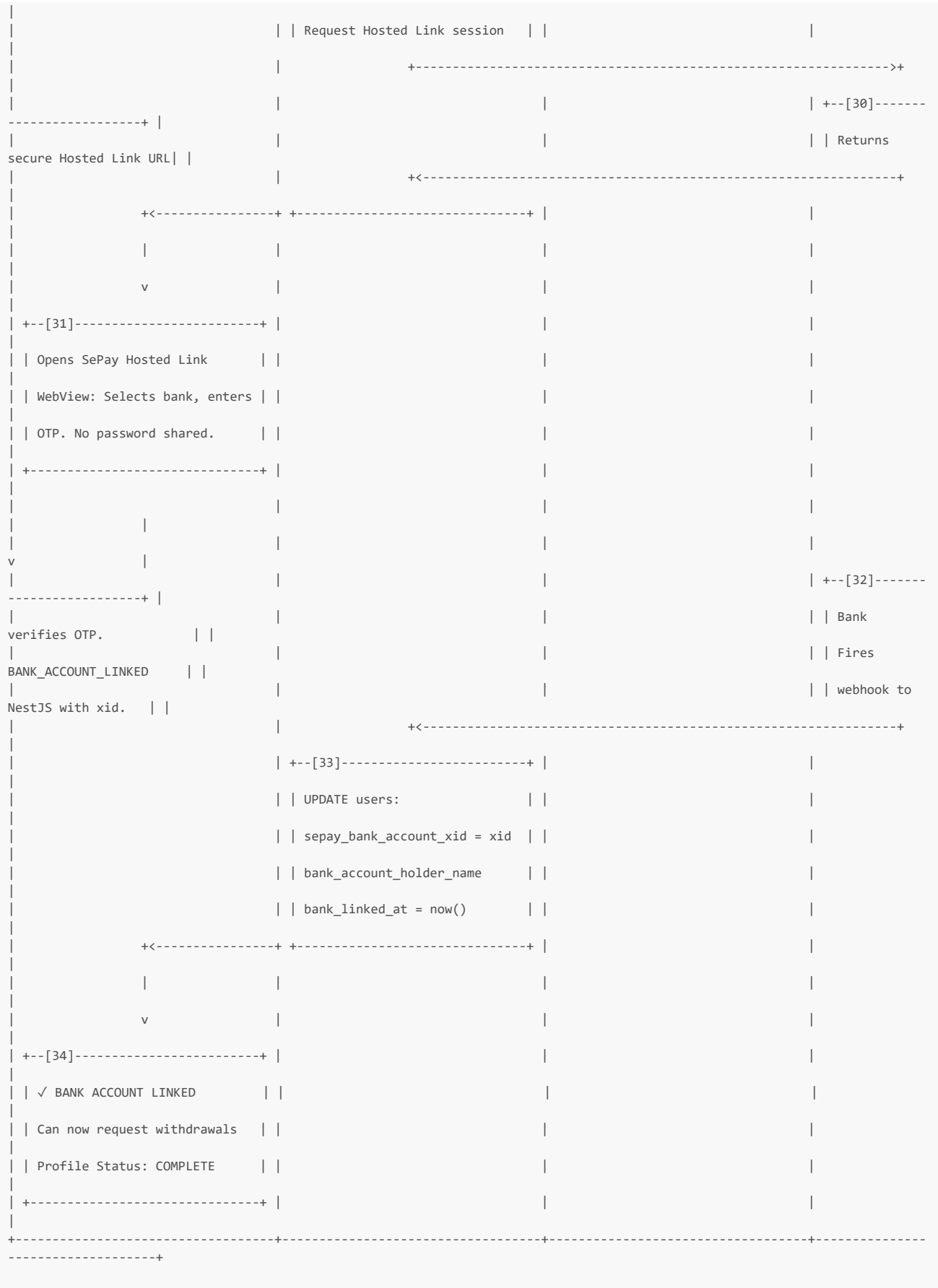
Detailed Swimlane Flow


Create WALLET_TOPUP VA via
SePay API
INSERT virtual_accounts
Generate JWT:
active_role: "EXPERT"
client_subtype: null
Expert Dashboard (Free Tier)
CTA: "Complete Profile"
=== PHASE B: TIER 1 PROFILE =====
Selects Domain Depths:
[A] Deep, [B] Working, etc.
Claims Seams (Tier 1):
Checks: A⇔C, A⇔D, D⇔E...
[Save Profile]
INSERT expert_profiles:
domain_depth JSONB
INSERT expert_seam_claims:
seam_id, tier=1, conf=0.20
Profile status: INCOMPLETE
Tiers 2 & 3 buttons LOCKED


"Subscribe to Expert Pro"
=== PHASE C: EXPERT PRO SUB =====
Top-up 300K VND (QR scan)
SePay IPN fires on VA credit
Credit wallet: avail += 300K
Ledger: TOP_UP
Clicks "Activate Expert Pro"
(300,000 VND / 6 months)
Guard: avail >= 300K ? YES
TX: avail -= 300K
Ledger: SUBSCRIPTION
users.sub_expert_tier = 'pro'
users.sub_expert_expires = +6m
Reissue JWT (Expert Pro)
✓ EXPERT PRO ACTIVE
Unlocked: Tier 2, Tier 3,
AI Service Gen, Bid on T2+
=== PHASE D: TIER 2 (LLM EXTRACT)


+--[17]-------------------------+
| Clicks "Verify Seam A⇔C" |
| Uploads portfolio: URLs, PDF |
+-------------------------------+
+----------------->+
+--[18]-------------------------+
| Guard: sub_expert_tier='pro'? |
| Map files to FastAPI payload |
+-------------------------------+
+---------------------------------->+
+--[19]-------------------------+
| LLM Extraction Engine: |
| Parse portfolio for A⇔C |
| signal types. |
| Calculate confidence score |
+-------------------------------+
+<----------------------------------+
+--[20]-------------------------+
| IF confidence >= 0.85: |
| UPDATE expert_seam_claims |
| tier = 2, conf_factor = 0.55 |
| ELSE: Mark as FAILED, reason |
+<-----------------+ +-------------------------------+
v
+--[21]-------------------------+
| Seam A⇔C now shows TIER 2 |
| Confidence Factor: 0.55 |
+-------------------------------+
=== PHASE E: TIER 3 (SCENARIO) ==
+--[22]-------------------------+
| Clicks "Take Scenario A⇔C" |
+-------------------------------+
+----------------->+


+--[24]
Answers scenario questions
[Submit Answers]
+--[23]
Generate Scenario Assessment:
Behavioral + diagnostic Qs
Rubric defined per dimension
+--[25]
LLM Rubric Evaluation:
Check all dimensions pass
+--[26]
IF all dimensions pass:
UPDATE expert_seam_claims
tier = 3, conf_factor = 0.80
+--[27]
Seam A⇔C now shows TIER 3
Confidence Factor: 0.80
=== PHASE F: BANK HUB LINKING ==
+--[28]
Clicks "Link Bank Account"
+--[29]
POST SePay Bank Hub:



Step-by-Step Narration

PHASE A - EXPERT REGISTRATION & WALLET SETUP

[1] - Opens Registration Page, Selects "I provide AI services"

- **Actor:** Expert (unauthenticated user)
- **Input:** User navigates to `/register`. Clicks the "I provide AI services" path selector.
- **Logic:** The path selector determines the `roles` array and `active_role` for the new account. Selecting "I provide AI services" sets the registration context to `EXPERT`.
- **Output:** The registration form renders with EXPERT-specific copy (emphasizing portfolio, skills, and earning potential).
- **State Change:** None - UI-only step.

---

[2] - Fills Registration Form

- **Actor:** Expert
- **Input (form fields):**

Field	Type	Constraints	Purpose
`email`	string, required	Valid email format; unique in `users`	Login identifier + communication
`password`	string, required	≥ 8 characters	Authentication credential
`full_name`	string, required	Non-empty	Display name on profile, bids, contracts
`phone`	string, required	Valid VN phone format	SePay VA reference; future 2FA

- **Logic:** Identical client-side validation as MF-1 Step [2].
- **Output:** `POST /auth/register` with payload `{ email, password, full_name, phone, registration_path: "EXPERT" }`.
- **State Change:** None yet.

---

[3] - Validate & Insert User Record

- **Actor:** System (NestJS)
- **Input:** Registration payload from [2].
- **Logic:** Server-side validation (email uniqueness, password length, phone format). If valid, hash password and insert into `users` table with Expert-specific fields:

Field	Value Set	Rationale
`roles`	`["EXPERT"]`	Single role at creation; can add CLIENT_CEO later via Account Settings
`active_role`	`"EXPERT"`	Current JWT context
`client_subtype`	`NULL`	Not a CLIENT; only applies when active_role = CLIENT
`subscription_expert_tier`	`"free"`	Default - Tier 2/3 verification, AI Service Generator, and Tier 2+ bidding are gated
`sepay_bank_account_xid`	`NULL`	Not yet linked; required before withdrawal (MF-12)

- **Output:** `user_id` generated.
- **State Change:** `users` table gains one row. Expert exists in system with free tier.
- **Tables Affected:** `users`

---

[4] - Create Wallet & Virtual Account

- **Actor:** System (NestJS + SePay API)
- **Input:** `user_id` from [3].
- **Logic:** Identical to MF-1 Steps [5]–[7].
    1. Create `wallets` row (`available_balance = 0`, `locked_balance = 0`).
    2. Call SePay Bank Hub API to create a permanent `WALLET_TOPUP` virtual account.
    3. Store mapping in `virtual_accounts` (`entity_type = "WALLET_TOPUP"`, `entity_id = user_id`, `fixed_amount = NULL`, `expires_at = NULL`, `status = "ACTIVE"`).
- **Output:** Wallet and VA ready for top-up.
- **State Change:** `wallets` and `virtual_accounts` each gain one row.
- **Tables Affected:** `wallets`, `virtual_accounts`

---

[5] - Generate JWT

- **Actor:** System (NestJS)
- **Input:** `user_id`, role data from [3].
- **Logic:** Construct JWT payload per F1 specification:

```
{
  "sub": "user_id",
  "active_role": "EXPERT",
  "client_subtype": null,
  "roles": ["EXPERT"],
```

```
  "self_technical_projects": [],
  "project_ids": []
}
```

- **Output:** JWT set as HTTP-only cookie.
- **State Change:** None - token generation is stateless.

---

[6] - Expert Dashboard Loads (Free Tier)

- **Actor:** Expert (authenticated)
- **Input:** JWT from [5].
- **Logic:** Dashboard renders in free-tier mode. Expert can create a profile (Tier 1 claims only) and browse available public info, but all LLM-powered features (Tier 2/3 verification, AI Service Generator, bidding on Tier 2+ projects) return `403 SUBSCRIPTION_REQUIRED`. A CTA prompts "Complete Profile" and "Subscribe to Expert Pro".
- **Output:** Expert Dashboard (free tier).
- **State Change:** None.

PHASE B - TIER 1 PROFILE & SEAM CLAIMS

---

[7] - Selects Domain Depths

- **Actor:** Expert
- **Input:** Expert navigates to Profile Builder. The UI displays the 6 Capability Domains (Section 0.1) as selectable cards. For each domain, the expert selects a depth level:

Depth Level	Meaning
Deep	Primary expertise; has production experience; can architect solutions
Working	Competent; has used in real projects; can implement under guidance
Aware	Conceptual understanding; knows when to escalate; cannot implement independently

Example selection:

Domain	Code	Selected Depth
LLM Application Engineering	A	Deep
MLOps / LLMOps Infrastructure	B	Working
AI Evaluation & Quality Systems	C	Deep
Vector DB & Embeddings	D	Aware
Data & Pipeline Engineering	E	Working
ML Modeling & Fine-Tuning	F	Aware

- **Logic:** The system enforces that at least one domain must be `Deep` or `Working` to create a viable profile - an expert with all `Aware` domains cannot meaningfully bid.
- **Output:** Domain depth selection stored in component state, ready for submission.
- **State Change:** None yet.

---

[8] - Claims Seams (Tier 1)

- **Actor:** Expert
- **Input:** Based on the domain depths selected in [7], the UI surfaces the relevant 10 Seams (Section 0.2) as claimable checkboxes. An expert can only claim seams where they hold at least `Working` depth in BOTH adjacent domains. For example, to claim **A↔C**, the expert must have `Working` or `Deep` in both A and C.

Example claimable seams based on [7] selection:

- **A↔C** (Deep A + Deep C) - Claimable
- **A↔B** (Deep A + Working B) - Claimable
- **D↔E** (Aware D + Working E) - NOT claimable (D is only Aware)
- **D↔F** (Aware D + Aware F) - NOT claimable (both Aware)
- **Logic:** Expert checks the seams they wish to claim. These are **self-declared** - no evidence is required at this tier. The expert clicks `[Save Profile]`.
- **Output:** `POST /expert/profile` with payload `{ domain_depth: {...}, claimed_seams: ["A↔C", "A↔B"] }`.

- **State Change:** None yet.

---

[9] - Store Profile & Tier 1 Seam Claims

- **Actor:** System (NestJS)
- **Input:** Profile payload from [8], `user_id` from JWT.
- **Logic:**
  1. **Upsert `expert_profiles`:**
     - `user_id` = from JWT
     - `domain_depth` = JSONB `{"A": "deep", "B": "working", "C": "deep", "D": "aware", "E": "working", "F": "aware"}`
     - `profile_status` = `INCOMPLETE` (subscription and bank link missing)
  2. **Insert `expert_seam_claims`** - one row per claimed seam:

Field	Value Set	Rationale
`user_id`	from JWT	FK to users
`seam_id`	`"A⇔C"`, `"A⇔B"`	From Section 0.2
`verification_tier`	`1`	Self-declared (Claimed)
`confidence_factor`	`0.20`	Per Section 0.4 - Tier 1 confidence factor
`evidence_payload`	`NULL`	No evidence at Tier 1
`verified_at`	`now()`	Tier 1 is instant upon claim

- **Output:** Profile and seam claims persisted. Return `200 OK` with current profile state.
- **State Change:** `expert_profiles` gains/updates one row. `expert_seam_claims` gains N rows (one per claimed seam). Profile is now at Tier 1 but `INCOMPLETE`.
- **Tables Affected:** `expert_profiles` (UPSERT), `expert_seam_claims` (INSERT)

---

[10] - Tiers 2 & 3 Locked, Subscription Prompt

- **Actor:** Expert
- **Input:** Profile saved confirmation from [9].
- **Logic:** The UI updates to show claimed seams at **Tier 1** with a `0.20` confidence badge. The "Verify with Portfolio (Tier 2)" and "Take Scenario Assessment (Tier 3)" buttons are visible but **disabled/locked** with a tooltip: "*Subscribe to Expert Pro to unlock advanced verification.*"
- **Output:** Profile page displays Tier 1 status, prompts Expert Pro subscription.
- **State Change:** None.

PHASE C - EXPERT PRO SUBSCRIPTION

---

[11] - Top-Up 300,000 VND via VietQR

- **Actor:** Expert
- **Input:** Expert clicks "Top Up Wallet" and scans their permanent `WALLET_TOPUP` VietQR with their banking app. They enter 300,000 VND and confirm.
- **Logic:** Identical to MF-1 Steps [10]–[12]. The transfer occurs outside the platform via the user's banking app.
- **Output:** Bank transfer of 300,000 VND initiated to platform's master account under the Expert's VA.
- **State Change:** None on platform side yet.

---

[12] - SePay IPN Fires on VA Credit

- **Actor:** SePay
- **Input:** Credit detected on Expert's `WALLET_TOPUP` VA.
- **Logic:** SePay sends `POST /webhook/sepay` with `{ va_number, amount: 300000, transfer_reference, transfer_type: "in" }`.
- **Output:** IPN payload delivered to NestJS.
- **State Change:** External - credit recorded on SePay-side VA.

---

[13] - Credit Wallet

- **Actor:** System (NestJS)
- **Input:** IPN payload from [12].
- **Logic:** Identical to MF-1 Step [15]. Atomic transaction:
  1. Verify IPN signature.
  2. Resolve `va_number` → `user_id` via `virtual_accounts`.
  3. Idempotency check on `transfer_reference`.
  4. `UPDATE wallets SET available_balance = available_balance + 300000 WHERE user_id = ?`

5. `INSERT INTO wallet_transactions (type: "TOP_UP", amount: 300000, reference: transfer_reference)`
6. Return 200 OK to SePay.

- **Output:** Wallet credited. Expert notified.
- **State Change:** `wallets.available_balance`: `0` → `300,000`. `wallet_transactions` gains one row.
- **Tables Affected:** `wallets` (UPDATE), `wallet_transactions` (INSERT)

---

[14] - Clicks "Activate Expert Pro"

- **Actor:** Expert
- **Input:** Expert clicks the "Activate Expert Pro" button. The price is displayed: **300,000 VND / 6 months** (per Section 0.9).
- **Logic:** Frontend sends `POST /subscriptions/activate` with `{ role_type: "expert" }`.
- **Output:** Subscription activation request sent.
- **State Change:** None.

---

[15] - Subscription Guard, Balance Guard & Atomic Activation

- **Actor:** System (NestJS)
- **Input:** JWT + `{ role_type: "expert" }` from [14].
- **Logic:**

1. **Subscription Guard:** Check `users.sub_expert_tier`. If `'pro'` and not expired → `409 ALREADY_SUBSCRIBED`.
2. **Balance Guard:** Check `wallets.available_balance >= 300,000`. If insufficient → `422 INSUFFICIENT_BALANCE`.
3. **Atomic DB Transaction:**

```
BEGIN;
-- 1. Debit wallet
UPDATE wallets SET available_balance = available_balance - 300000 WHERE user_id = ?;

-- 2. Ledger entry
INSERT INTO wallet_transactions (wallet_id, amount, transaction_type, reference_id, created_at)
VALUES (?, 300000, 'SUBSCRIPTION', 'SUB-EXPERT-{user_id}', now());

-- 3. Update user subscription fields
UPDATE users
  SET subscription_expert_tier = 'pro',
      sub_expert_expires_at = now() + interval '6 months'
  WHERE id = ?;

-- 4. Audit record
INSERT INTO user_subscriptions (user_id, role_type, tier, price_paid, activated_at, expires_at)
VALUES (?, 'expert', 'pro', 300000, now(), now() + interval '6 months');

COMMIT;
```

- **Output:** Transaction committed. New JWT reissued with updated subscription claims. Notification sent: "*Expert Pro active until {date}*".
- **State Change:**
  - `wallets.available_balance`: `300,000` → `0`
  - `users.subscription_expert_tier`: `"free"` → `"pro"`
  - `users.sub_expert_expires_at`: `NULL` → `now() + 6 months`
- **Tables Affected:** `wallets`, `wallet_transactions`, `users`, `user_subscriptions`

---

[16] - Expert Pro Active

- **Actor:** Expert
- **Input:** New JWT and dashboard refresh.
- **Logic:** Dashboard re-renders with Pro features unlocked:
  - Tier 2 Portfolio Verification
  - Tier 3 Scenario Assessment
  - AI Service Generator
  - Bid on Tier 2+ projects
  - Earnings analytics dashboard (full)
- **Output:** Expert Dashboard in Pro mode. Profile page now shows Tier 2 and Tier 3 buttons as **enabled**.
- **State Change:** None - UI reflects committed state.

PHASE D - TIER 2 VERIFICATION (LLM PORTFOLIO EXTRACTION)

---

[17] - Initiates Tier 2 Verification, Uploads Portfolio

- **Actor:** Expert
- **Input:** Expert clicks "Verify" next to seam **A↔C** (Ground truth-driven iteration). The UI prompts for portfolio evidence. Expert uploads:
  - **GitHub repo URL** (to a project with evaluation pipeline code)
  - **PDF case study** (describing a project where they built ground truth and iterated prompts)
  - **StackOverflow answer URL** (demonstrating conceptual understanding of A↔C boundary)
- **Logic:** Frontend sends `POST /expert/seams/verify` with `FormData { seam_id: "A↔C", target_tier: 2, evidence_files: [...], evidence_urls: [...] }`.
- **Output:** Verification request with attached evidence sent.
- **State Change:** None yet.

---

[18] - Subscription Guard & Payload Mapping

- **Actor:** System (NestJS)
- **Input:** Verification request from [17], JWT.
- **Logic:**
  1. **Subscription Guard:** Verify `users.sub_expert_tier = 'pro'`. If not → `403 SUBSCRIPTION_REQUIRED`.
  2. **Payload Mapping:** NestJS extracts text from PDFs, fetches URL content (with timeout), and constructs a structured payload for FastAPI containing:
     - `seam_id`: `"A↔C"`
     - `domain_context`: Expert's declared depths for A and C
     - `evidence_texts`: Array of extracted text content from all submitted evidence
- **Output:** Structured payload ready for FastAPI.
- **State Change:** None.

---

[19] - LLM Extraction Engine (FastAPI + Claude API)

- **Actor:** AI Service (FastAPI)
- **Input:** Structured evidence payload from [18].
- **Logic:** The FastAPI service runs the Tier 2 verification pipeline:
  1. **Signal Type Detection:** The LLM is prompted to find **all required signal types** for the A↔C seam. Required signals for A↔C include:
     - Evidence of creating or using evaluation datasets (ground truth)
     - Evidence of iterating prompts/system prompts based on evaluation metrics
     - Evidence of diagnosing failures at the boundary of prompt design and evaluation
  2. **Confidence Scoring:** For each found signal, the LLM assigns a confidence. The overall confidence is computed as a weighted combination.
  3. **Threshold Check:** Per Section 0.4, Tier 2 requires `confidence ≥ 0.85` AND all required signal types found.
- **Output:** FastAPI returns `{ seam_id: "A↔C", tier_2_passed: true/false, confidence: 0.89, missing_signals: [] }`.
- **State Change:** None on AI side - stateless computation.

---

[20] - Update Seam Verification (Tier 2 Result)

- **Actor:** System (NestJS)
- **Input:** FastAPI response from [19].
- **Logic:**
  - **If `tier_2_passed = true` (confidence ≥ 0.85):**
    - `UPDATE expert_seam_claims SET verification_tier = 2, confidence_factor = 0.55, evidence_payload = {fastapi_response}, verified_at = now() WHERE user_id = ? AND seam_id = 'A↔C'`
    - Notify expert: "*Seam A↔C verified at Tier 2 (Evidence-backed). Your confidence factor is now 0.55.*"
  - **If `tier_2_passed = false`:**
    - `UPDATE expert_seam_claims SET verification_tier = 1, confidence_factor = 0.20, evidence_payload = {fastapi_response_with_missing_signals} WHERE user_id = ? AND seam_id = 'A↔C'`
    - Notify expert: "*Tier 2 verification for A↔C failed. Missing signals: [list]. Your seam remains at Tier 1.*"
    - Expert can re-attempt with new evidence (no limit on attempts documented, but practically rate-limited).
- **Output:** DB updated. Expert notified of result.
- **State Change:** `expert_seam_claims` row for A↔C updated to `verification_tier = 2`, `confidence_factor = 0.55` (if passed).
- **Tables Affected:** `expert_seam_claims` (UPDATE)

---

[21] - Seam Displays Tier 2 Status

- **Actor:** Expert
- **Input:** Notification and UI refresh from [20].
- **Logic:** The Profile page updates the A↔C seam badge from "Tier 1 (0.20)" to "Tier 2 - Evidence-backed (0.55)". The "Take Scenario Assessment (Tier 3)" button becomes active for this seam.
- **Output:** Updated profile UI.
- **State Change:** None.

PHASE E - TIER 3 VERIFICATION (SCENARIO ASSESSMENT)

---

[22] - Initiates Tier 3 Scenario Assessment

- **Actor:** Expert
- **Input:** Expert clicks "Take Scenario Assessment" for seam A↔C.
- **Logic:** Frontend sends `POST /expert/seams/scenario` with `{ seam_id: "A↔C", target_tier: 3 }`.
- **Output:** Scenario assessment request sent.
- **State Change:** None.

---

[23] - Generate Scenario Assessment (FastAPI + Claude API)

- **Actor:** AI Service (FastAPI)
- **Input:** Request from NestJS (forwarded from [22]).
- **Logic:** The FastAPI service generates a dynamic scenario assessment:
  1. **Scenario Context:** Generates a realistic project scenario where seam A↔C is the load-bearing failure point. For example: *"A client has an LLM-based content moderation system. The prompt reduces harmful content by 40%, but the client cannot tell if changes to the prompt help or hurt because there is no evaluation dataset. The client wants to 'just fine-tune it.'"*
  2. **Diagnostic Questions:** Creates 3–5 behavioral/diagnostic questions that test whether the expert can:
     - Recognize the ground truth void (evaluation baseline missing)
     - Propose the correct intervention (build evaluation dataset first, then iterate prompt)
     - Resist the client's prescribed but wrong solution (fine-tuning before establishing baseline)
  3. **Rubric Definition:** Each question has a specific rubric defining what a passing answer contains. All dimensions must individually pass.
- **Output:** FastAPI returns `{ scenario_text, questions: [...], rubric: [...] }` to NestJS, which forwards it to the Expert's frontend.
- **State Change:** None - generated on-the-fly, stateless.

---

[24] - Answers Scenario Questions

- **Actor:** Expert
- **Input:** Scenario assessment UI displaying the scenario context and questions.
- **Logic:** Expert reads the scenario and types free-form answers to each diagnostic question. The interface warns: "*Your answers will be evaluated against a rubric. All dimensions must pass for Tier 3 verification.*" Expert clicks `[Submit Answers]`.
- **Output:** `POST /expert/seams/scenario/submit` with `{ seam_id: "A↔C", answers: [...] }`.
- **State Change:** None.

---

[25] - LLM Rubric Evaluation (FastAPI + Claude API)

- **Actor:** AI Service (FastAPI)
- **Input:** Expert's answers from [24], rubric from [23].
- **Logic:** The FastAPI service evaluates each answer against its corresponding rubric dimension:
  1. **Per-question evaluation:** LLM compares the expert's answer to the rubric criteria. Each question is scored as `PASS` or `FAIL` with a justification.
  2. **Overall assessment:** Per Section 0.4, Tier 3 requires **all required dimensions individually pass the per-question rubric**. A single dimension failure means the assessment fails.
  3. **Feedback generation:** For any failed dimension, the LLM generates specific feedback explaining what was expected and what was missing.
- **Output:** FastAPI returns `{ seam_id: "A↔C", tier_3_passed: true/false, dimension_results: [{question_id, passed, feedback}], overall_feedback: "..." }`.
- **State Change:** None.

---

[26] - Update Seam Verification (Tier 3 Result)

- **Actor:** System (NestJS)
- **Input:** FastAPI response from [25].
- **Logic:**
  - **If `tier_3_passed = true` (all dimensions pass):**
    - `UPDATE expert_seam_claims SET verification_tier = 3, confidence_factor = 0.80, evidence_payload = {fastapi_response}, verified_at = now() WHERE user_id = ? AND seam_id = 'A↔C'`
    - Notify expert: "*Seam A↔C verified at Tier 3 (Scenario-verified). Your confidence factor is now 0.80.*"
  - **If `tier_3_passed = false`:**
    - Seam remains at Tier 2. Expert receives dimension-by-dimension feedback and can re-attempt after a cooldown (rate-limited to prevent brute-forcing the rubric).
- **Output:** DB updated. Expert notified.
- **State Change:** `expert_seam_claims` row for A↔C updated to `verification_tier = 3`, `confidence_factor = 0.80` (if passed).
- **Tables Affected:** `expert_seam_claims` (UPDATE)

---

[27] - Seam Displays Tier 3 Status

- **Actor:** Expert
- **Input:** Notification and UI refresh from [26].
- **Logic:** The Profile page updates the A↔C seam badge from "Tier 2 (0.55)" to "Tier 3 - Scenario-verified (0.80)". This is the highest verification tier achievable before real engagement (Tier 4 is platform-evaluated post-engagement, per Section 0.4).

- **Output:** Updated profile UI.
- **State Change:** None.

PHASE F - BANK ACCOUNT LINKING (SEPAY BANK HUB HOSTED LINK)

---

[28] - Clicks "Link Bank Account"

- **Actor:** Expert
- **Input:** Expert navigates to Profile → Earnings / Withdrawal section and clicks "Link Bank Account for Withdrawals".
- **Logic:** This step is required before the expert can request any withdrawal (MF-12). Per Section 0.8, the `sepay_bank_account_xid` must be set via the Bank Hub Hosted Link process - the platform never asks for or stores bank passwords.
- **Output:** `POST /expert/bank-link/initiate` request sent.
- **State Change:** None.

---

[29] - Request Bank Hub Hosted Link Session

- **Actor:** System (NestJS)
- **Input:** Request from [28], JWT identifying the expert.
- **Logic:** NestJS calls the SePay Bank Hub API to generate a secure, one-time Hosted Link session for this user. The request includes:
  - `user_reference`: `user_id` (to correlate the webhook callback)
  - `redirect_url`: The URL the expert will be redirected to after completing the link
- **Output:** SePay API call prepared.
- **State Change:** None.

---

[30] - SePay Returns Hosted Link URL

- **Actor:** SePay (Bank Hub API)
- **Input:** Session creation request from [29].
- **Logic:** SePay generates a secure, time-limited URL for its Hosted Link WebView. This URL is specific to this session and this user.
- **Output:** `{ hosted_link_url: "https://bankhub.sepay.vn/link/xxxxx", expires_in: 3600 }`.
- **State Change:** External - SePay creates a pending link session.

---

[31] - Expert Opens Hosted Link, Verifies via OTP

- **Actor:** Expert (via SePay WebView)
- **Input:** NestJS returns the `hosted_link_url` to the frontend. The frontend either redirects the expert or opens the URL in an iframe/modal.
- **Logic:** The expert interacts with SePay's secure interface:
  1. Selects their bank from a list of supported Vietnamese banks.
  2. Enters their bank account number and account holder name.
  3. SePay sends an OTP to the phone number registered with the bank.
  4. Expert enters OTP to verify ownership.
  5. **No bank password is ever entered** - this is the key security feature of Bank Hub Hosted Link. The OTP proves account ownership without credential sharing.
- **Output:** Expert completes the OTP verification in the WebView. SePay processes the link.
- **State Change:** External - SePay establishes the link between the expert's bank account and the platform's Bank Hub.

---

[32] - Bank Verifies OTP, Fires Webhook

- **Actor:** SePay (automated webhook)
- **Input:** OTP verification success from [31].
- **Logic:** Upon successful OTP verification, SePay:
  1. Generates a unique `bank_account_xid` - a tokenized identifier for the expert's bank account. This xid is used for all future chi hộ (disbursement) API calls. The platform never sees the actual bank account number after this point.
  2. Fires a `BANK_ACCOUNT_LINKED` webhook to NestJS.
- **Output (Webhook payload):**

```
{
  "event": "BANK_ACCOUNT_LINKED",
  "user_reference": "user_id",
  "bank_account_xid": "bax_abc123xyz",
  "bank_account_holder_name": "NGUYEN VAN A",
  "bank_name": "Vietcombank",
  "timestamp": "2026-05-15T11:00:00Z"
}
```

- **State Change:** External - SePay records the bank account link.

---

[33] - Store Bank Account Link

- **Actor:** System (NestJS)
- **Input:** `BANK_ACCOUNT_LINKED` webhook payload from [32].
- **Logic:**
  1. **Verify webhook signature** (using SePay API key).
  2. **Resolve user:** Find user by `user_reference` (which was set to `user_id` in [29]).
  3. **Update user record:**

```
UPDATE users
  SET sepay_bank_account_xid = 'bax_abc123xyz',
      bank_account_holder_name = 'NGUYEN VAN A',
      bank_linked_at = now()
  WHERE id = ?;
```

  4. **Update profile status:** If all other required fields are complete, set `expert_profiles.profile_status = 'COMPLETE'`.
  5. **Notify expert:** "*Your bank account has been successfully linked. You can now request withdrawals.*"
- **Output:** DB updated. Expert notified.
- **State Change:** `users` row updated with `sepay_bank_account_xid`, `bank_account_holder_name`, `bank_linked_at`. Profile status potentially updated to `COMPLETE`.
- **Tables Affected:** `users` (UPDATE), `expert_profiles` (UPDATE)

---

[34] - Bank Account Linked, Profile Complete

- **Actor:** Expert
- **Input:** Notification and UI refresh from [33].
- **Logic:** The Expert Dashboard updates to show:
  - **Bank Account Linked** (with verified holder name and bank name displayed)
  - **Withdrawal capability unlocked** (can request chi hộ disbursement)
  - **Profile Status: COMPLETE** (all required steps finished: profile created, subscription active, bank linked)
  - The expert is now fully onboarded and can:
    - Bid on projects (MF-6) with seam-verified confidence factors
    - Publish service listings (MF-9)
    - Receive milestone payments (MF-7)
    - Request withdrawals (MF-12)
- **Output:** Complete Expert Dashboard.
- **State Change:** None - UI reflects committed state.

Summary: End-to-End State Trace for MF-2

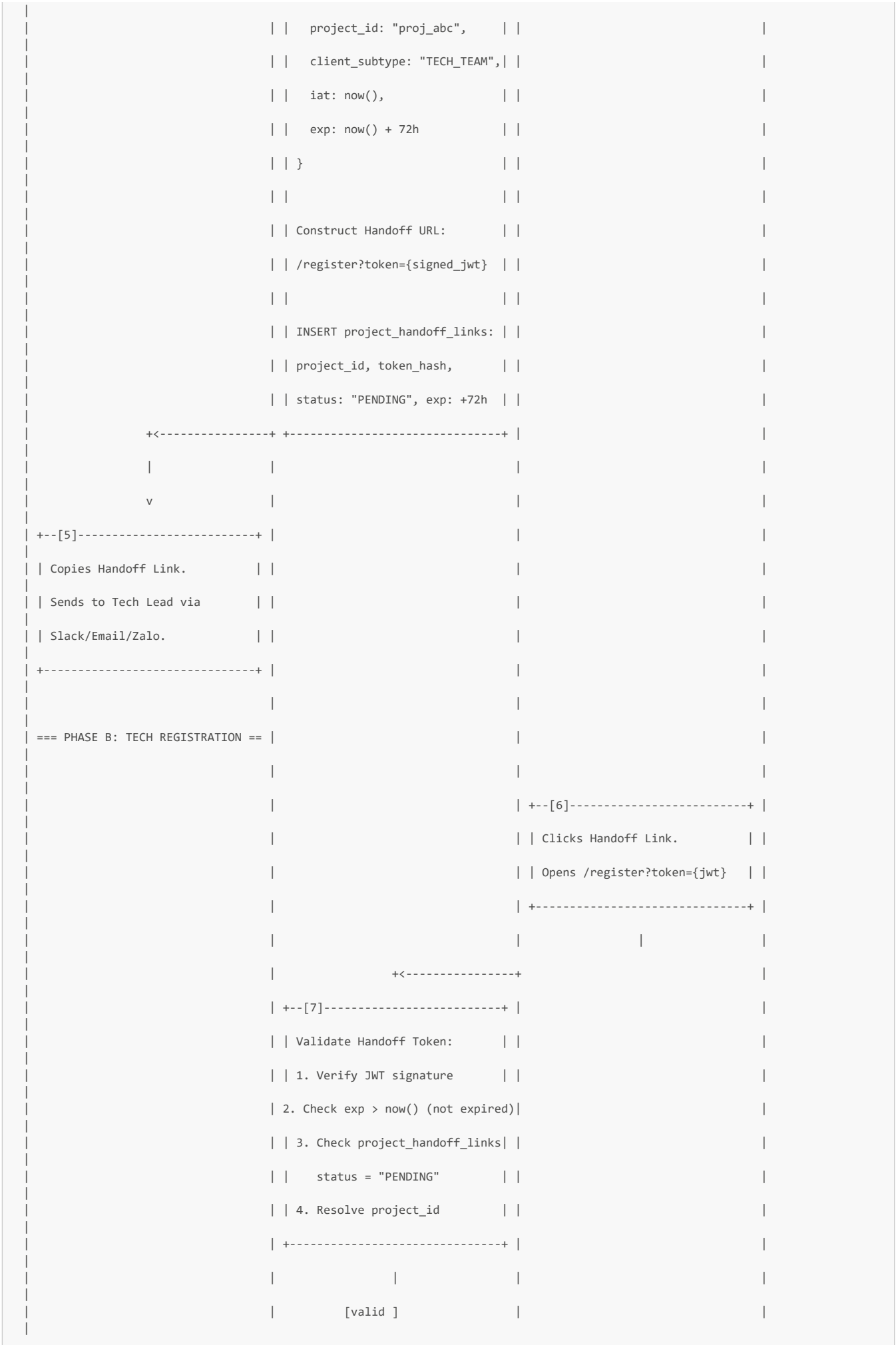
Step	`users`	`wallets` / `wallet_transactions`	`expert_profiles`	`expert_seam_claims`	`virtual_accounts`	`user_subsc`
[3] Registration	roles=["EXPERT"], active_role="EXPERT", sub_expert_tier="free", sepay_bank_account_xid=NULL	-	-	-	-	-
[4] Wallet + VA	-	wallets: avail=0, locked=0	-	-	WALLET_TOPUP VA created	-
[9] Tier 1 Profile	-	-	domain_depth JSONB, status=INCOMPLETE	seam A↔C: tier=1, conf=0.20	-	-
[13] Top-up	-	avail=300K, ledger: TOP_UP	-	-	-	-
[15] Expert Pro	sub_expert_tier="pro", sub_expert_expires_at=now+6mo	avail=0, ledger: SUBSCRIPTION	-	-	-	tier=pro, pr
[20] Tier 2	-	-	-	seam A↔C: tier=2, conf=0.55	-	-
[26] Tier 3	-	-	-	seam A↔C: tier=3, conf=0.80	-	-

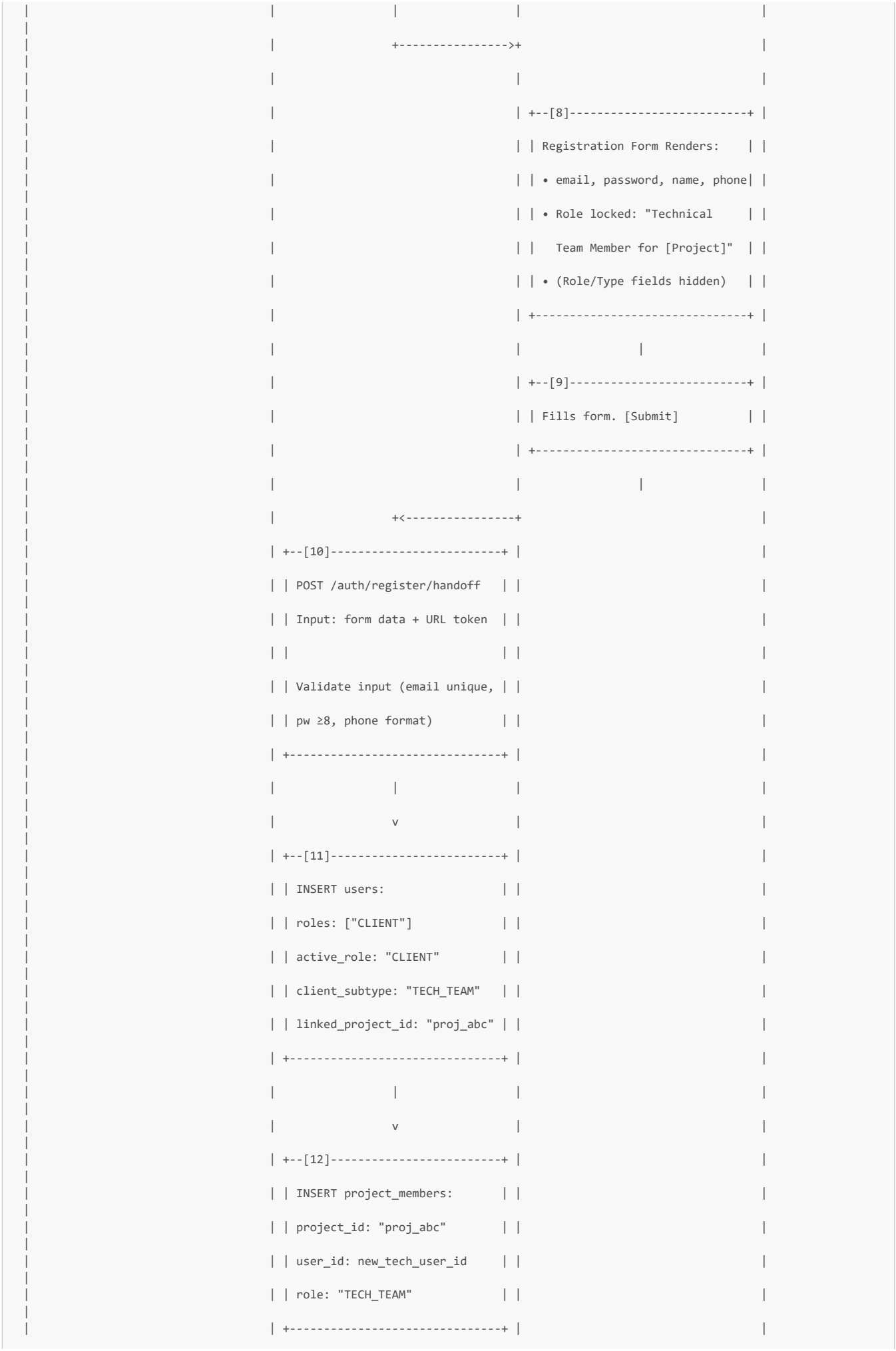
Step	users	wallets / wallet_transactions	expert_profiles	expert_seam_claims	virtual_accounts	user_subsc
[33] Bank Link	sepay_bank_account_xid="bax_...", bank_linked_at=now	-	status=COMPLETE	-	-	-

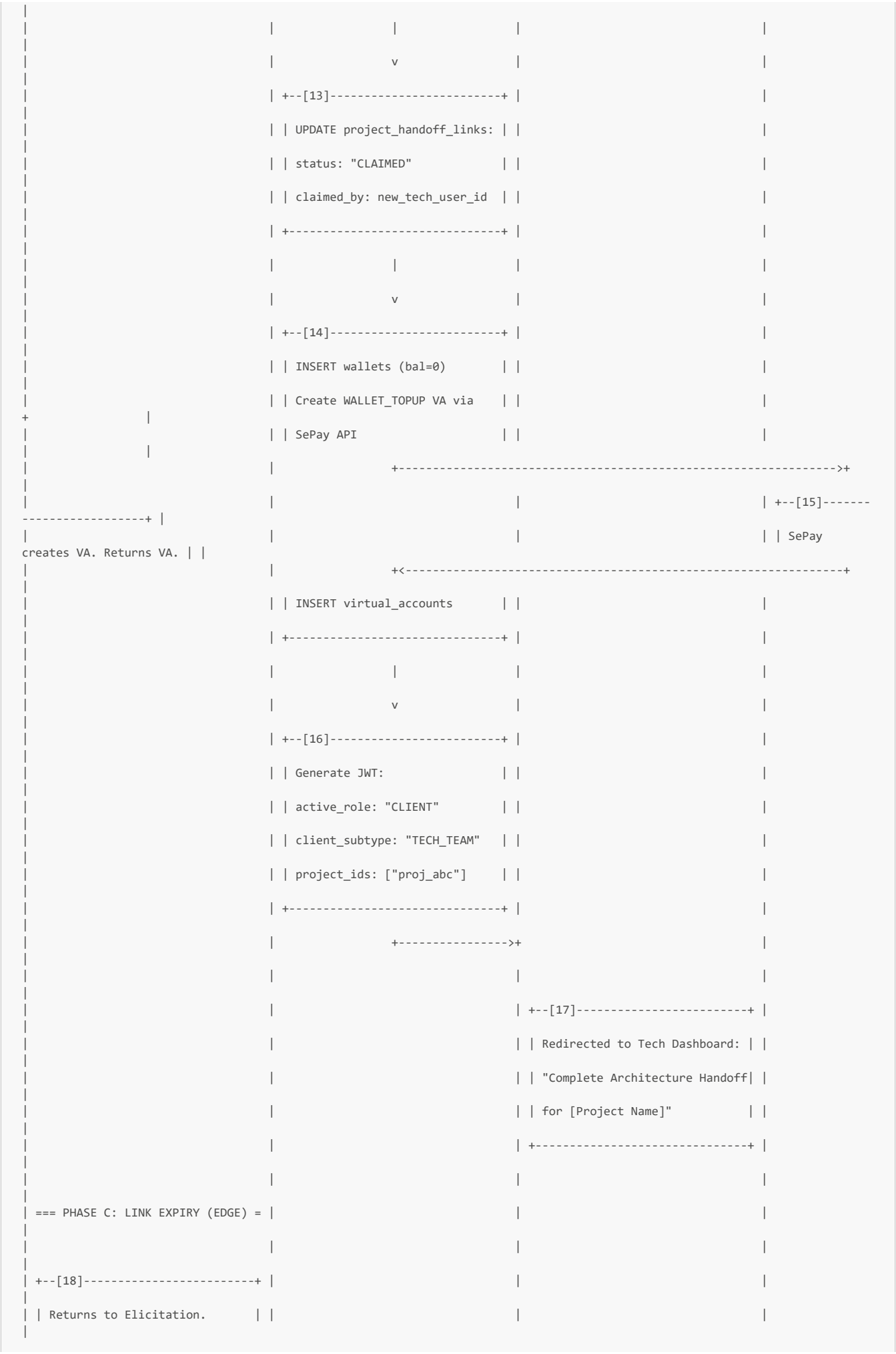
MF-2 is now complete. The Expert holds an active Expert Pro subscription, has verified seams with confidence factors up to 0.80, has a linked bank account for chi hộ disbursement, and is ready to participate in matching (MF-5), bidding (MF-6), and service publishing (MF-9).

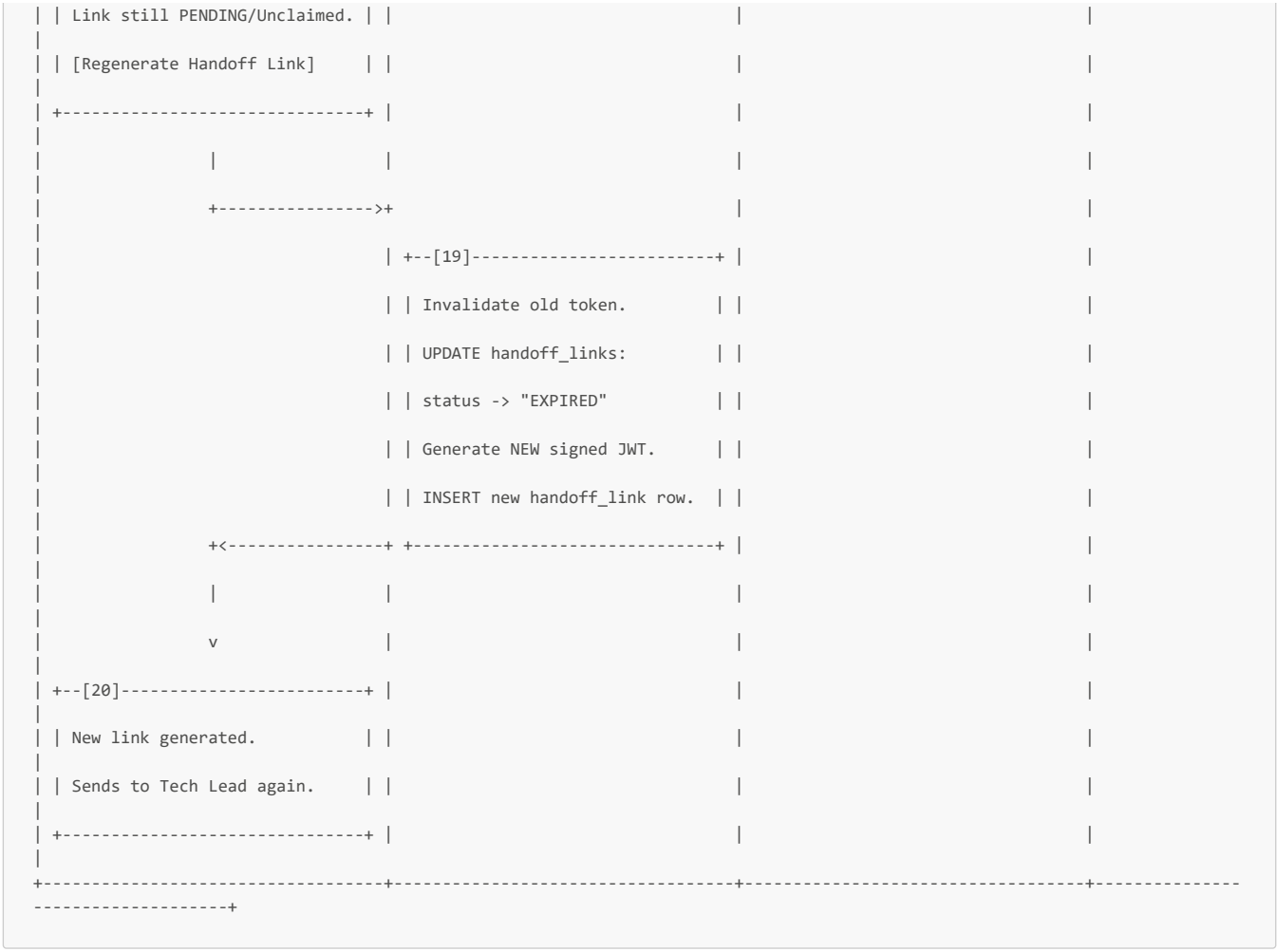
MF-3: Tech Team Account Creation via Handoff Link

Detailed Swimlane Flow


project_id: "proj_abc",
client_subtype: "TECH_TEAM",
iat: now(),
exp: now() + 72h
}
Construct Handoff URL:
/register?token={signed_jwt}
INSERT project_handoff_links:
project_id, token_hash,
status: "PENDING", exp: +72h
[5]
Copies Handoff Link.
Sends to Tech Lead via
Slack/Email/Zalo.
=== PHASE B: TECH REGISTRATION ==
[6]
Clicks Handoff Link.
Opens /register?token={jwt}
[7]
Validate Handoff Token:
1. Verify JWT signature
2. Check exp > now() (not expired)
3. Check project_handoff_links
status = "PENDING"
4. Resolve project_id
[valid]


+----------------->+
+--[8]-------------------------+
| Registration Form Renders: |
| • email, password, name, phone|
| • Role locked: "Technical |
| Team Member for [Project]" |
| • (Role/Type fields hidden) |
+--[9]-------------------------+
| Fills form. [Submit] |
+<-----------------+
+--[10]------------------------+
| POST /auth/register/handoff |
| Input: form data + URL token |
| Validate input (email unique, |
| pw ≥8, phone format) |
+--[11]------------------------+
| INSERT users: |
| roles: ["CLIENT"] |
| active_role: "CLIENT" |
| client_subtype: "TECH_TEAM" |
| linked_project_id: "proj_abc" |
+--[12]------------------------+
| INSERT project_members: |
| project_id: "proj_abc" |
| user_id: new_tech_user_id |
| role: "TECH_TEAM" |


	v		
+--[13]-------------------------+		
	UPDATE project_handoff_links:			
	status: "CLAIMED"			
	claimed_by: new_tech_user_id			
+--------------------------------+		
	v		
+--[14]-------------------------+		
	INSERT wallets (bal=0)			
	Create WALLET_TOPUP VA via			
+		SePay API			
	+-->+
			+--[15]-------
------------------+
				SePay
creates VA. Returns VA.	
	+<--+
	INSERT virtual_accounts			
+--------------------------------+		
	v		
+--[16]-------------------------+		
	Generate JWT:			
	active_role: "CLIENT"			
	client_subtype: "TECH_TEAM"			
	project_ids: ["proj_abc"]			
+--------------------------------+		
	+---------------->+	
		+--[17]-------------------------+
			Redirected to Tech Dashboard:	
			"Complete Architecture Handoff	
			for [Project Name]"	
		+--------------------------------+
=== PHASE C: LINK EXPIRY (EDGE) =			
+--[18]-------------------------+			
	Returns to Elicitation.				



Step-by-Step Narration

PHASE A - TRIGGER & HANDOFF LINK GENERATION

---

[1] - CEO Reaches Elicitation Stage 4, System Detects Infrastructure Signals

- **Actor:** Client / CEO
- **Context:** This flow is a sub-process of MF-4 (AI Elicitation Engine). The CEO has completed Stages 1–3 (symptom intake, archetype confirmation, architecture/scale probe). During Stage 3, the CEO answered behavioral questions that revealed signals of production integration (async processing, stateful memory, legacy data, etc.).
- **Input:** CEO attempts to proceed to Stage 4 (Tech team handoff).
- **Logic:** The frontend sends a request to advance the elicitation state machine. The backend evaluates the Stage 3 responses and the `project.self_technical` flag.
- **Output:** Request sent to Stage 4 route.
- **State Change:** None yet.

---

[2] - Stage 4 Route Guard: Hard-Block CEO

- **Actor:** System (NestJS - ElicitationGuard)
- **Input:** CEO's request to access Stage 4, JWT (`active_role = "CLIENT"`, `client_subtype = "CEO"`), and `project.self_technical = false`.
- **Logic:**
  - The system checks if `project.self_technical = true`. If true, the CEO would be allowed to complete Stage 4 themselves (Scenario B). In this case, it is `false`.
  - The Stage 4 route is guarded by `client_subtype = "TECH_TEAM"`. Since the CEO's subtype is `"CEO"`, the guard returns **403 FORBIDDEN**.
  - This is a **design-intentional hard-block** - the CEO cannot see or fill in the technical architecture form because:
    1. They lack the technical vocabulary to accurately describe the system.
    2. The system needs specific stack tags, integration methods, and data volumes that only the technical lead knows.
    3. Artifact B (the sensitive technical IP vault) must be populated by someone who understands what is and isn't sensitive.
- **Output:** `403 { code: "TECH_HANDOFF_REQUIRED", message: "Your project involves live infrastructure. A technical team member must complete the architecture handoff.", project_id: "proj_abc" }`.
- **State Change:** None.

---

[3] - UI Displays Hard-Block Message & CTA

- **Actor:** Client / CEO
- **Input:** 403 response from [2].
- **Logic:** The frontend renders the hard-block message in plain business language:
  - "*Your project involves live infrastructure. To protect your system's IP and ensure the hired expert understands your architecture before being selected, your technical lead must confirm the following details.*"
  - A prominent button appears: **[Generate Handoff Link]**.
  - The UI explains that the link is secure, expires in 72 hours, and can only be used by one person.
- **Output:** UI ready for CEO action.
- **State Change:** None.

---

[4] - Generate Signed Handoff JWT & Persist Record

- **Actor:** System (NestJS - HandoffService)
- **Input:** CEO clicks [Generate Handoff Link]. `POST /projects/{project_id}/handoff/generate`.
- **Logic:**

  1. **Generate signed JWT:** The system creates a token specifically for this handoff. The token is not a login credential - it is a single-use invitation token that encodes the project context.

```
{
  "project_id": "proj_abc",
  "client_subtype": "TECH_TEAM",
  "iat": 1747300000,
  "exp": 1747568000  // now() + 72 hours
}
```

  - `project_id` binds the token to this specific project. The registration endpoint will use this to set `linked_project_id` on the new account.
  - `client_subtype` locks the registration path - even if someone modifies the frontend, the backend enforces that this token creates a TECH_TEAM account.
  - `exp` enforces the 72-hour deadline.

  2. **Construct the Handoff URL:** `https://aitasker.com/register?token={signed_jwt}`

  3. **Persist the handoff link record** (for tracking and invalidation):

Field	Value Set	Rationale
`project_id`	`"proj_abc"`	Links token to project
`token_hash`	SHA-256 of JWT	Stored instead of raw token for security; used for lookup on claim
`status`	`"PENDING"`	Not yet claimed
`created_by`	CEO `user_id`	Audit trail
`expires_at`	`now() + 72h`	For quick DB-level expiry checks
`claimed_by`	`NULL`	Populated when Tech Team registers

- **Output:** Handoff URL returned to CEO.
- **State Change:** `project_handoff_links` table gains one row with status `PENDING`.
- **Tables Affected:** `project_handoff_links` (INSERT)

---

[5] - CEO Copies & Sends Handoff Link

- **Actor:** Client / CEO
- **Input:** Handoff URL from [4].
- **Logic:** The UI displays the link in a copyable text field with a [Copy Link] button. The CEO sends this link to their technical lead via their preferred communication channel (Slack, Email, Zalo). The platform does not send the link directly - the CEO is the trusted delivery mechanism.
- **Output:** Link leaves the platform's control. CEO waits for Tech Team to complete handoff.
- **State Change:** None.

PHASE B - TECH TEAM REGISTRATION VIA HANDOFF LINK

---

[6] - Tech Team Member Clicks Handoff Link

- **Actor:** Client / Tech Team (unauthenticated user)

- **Input:** Tech Team member receives the link from the CEO and clicks it.
- **Logic:** The browser navigates to `https://aitasker.com/register?token={jwt}`. The frontend parses the URL, extracts the token, and sends it to the backend for validation before rendering the form.
- **Output:** `POST /auth/register/handoff/validate` with `{ token: "eyJhbG..." }`.
- **State Change:** None.

---

[7] - Validate Handoff Token

- **Actor:** System (NestJS - HandoffController)
- **Input:** Token from [6].
- **Logic:** Four sequential validation checks:
  1. **JWT signature verification:** Ensure the token was issued by the platform and hasn't been tampered with.
  2. **Expiration check:** `exp > now()`. If the token has expired (> 72 hours old), return `410 GONE { code: "HANDOFF_LINK_EXPIRED", message: "This handoff link has expired. Ask the CEO to generate a new one." }`.
  3. **Database status check:** Look up `project_handoff_links` by `token_hash`. If `status != "PENDING"` (e.g., already `CLAIMED` or `EXPIRED`), return `410 GONE { code: "HANDOFF_LINK_ALREADY_USED" }`.
  4. **Project existence check:** Verify `project_id` in the token refers to an existing, non-deleted project.
- **Output (valid):** `200 OK { project_id: "proj_abc", project_name: "AdTech Compliance System", client_subtype: "TECH_TEAM" }`. Frontend uses this to personalize the registration form.
- **Output (invalid):** HTTP 410 with specific error code.
- **State Change:** None - read-only validation.

---

[8] - Registration Form Renders (TECH_TEAM Context)

- **Actor:** Client / Tech Team
- **Input:** Validated token response from [7].
- **Logic:** The registration form renders with TECH_TEAM-specific framing:
  - **Header:** "*Your CEO is scoping an AI integration for [Project Name]. To protect your system's IP and ensure the hired expert understands your architecture before being selected, please create your account and complete the architecture handoff.*"
  - **Visible fields:** `email`, `password`, `full_name`, `phone` (same as CEO/Expert registration).
  - **Hidden/locked fields:** Role selection is NOT shown. The token has already determined that this account will be `CLIENT / TECH_TEAM`. The user cannot change this.
  - **Project context:** The project name is displayed so the Tech Team member knows which project they're being invited to.
- **Output:** Registration form rendered.
- **State Change:** None.

---

[9] - Tech Team Fills Form & Submits

- **Actor:** Client / Tech Team
- **Input:** Form fields (`email`, `password`, `full_name`, `phone`).
- **Logic:** Client-side validation runs (same as MF-1/2). The token from the URL is preserved in a hidden field or local state to be sent with the submission.
- **Output:** `POST /auth/register/handoff` with payload `{ email, password, full_name, phone, token: "eyJhbG..." }`.
- **State Change:** None.

---

[10] - Validate Input & Token (Server-Side)

- **Actor:** System (NestJS - HandoffController + AuthService)
- **Input:** Registration payload + token from [9].
- **Logic:**
  1. **Re-validate the token:** Same checks as [7] (signature, expiry, DB status). This prevents a race condition where the token expires between form load and submission.
  2. **Validate input fields:** Email uniqueness, password length, phone format.
  3. **Extract token claims:** `project_id` and `client_subtype` are taken from the **verified token**, NOT from the request body. This prevents a user from manipulating the payload to register as a different role.
- **Output:** Validated input + token claims ready for account creation.
- **State Change:** None.

---

[11] - Insert User Record (TECH_TEAM Role)

- **Actor:** System (NestJS - AuthService)
- **Input:** Validated registration data + token claims from [10].
- **Logic:** Hash password and insert into `users` table with TECH_TEAM-specific fields:

Field	Value Set	Rationale

Field	Value Set	Rationale
`email`	from input	Login identifier
`password_hash`	bcrypt hash	Authentication credential
`full_name`	from input	Display name
`phone`	from input	Contact + VA reference
`roles`	`["CLIENT"]`	TECH_TEAM is a subtype of CLIENT, not a separate role
`active_role`	`"CLIENT"`	Same as CEO - the distinction is in `client_subtype`
`client_subtype`	`"TECH_TEAM"`	**From token** - this is the critical field that gates Stage 4 access, bid review, Artifact B, etc.
`linked_project_id`	`"proj_abc"`	**From token** - binds this account to the specific project that triggered the handoff
`subscription_client_tier`	`"free"`	Default (Tech Team cannot purchase subscriptions)
`sepay_bank_account_xid`	`NULL`	Not applicable for Tech Team

Key design decision: The `linked_project_id` field ensures that this Tech Team account is scoped to the specific project. If the CEO creates a second project later, they can invite the same Tech Team member (adding them to `project_members` for the new project), but the account's primary association remains.

- **Output:** `tech_user_id` generated.
- **State Change:** `users` table gains one row. TECH_TEAM account now exists.
- **Tables Affected:** `users`

---

[12] - Insert Project Membership Record

- **Actor:** System (NestJS - ProjectService, called from AuthService within same transaction)
- **Input:** `tech_user_id` from [11], `project_id` from token.
- **Logic:** The Tech Team member is added to the project's member list. This is the record that the matching engine and self-exclusion guard check.

Field	Value Set	Rationale
`project_id`	`"proj_abc"`	From token
`user_id`	`tech_user_id`	From [11]
`role`	`"TECH_TEAM"`	Distinguishes from CEO and Expert members
`joined_at`	`now()`	Audit trail

- **Output:** Membership record persisted.
- **State Change:** `project_members` table gains one row. Tech Team is now a member of the project. The self-exclusion guard (`expert.user_id NOT IN project.involved_parties`) will now include this user.
- **Tables Affected:** `project_members`

---

[13] - Mark Handoff Link as Claimed

- **Actor:** System (NestJS - HandoffService)
- **Input:** `token_hash` from [10], `tech_user_id` from [11].
- **Logic:** The handoff link is consumed - it cannot be used again.

```
UPDATE project_handoff_links
  SET status = 'CLAIMED',
      claimed_by = tech_user_id,
      claimed_at = now()
  WHERE token_hash = ?;
```

- **Output:** Handoff link marked as claimed.
- **State Change:** `project_handoff_links` row updated from `PENDING` to `CLAIMED`. Any subsequent attempt to use this token will fail at [7].
- **Tables Affected:** `project_handoff_links` (UPDATE)

---

[14] - Create Wallet & Virtual Account

- **Actor:** System (NestJS + SePay API)
- **Input:** `tech_user_id` from [11].
- **Logic:** Identical to MF-1 Steps [5]–[7] and MF-2 Step [4]. Every user gets a wallet and a permanent `WALLET_TOPUP` VA, regardless of role. Even though TECH_TEAM cannot fund milestones or purchase subscriptions (per RBAC matrix), the wallet exists for architectural consistency and potential future use (e.g., if they add an EXPERT role later).
    1. Create `wallets` row (`available_balance = 0`, `locked_balance = 0`).
    2. Call SePay Bank Hub API to create VA.

3. Store mapping in `virtual_accounts`.

- **Output:** Wallet and VA created.
- **State Change:** `wallets` and `virtual_accounts` each gain one row.
- **Tables Affected:** `wallets`, `virtual_accounts`

---

[15] - SePay Processes VA Creation

- **Actor:** SePay / Bank Hub
- **Input:** VA creation request from [14].
- **Logic:** SePay creates the sub-account under the platform's master bank account and returns the `va_number`.
- **Output:** `{ va_number: "XXXXXX" }` returned to NestJS.
- **State Change:** External - VA created on SePay side.

---

[16] - Generate JWT (TECH_TEAM Context)

- **Actor:** System (NestJS - AuthService)
- **Input:** `tech_user_id`, role data, `linked_project_id` from [11].
- **Logic:** Construct JWT payload per F1 specification:

```
{
  "sub": "tech_user_id",
  "active_role": "CLIENT",
  "client_subtype": "TECH_TEAM",
  "roles": ["CLIENT"],
  "self_technical_projects": [],
  "project_ids": ["proj_abc"]
}
```

Key differences from CEO JWT:

- `client_subtype` is `"TECH_TEAM"` - this is what the Stage 4 guard, bid review guard, and Artifact B guard check.
- `project_ids` includes `"proj_abc"` immediately - the account is born with project context.
- **Output:** JWT set as HTTP-only cookie.
- **State Change:** None.

---

[17] - Redirected to Tech Dashboard (Architecture Handoff)

- **Actor:** Client / Tech Team
- **Input:** JWT from [16]. Frontend reads JWT claims.
- **Logic:** The Tech Team member is redirected to the **Tech Dashboard**, NOT the CEO Dashboard. The Tech Dashboard is a completely different interface:

Tech Dashboard Feature	Purpose
Architecture Handoff Form	The primary CTA - "Complete Architecture Handoff for [Project Name]" → This is Elicitation Stage 4 (MF-4)
Artifact B View	Accessible once engagement is CONNECTED
Bid Technical Review	Review bids, flag Recommended/Not Recommended, request revisions (Surface B)
Sprint View	Comment on sprint updates, provide technical feedback
Milestone Technical Sign-off	Sign off technical milestones
Spec Clarification Response	Respond to expert questions (Surface A)

What is NOT visible on the Tech Dashboard:

- Wallet top-up / subscription management (RBAC: TECH_TEAM cannot purchase)
- Project initiation / elicitation Stages 1–3
- Connection requests / expert selection
- Milestone funding
- CEO-specific views

Primary CTA: The dashboard prominently displays the pending architecture handoff for the linked project. When the Tech Team member clicks it, they enter MF-4 Stage 4, where they will input stack tags, integration methods, data volumes, and upload sensitive architecture documents (populating Artifact B).

- **Output:** Tech Dashboard rendered with architecture handoff CTA.
- **State Change:** None.

PHASE C - LINK EXPIRY & REGENERATION (EDGE FLOW)

[18] - CEO Returns to Elicitation, Link Still Pending/Expired

- **Actor:** Client / CEO
- **Context:** The CEO returns to the project elicitation page (perhaps after 72+ hours) and the Tech Team has not claimed the link, or the link has expired.
- **Input:** CEO views the project status. The UI shows: *"Handoff link sent but not yet claimed. Expires in [X hours]."* or *"Handoff link has expired."*
- **Logic:** CEO clicks **[Regenerate Handoff Link]**.
- **Output:** `POST /projects/{project_id}/handoff/regenerate` request sent.
- **State Change:** None.

---

[19] - Invalidate Old Token & Generate New One

- **Actor:** System (NestJS - HandoffService)
- **Input:** Regeneration request from [18], CEO JWT.
- **Logic:**
  1. **Invalidate the old token:** Mark the existing `PENDING` handoff link as expired.

```
UPDATE project_handoff_links
  SET status = 'EXPIRED'
  WHERE project_id = ? AND status = 'PENDING';
```

  2. **Generate a new signed JWT** (same logic as [4], new `iat` and `exp`).
  3. **Persist new handoff link record** (new row with status `PENDING`, new `token_hash`, new `expires_at`).
- **Output:** New handoff URL returned to CEO.
- **State Change:** Old `project_handoff_links` row set to `EXPIRED`. New row inserted with `PENDING`.
- **Tables Affected:** `project_handoff_links` (UPDATE + INSERT)

---

[20] - New Link Generated & Sent

- **Actor:** Client / CEO
- **Input:** New handoff URL from [19].
- **Logic:** CEO copies the new link and sends it to the Tech Team member. The old link (if still clicked) will fail validation at Step [7] with `410 HANDOFF_LINK_EXPIRED`.
- **Output:** New link delivered. Flow returns to Phase B, Step [6].
- **State Change:** None.

Summary: End-to-End State Trace for MF-3

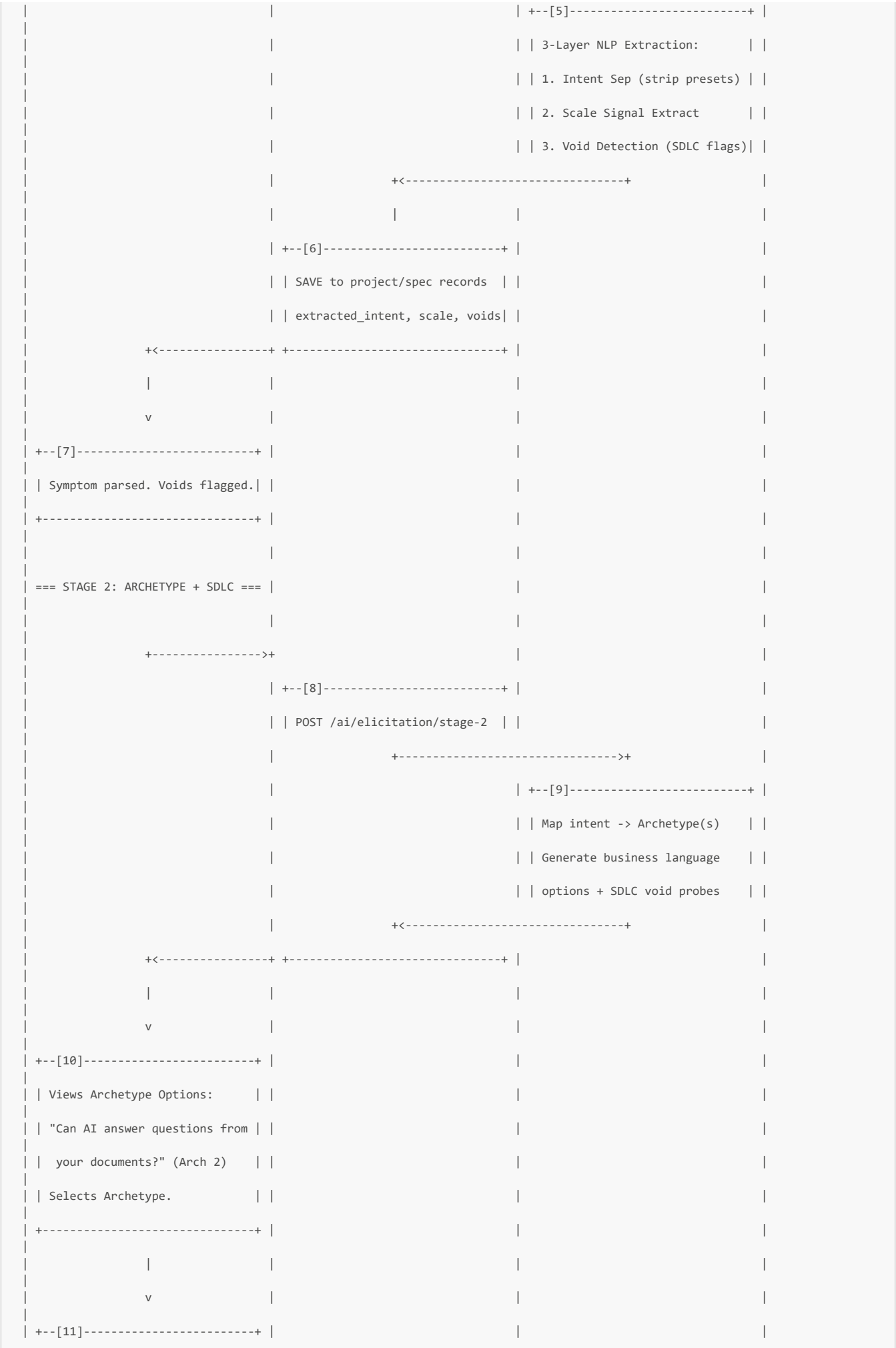
Step	`users`	`project_members`	`project_handoff_links`	`wallets` / `virtual_accounts`	JWT Claims
[4] Link generated	-	-	status=PENDING, token_hash=xxx, claimed_by=NULL	-	-
[11] Tech Team registers	roles=["CLIENT"], active_role="CLIENT", client_subtype="TECH_TEAM", linked_project_id="proj_abc"	-	-	-	-
[12] Project membership	-	project_id="proj_abc", user_id=tech_id, role="TECH_TEAM"	-	-	-
[13] Link claimed	-	-	status=CLAIMED, claimed_by=tech_id	-	-
[14] Wallet + VA	-	-	-	wallet: avail=0, locked=0; VA: WALLET_TOPUP	-
[16] JWT issued	-	-	-	-	active_role="CLIENT", client_subtype="TECH_TEAM", project_ids=["proj_abc"]

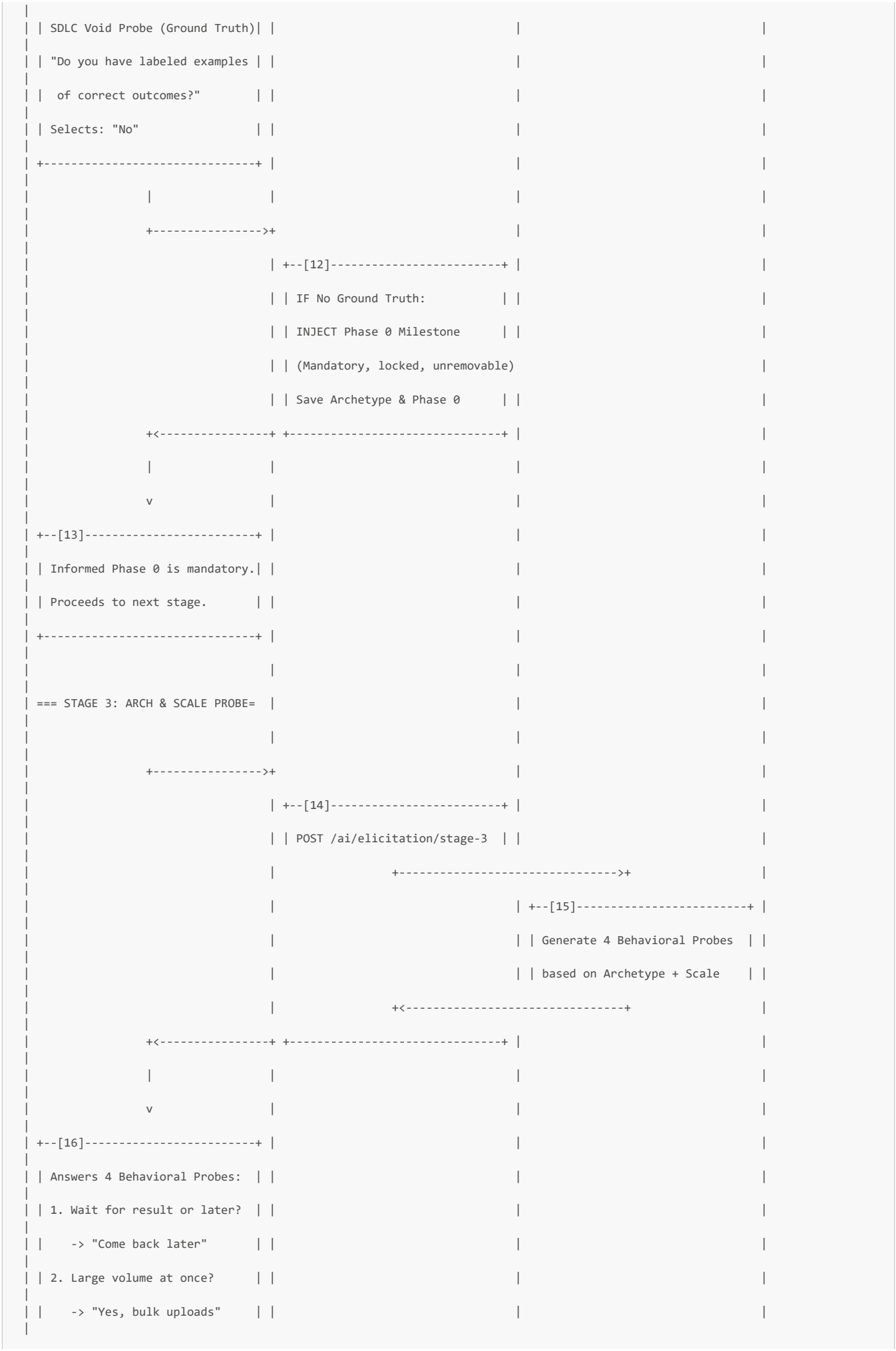
Step	users	project_members	project_handoff_links	wallets / virtual_accounts	JWT Claims
[19] Link regenerated	-	-	old: status=EXPIRED; new: status=PENDING	-	-

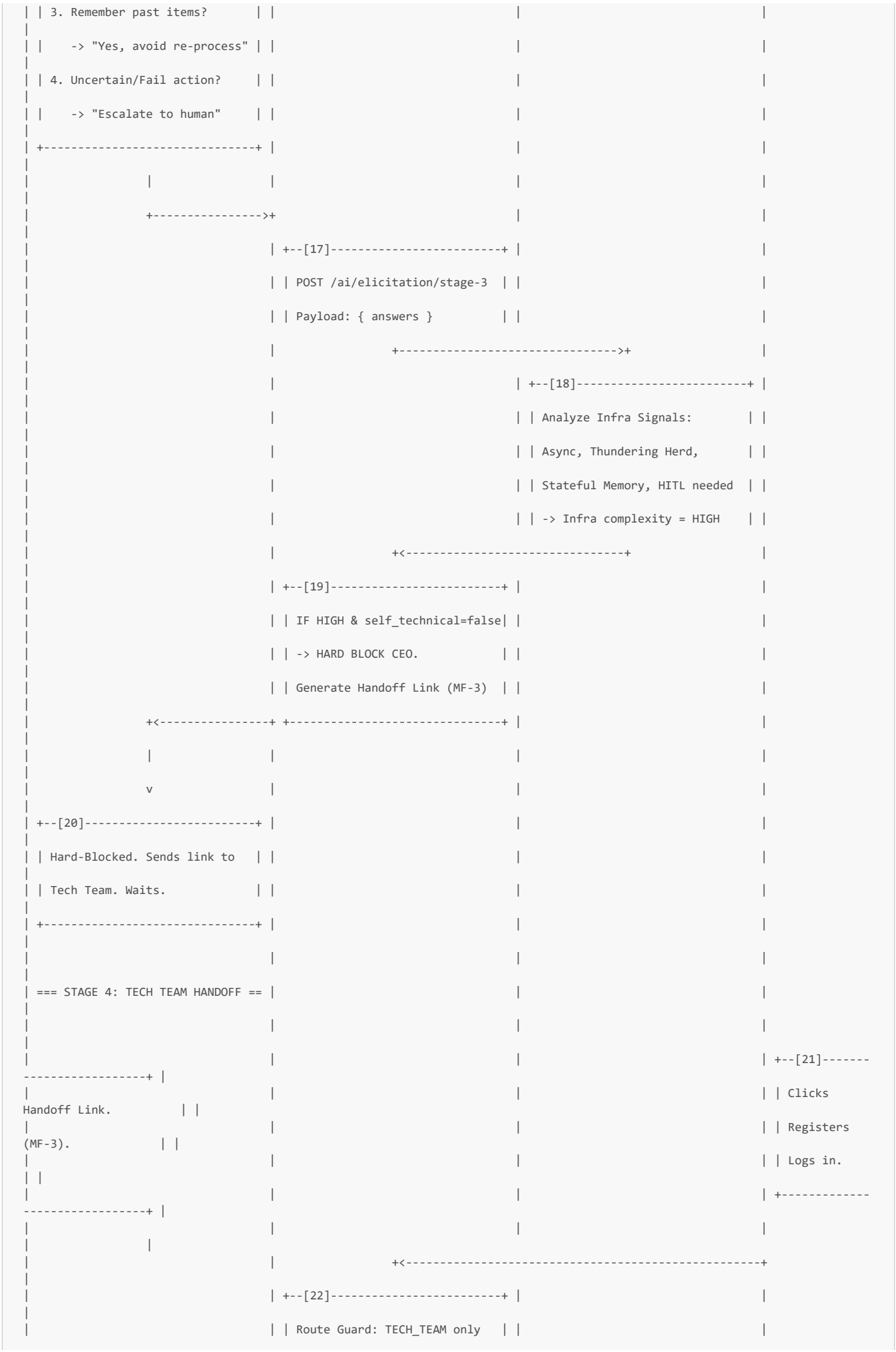
MF-3 is now complete. The TECH_TEAM account exists, is bound to the project, and has the `client_subtype = "TECH_TEAM"` required to access Elicitation Stage 4. The Tech Team member can now complete the architecture handoff (MF-4 Stage 4), populating Artifact B with sensitive technical details that the CEO could not provide. The self-exclusion guard is active - this user cannot bid on this project as an Expert.

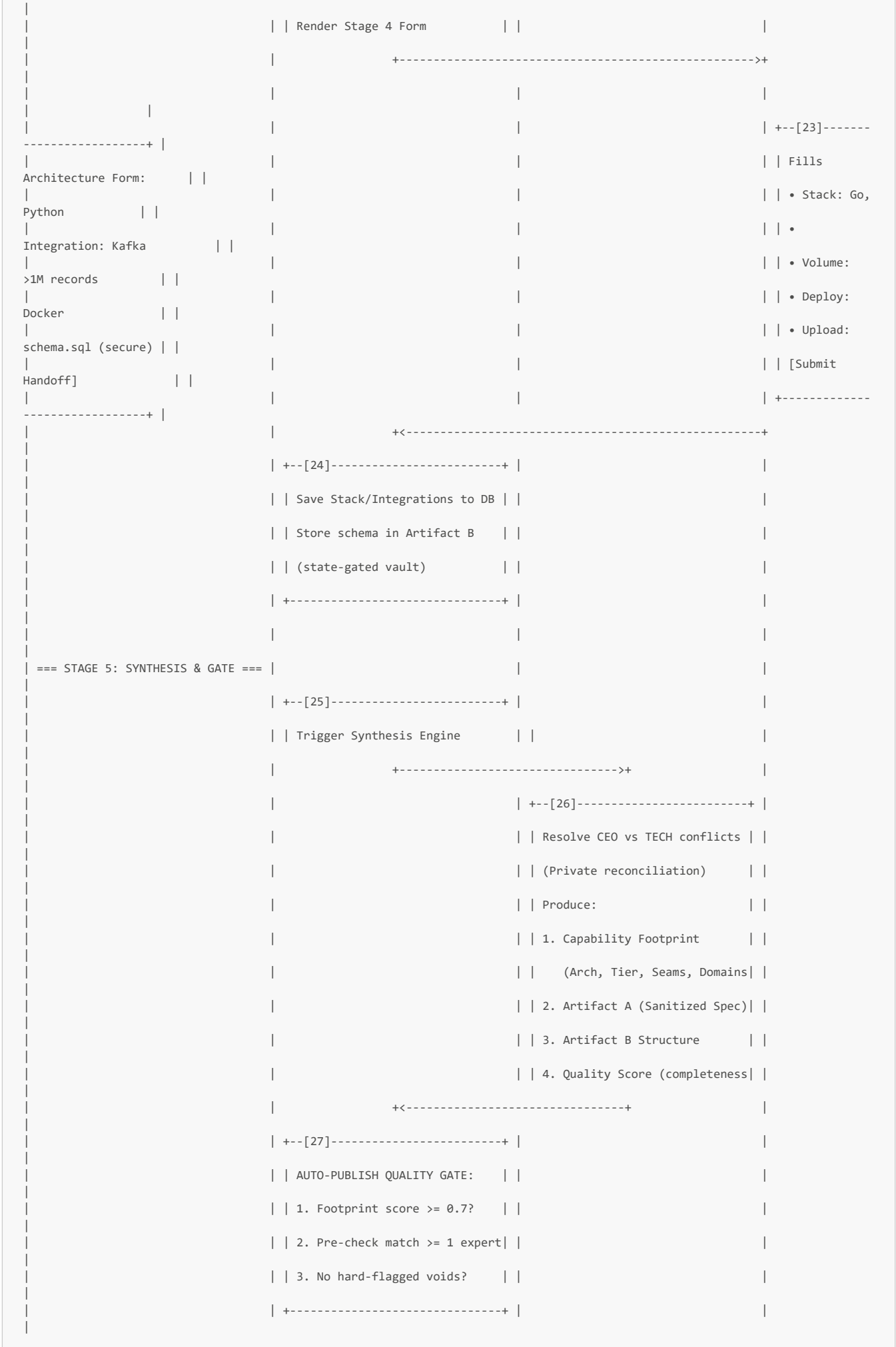
MF-4: AI Elicitation Engine (Project Intake)

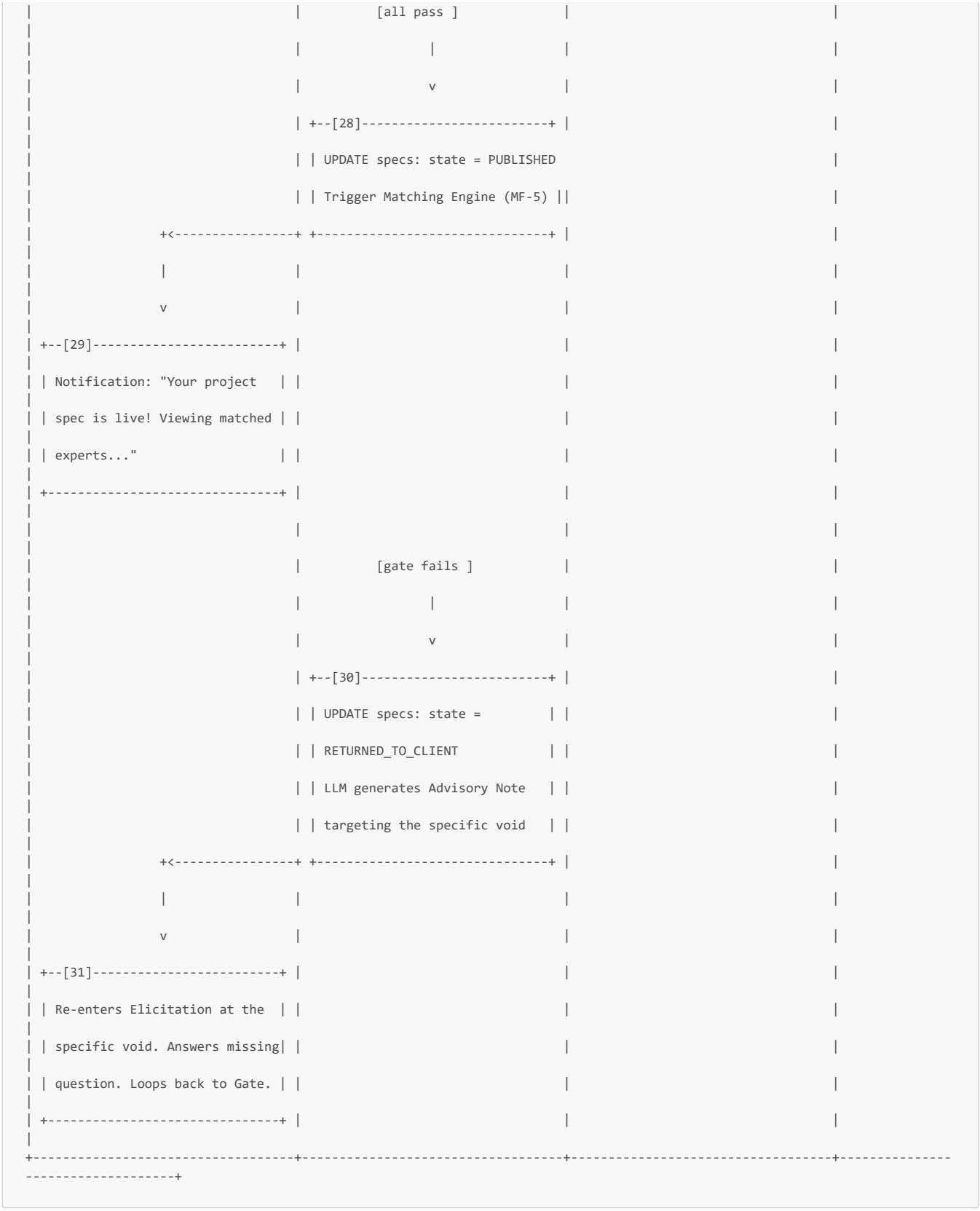
Detailed Swimlane Flow


	+--[5]--------------------------+
	3-Layer NLP Extraction:	
	1. Intent Sep (strip presets)	
	2. Scale Signal Extract	
	3. Void Detection (SDLC flags)|
+<--------------------------------+
| | |
| +--[6]--------------------------+ |
| | SAVE to project/spec records | |
| | extracted_intent, scale, voids| |
+<----------------+ +--------------------------------+ |
| | |
v | |
+--[7]--------------------------+ | |
| Symptom parsed. Voids flagged.| | |
+--------------------------------+ | |
| | |
=== STAGE 2: ARCHETYPE + SDLC === | |
| | |
+---------------->+ | |
| +--[8]--------------------------+ | |
| | POST /ai/elicitation/stage-2 | | |
| +--------------------------------->+ |
	+--[9]--------------------------+
	Map intent -> Archetype(s)	
	Generate business language	
	options + SDLC void probes	
+<--------------------------------+
+<----------------+ +--------------------------------+	
v		
+--[10]-------------------------+		
Views Archetype Options:			
"Can AI answer questions from			
your documents?" (Arch 2)			
Selects Archetype.			
+--------------------------------+		
v | | |
+--[11]-------------------------+ | | |


SDLC Void Probe (Ground Truth)
"Do you have labeled examples
of correct outcomes?"
Selects: "No"
[12]
IF No Ground Truth:
INJECT Phase 0 Milestone
(Mandatory, locked, unremovable)
Save Archetype & Phase 0
[13]
Informed Phase 0 is mandatory.
Proceeds to next stage.
=== STAGE 3: ARCH & SCALE PROBE=
[14]
POST /ai/elicitation/stage-3
[15]
Generate 4 Behavioral Probes
based on Archetype + Scale
[16]
Answers 4 Behavioral Probes:
1. Wait for result or later?
-> "Come back later"
2. Large volume at once?
-> "Yes, bulk uploads"


| 3. Remember past items? |
| -> "Yes, avoid re-process" |
| 4. Uncertain/Fail action? |
| -> "Escalate to human" |
[17]
POST /ai/elicitation/stage-3
Payload: { answers }
[18]
Analyze Infra Signals:
Async, Thundering Herd,
Stateful Memory, HITL needed
-> Infra complexity = HIGH
[19]
IF HIGH & self_technical=false
-> HARD BLOCK CEO.
Generate Handoff Link (MF-3)
[20]
Hard-Blocked. Sends link to
Tech Team. Waits.
=== STAGE 4: TECH TEAM HANDOFF ==
[21]
Clicks
Handoff Link.
Registers
(MF-3).
Logs in.
[22]
Route Guard: TECH_TEAM only


| | | Render Stage 4 Form | | | |
| | +----------->+
| | | | | | +--[23]-------
| | | | | | | Fills
Architecture Form: | |
| | | | | | | • Stack: Go,
Python | |
| | | | | | | •
Integration: Kafka | |
| | | | | | | • Volume:
>1M records | |
| | | | | | | • Deploy:
Docker | |
| | | | | | | • Upload:
schema.sql (secure) | |
| | | | | | | [Submit
Handoff] | |
| | | | | | +-------------
| | +<-----------+
	+--[24]-------------------------+		
		Save Stack/Integrations to DB			
		Store schema in Artifact B			
		(state-gated vault)			
	+--------------------------------+		
=== STAGE 5: SYNTHESIS & GATE ===		
	+--[25]-------------------------+		
		Trigger Synthesis Engine			
	+--------------------------------->+	
		+--[26]-------------------------+	
			Resolve CEO vs TECH conflicts		
			(Private reconciliation)		
			Produce:		
			1. Capability Footprint		
			(Arch, Tier, Seams, Domains		
			2. Artifact A (Sanitized Spec)		
			3. Artifact B Structure		
			4. Quality Score (completeness		
	+<---------------------------------+	
	+--[27]-------------------------+		
		AUTO-PUBLISH QUALITY GATE:			
		1. Footprint score >= 0.7?			
		2. Pre-check match >= 1 expert			
		3. No hard-flagged voids?			
	+--------------------------------+		



Step-by-Step Narration

STAGE 1 - SYMPTOM INTAKE

[1] - Clicks "Start New AI Project"

- **Actor:** Client / CEO
- **Input:** CEO clicks the primary CTA on their dashboard.
- **Logic:** Frontend prepares to initialize the elicitation engine session.
- **Output:** `POST /projects/initiate` request sent.

- **State Change:** None.

---

[2] - Subscription Guard, Create DRAFT Project & Spec

- **Actor:** System (NestJS)
- **Input:** Request from [1], JWT (`active_role = "CLIENT"`, `client_subtype = "CEO"`).
- **Logic:**
  1. **Subscription Guard:** Verify `users.subscription_client_tier = 'pro'`. If not → `403 SUBSCRIPTION_REQUIRED`.
  2. **Initialize Project:** Create a shell project record to bind the elicitation data to.
     - `INSERT INTO projects (ceo_user_id, status) VALUES (?, 'DRAFT')`
  3. **Initialize Spec:** Create a shell spec record to track the artifact state.
     - `INSERT INTO specs (project_id, state) VALUES (?, 'DRAFT')`
- **Output:** Return `project_id` to frontend. Frontend loads the conversational Elicitation UI.
- **State Change:** `projects` and `specs` tables each gain one row in `DRAFT` state.
- **Tables Affected:** `projects`, `specs`

---

[3] - Types Free-Form Business Pain

- **Actor:** Client / CEO
- **Input:** The Elicitation UI presents a text area with explicit instructions: "*Tell us what hurts: what you're currently doing, what costs too much or doesn't work, and what business goal you're trying to reach. DO NOT write a technical spec.*"
  - CEO types: "*Our AI output is inconsistent. We process about 600K ads a month and the compliance checks are all over the place. We want to fine-tune a model to fix this.*"
- **Logic:** CEO clicks `[Submit Symptom]`. Frontend sends the raw text.
- **Output:** `POST /ai/elicitation/stage-1` with `{ raw_intent: "...", project_id: "..." }`.
- **State Change:** None.

---

[4] - Forward to AI Service

- **Actor:** System (NestJS)
- **Input:** Stage-1 payload from [3].
- **Logic:** NestJS acts as the orchestrator. It constructs the FastAPI payload, appending the project's accumulated context (currently empty) and the Master Reference Sheet taxonomy definitions as a system prompt constraint.
- **Output:** Internal HTTP request to FastAPI `/elicitation/process-stage-1`.
- **State Change:** None.

---

[5] - 3-Layer NLP Extraction (FastAPI + Claude API)

- **Actor:** AI Service (FastAPI)
- **Input:** Raw intent text.
- **Logic:** The LLM runs three sequential extraction passes:
  1. **Intent Separation:** Strips prescribed solutions from actual symptoms.
     - *Prescribed solution stripped:* "We want to fine-tune a model"
     - *Actual symptom retained:* "Compliance checks are all over the place for 600K ads/month"
  2. **Scale Signal Extraction:** Identifies volume, frequency, cost indicators determining project tier.
     - *Extracted:* Volume = ~600K/month (indicates Tier 3 / Complex)
  3. **Void Detection:** Checks for absent mandatory AI SDLC components.
     - *Void 1:* No evaluation ground truth mentioned.
     - *Void 2:* No error handling/fallback strategy mentioned.
     - *Void 3:* No existing system integration mentioned.
- **Output:** Returns structured JSON `{ extracted_symptoms, prescribed_solutions, scale_signals, voids_detected }`.
- **State Change:** None (stateless AI computation).

---

[6] - Save Extraction Results

- **Actor:** System (NestJS)
- **Input:** FastAPI response from [5].
- **Logic:** NestJS persists the extracted data into the project/spec context records. This accumulated context is sent with every subsequent AI call to maintain conversation coherence.
- **Output:** Data saved. Frontend notified to proceed.
- **State Change:** Project context updated with Stage 1 extractions.
- **Tables Affected:** `project_contexts` (UPSERT)

---

[7] - Symptom Parsed, Voids Flagged

- **Actor:** Client / CEO
- **Input:** UI update from [6].

- **Logic:** The conversational UI acknowledges the input and subtly indicates that it has identified structural gaps: *"We understand the core issue. Before we match you, we need to clarify a few things about how you measure success and your existing systems."*
- **Output:** UI ready for Stage 2.
- **State Change:** None.

STAGE 2 - ARCHETYPE CONFIRMATION & SDLC INJECTION

---

[8] - Request Archetype Mapping

- **Actor:** System (NestJS)
- **Input:** CEO advances to Stage 2.
- **Logic:** NestJS calls FastAPI with the updated project context (including Stage 1 voids) to generate the archetype options and SDLC probes.
- **Output:** Internal request to FastAPI `/elicitation/process-stage-2`.
- **State Change:** None.

---

[9] - Map to Archetype & Generate Probes (FastAPI)

- **Actor:** AI Service (FastAPI)
- **Input:** Project context.
- **Logic:**
  1. **Archetype Mapping:** Based on "compliance checks on 600K ads", the LLM identifies **Archetype 1 (Automated Decision System)** as the probable fit.
  2. **Business Language Translation:** Instead of showing "Archetype 1", it generates a plain-English question: *"Can AI replace or augment a human decision your team makes repeatedly - like checking if an ad complies with policy?"*
  3. **SDLC Void Probes:** Generates behavioral probes for the voids detected in Step [5].
- **Output:** Returns `{ archetype_options, sdlc_probes }`.
- **State Change:** None.

---

[10] - Views Archetype Options

- **Actor:** Client / CEO
- **Input:** FastAPI response rendered by NestJS into conversational UI.
- **Logic:** The UI presents the archetype confirmation. The CEO sees the plain-English description and selects it.
- **Output:** CEO selects "Yes, that's exactly the problem."
- **State Change:** None.

---

[11] - Answers SDLC Void Probe (Ground Truth)

- **Actor:** Client / CEO
- **Input:** The UI asks the SDLC probe generated for the evaluation void: *"To measure whether the AI is working, your expert will need a reference dataset of past decisions. Do you currently have labeled examples of correct outcomes?"*
- **Logic:** The CEO answers "No".
- **Output:** Selection sent to backend.
- **State Change:** None.

---

[12] - Inject Phase 0 Milestone (Mandatory)

- **Actor:** System (NestJS)
- **Input:** CEO's "No" response regarding ground truth.
- **Logic:** This is a critical structural intervention. The system does not allow the project to proceed without an evaluation baseline.
  1. **Inject Milestone:** `INSERT INTO milestones (project_id, title, type, is_mandatory) VALUES (?, 'Phase 0: Ground Truth Baseline Setup', 'EVALUATION_BASELINE', true)`
  2. **Lock Milestone:** The milestone is flagged as `is_locked = true` and `is_removable = false`.
  3. **Update Project Context:** Mark the ground truth void as "RESOLVED_VIA_INJECTION".
- **Output:** Context updated. Milestone injected.
- **State Change:** `milestones` table gains one locked row. Project context updated.
- **Tables Affected:** `milestones`, `project_contexts`

---

[13] - Informed of Mandatory Phase 0

- **Actor:** Client / CEO
- **Input:** UI update from [12].
- **Logic:** The conversational UI explains the injection in business language: *"Without a reference dataset, there is no objective way to verify the AI is accurate - which means you cannot evaluate the expert's work. We have added a mandatory 'Ground Truth Setup' phase as the first step. This protects your investment."*
  - If the CEO tries to remove it, the UI blocks them: *"This phase is contractually mandatory to define acceptance criteria for subsequent milestones."*
- **Output:** CEO accepts and proceeds.
- **State Change:** None.

---

[14] - Request Behavioral Probes

- **Actor:** System (NestJS)
- **Input:** CEO advances to Stage 3.
- **Logic:** NestJS requests the 4 behavioral probes from FastAPI based on the Archetype and Scale.
- **Output:** Internal request to FastAPI.
- **State Change:** None.

---

[15] - Generate 4 Behavioral Probes (FastAPI)

- **Actor:** AI Service (FastAPI)
- **Input:** Project context (Archetype 1, Tier 3 scale signals).
- **Logic:** The LLM generates the 4 specific behavioral questions defined in the F2 spec, tailored to the project's context.
- **Output:** Returns the 4 questions.
- **State Change:** None.

---

[16] - Answers 4 Behavioral Probes

- **Actor:** Client / CEO
- **Input:** The 4 questions rendered in the UI.
- **Logic:** The CEO answers from a business perspective, without needing technical vocabulary:
  1. "*When a user submits content for AI processing, do they wait for an immediate result, or do they come back later to check?*" → **"Come back later"** (Signals: Async architecture needed)
  2. "*Are there times when a very large volume arrives all at once, like a bulk upload?*" → **"Yes, bulk uploads"** (Signals: Thundering Herd risk, queue infrastructure)
  3. "*Does the AI need to remember past items it has processed - for example, to avoid re-processing something it already handled?*" → **"Yes, avoid re-processing"** (Signals: Stateful memory / Vector DB needed)
  4. "*If the AI is uncertain or fails, what should happen - reject automatically, approve automatically, or escalate to a human?*" → **"Escalate to human"** (Signals: HITL workflow requirement)
- **Output:** CEO submits answers.
- **State Change:** None.

---

[17] - Submit Answers

- **Actor:** System (NestJS)
- **Input:** CEO's answers from [16].
- **Logic:** NestJS forwards the answers to FastAPI for infrastructure signal analysis.
- **Output:** Internal request to FastAPI `/elicitation/analyze-stage-3`.
- **State Change:** None.

---

[18] - Analyze Infrastructure Signals (FastAPI)

- **Actor:** AI Service (FastAPI)
- **Input:** CEO's behavioral answers.
- **Logic:** The LLM translates behavioral answers into technical infrastructure requirements:
  - Async + Queue + Stateful Memory + HITL + 600K volume = **High Infrastructure Complexity**.
  - *Result:* This project requires integration with an existing production system. `requires_tech_handoff = true`.
- **Output:** Returns `{ infra_signals, complexity: "HIGH", requires_tech_handoff: true }`.
- **State Change:** None.

---

[19] - Hard Block CEO, Generate Handoff Link

- **Actor:** System (NestJS)
- **Input:** FastAPI response indicating `requires_tech_handoff = true` AND `project.self_technical = false`.
- **Logic:** The system enforces the hard block. The CEO cannot proceed past Stage 3.
  1. Check `project.self_technical`. If true, skip to Stage 4 with CEO acting as TECH_TEAM. If false → Hard Block.
  2. Execute MF-3 Step [4]: Generate signed handoff JWT, persist in `project_handoff_links`.
- **Output:** Handoff link returned to CEO.
- **State Change:** `project_handoff_links` gains one row (status PENDING).

---

[20] - Sends Link to Tech Team

- **Actor:** Client / CEO
- **Input:** Handoff link from [19].
- **Logic:** UI displays the hard block message and the link. CEO copies and sends it. The elicitation flow pauses in a `WAITING_FOR_TECH_HANDOFF` state.
- **Output:** CEO waits.

- **State Change:** None.

STAGE 4 - TECH TEAM HANDOFF

---

[21] - Tech Team Registers & Logs In

- **Actor:** Client / Tech Team
- **Input:** Handoff link received from CEO.
- **Logic:** Tech Team member executes MF-3 (clicks link, validates token, registers, receives TECH_TEAM JWT). They are redirected to the Tech Dashboard.
- **Output:** Tech Team authenticated and ready.
- **State Change:** `users`, `project_members`, `project_handoff_links` updated per MF-3.

---

[22] - Route Guard & Render Stage 4 Form

- **Actor:** System (NestJS)
- **Input:** Tech Team attempts to access Stage 4 form.
- **Logic:** NestJS enforces the guard: `client_subtype = "TECH_TEAM"`. The UI presents a completely different framing: "*Your CEO is scoping an AI integration. To protect your system's IP and ensure the hired expert understands your architecture before being selected, please confirm the following.*"
- **Output:** Stage 4 Architecture Handoff form rendered.
- **State Change:** None.

---

[23] - Fills Architecture Form

- **Actor:** Client / Tech Team
- **Input:** The Stage 4 form fields.
- **Logic:** The Tech Team provides the technical reality that the CEO could not:
  - **Stack tags:** Go, Python (Multiple choice)
  - **Integration method:** Kafka (Not just HTTP REST)
  - **Legacy data volume range:** > 1 Million records
  - **Deployment expectation:** Docker container
  - **Sensitive schema upload:** Uploads `schema.sql` and `kafka_topology.json` (Incentivized: "*These go into the Artifact B vault, inaccessible until an expert is selected.*")
- **Output:** `[Submit Handoff]` sends `POST /projects/{id}/elicitation/stage-4` with form data + files.
- **State Change:** None.

---

[24] - Save Architecture Data & Populate Artifact B

- **Actor:** System (NestJS)
- **Input:** Stage 4 payload from [23].
- **Logic:**
  1. Save stack/integration/volume/deployment data to the project context.
  2. **Secure File Storage:** Upload `schema.sql` and `kafka_topology.json` to S3/private storage.
  3. **Populate Artifact B:** Create/update the `artifacts` record for this project.
     - `artifact_type = "B"`
     - `storage_path = "s3://..."`
     - `access_level = "CONNECTED"` (State-gated: inaccessible until engagement state is CONNECTED)
- **Output:** Handoff complete. Elicitation engine unpauses and triggers Stage 5.
- **State Change:** Project context updated. `artifacts` table gains/updates Artifact B row.
- **Tables Affected:** `project_contexts`, `artifacts`

STAGE 5 - SYNTHESIS & AUTO-PUBLISH QUALITY GATE

---

[25] - Trigger Synthesis Engine

- **Actor:** System (NestJS)
- **Input:** Stage 4 completion signal.
- **Logic:** NestJS has accumulated all signals (CEO Stages 1-3 + Tech Team Stage 4). It sends the full project context to FastAPI for final synthesis.
- **Output:** Internal request to FastAPI `/elicitation/synthesize`.
- **State Change:** None.

---

[26] - Synthesize Footprint & Artifacts (FastAPI)

- **Actor:** AI Service (FastAPI)
- **Input:** Full project context (Symptoms, Archetype 1, Tier 3 scale, Behavioral answers, Tech stack, Schema).
- **Logic:** The most complex AI operation in the elicitation flow:

1. **Conflict Resolution:** If the CEO said "we don't have legacy data" but Tech Team said ">1M records", the system **privately reconciles** in favor of the Tech Team's reality. Neither party is told the other contradicted them.
2. **Produce Capability Footprint:** Structured internal record:
    - *Archetype:* 1 (Automated Decision System)
    - *Tier:* 3 (Complex / Production)
    - *Primary Domains:* A, C, E
    - *Load-bearing Seam:* A↔C
    - *Significant Seams:* C↔E, D↔E
    - *SDLC Framework:* Phase 0 (Ground Truth) → Phase 1 (Core Logic) → Phase 2 (Pipeline Integration)
3. **Produce Artifact A (Public Spec):** Sanitized version for experts. Contains architecture *categories* (Async, Kafka-based, Stateful) but strips proprietary schema details. Includes escrow lock notice for Artifact B.
4. **Produce Artifact B Structure:** Metadata index pointing to the Tech Team's secure uploads.
5. **Calculate Footprint Completeness Score:** Evaluates the structural integrity of the spec.

- **Output:** Returns `{ footprint, artifact_a, artifact_b_index, completeness_score }`.
- **State Change:** None.

---

[27] - Auto-Publish Quality Gate

- **Actor:** System (NestJS)
- **Input:** Synthesis results from [26].
- **Logic:** NestJS runs three automated validations before the spec can go live. **No admin approval is required.**
    1. **Footprint completeness score ≥ 0.7?** (Checks: mandatory injections present, required seam count meets archetype minimum, SDLC framework populated).
    2. **Matching pre-check passes?** Queries the matching engine: `SELECT COUNT(*) FROM expert_seam_claims WHERE seam_id IN (footprint.seams) AND verification_tier >= 2`. Must find at least 1 expert above minimum threshold.
    3. **No hard-flagged voids remain?** (Checks: ground truth void resolved, integration reality confirmed).
- **Output:** Gate decision (Pass or Fail).
- **State Change:** None.

---

[28] - GATE PASSES → Spec PUBLISHED

- **Actor:** System (NestJS)
- **Input:** Gate passes .
- **Logic:**
    1. Update spec state: `UPDATE specs SET state = 'PUBLISHED' WHERE project_id = ?`
    2. Persist Capability Footprint to DB.
    3. Persist Artifact A content to DB.
    4. **Trigger Matching Engine (MF-5):** The shortlisting process begins immediately in the background.
- **Output:** Spec is live. Notification sent to CEO.
- **State Change:** `specs.state` changes from `DRAFT` to `PUBLISHED`. `project_footprints` and `artifacts` (Artifact A) updated.
- **Tables Affected:** `specs`, `project_footprints`, `artifacts`

---

[29] - CEO Notification: Project Live

- **Actor:** Client / CEO
- **Input:** Push notification / email.
- **Logic:** "*Your project spec 'AdTech Compliance System' is live! We are currently matching you with qualified experts. You will be notified when your shortlist is ready.*"
- **Output:** CEO waits for shortlist (MF-5).
- **State Change:** None.

---

[30] - GATE FAILS → Spec RETURNED_TO_CLIENT

- **Actor:** System (NestJS)
- **Input:** Gate fails (e.g., completeness score 0.4, or 0 experts found for A↔C seam).
- **Logic:** The system refuses to publish a broken spec. There is no admin exception queue.
    1. Update spec state: `UPDATE specs SET state = 'RETURNED_TO_CLIENT' WHERE project_id = ?`
    2. **Generate Advisory Note:** NestJS calls FastAPI with the specific gate failure reasons. The LLM generates a targeted advisory note: "*Your spec could not be published because we couldn't find any experts in our current pool who cover the A↔C seam at Tier 3 complexity. Additionally, your integration method is undefined. Please clarify the following...*"
- **Output:** Spec returned. Advisory note generated.
- **State Change:** `specs.state` changes to `RETURNED_TO_CLIENT`.
- **Tables Affected:** `specs`

---

[31] - CEO Re-enters Elicitation at Void

- **Actor:** Client / CEO
- **Input:** Notification with advisory note from [30].

- **Logic:** The CEO clicks the notification and is taken **directly back into the Elicitation Engine at the specific point of failure** - not back to Stage 1. They answer the missing question, and the flow loops back to Step [25] (Synthesis) for re-evaluation.
- **Output:** Corrective action taken. Flow resumes.
- **State Change:** None.

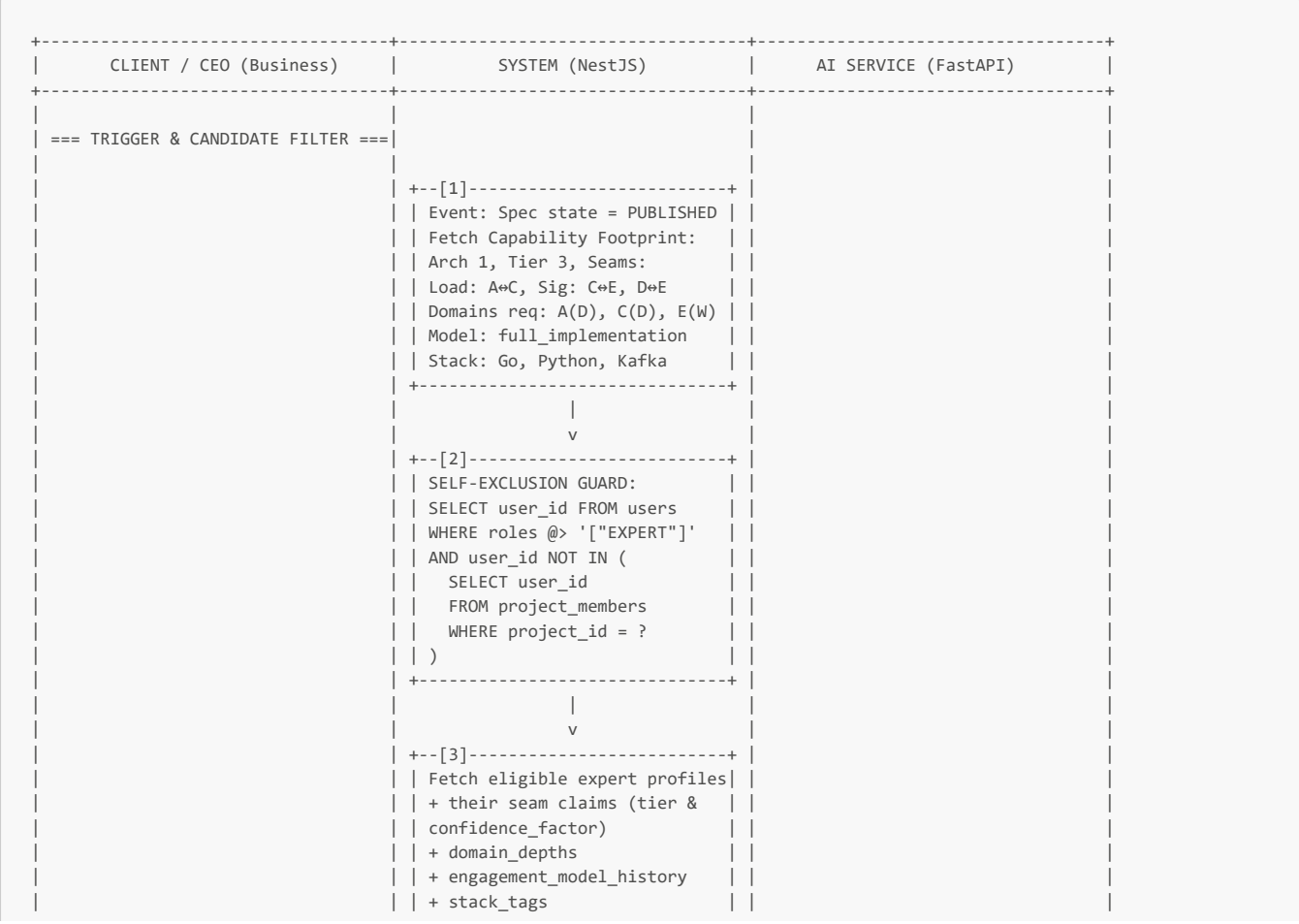
Summary: End-to-End State Trace for MF-4

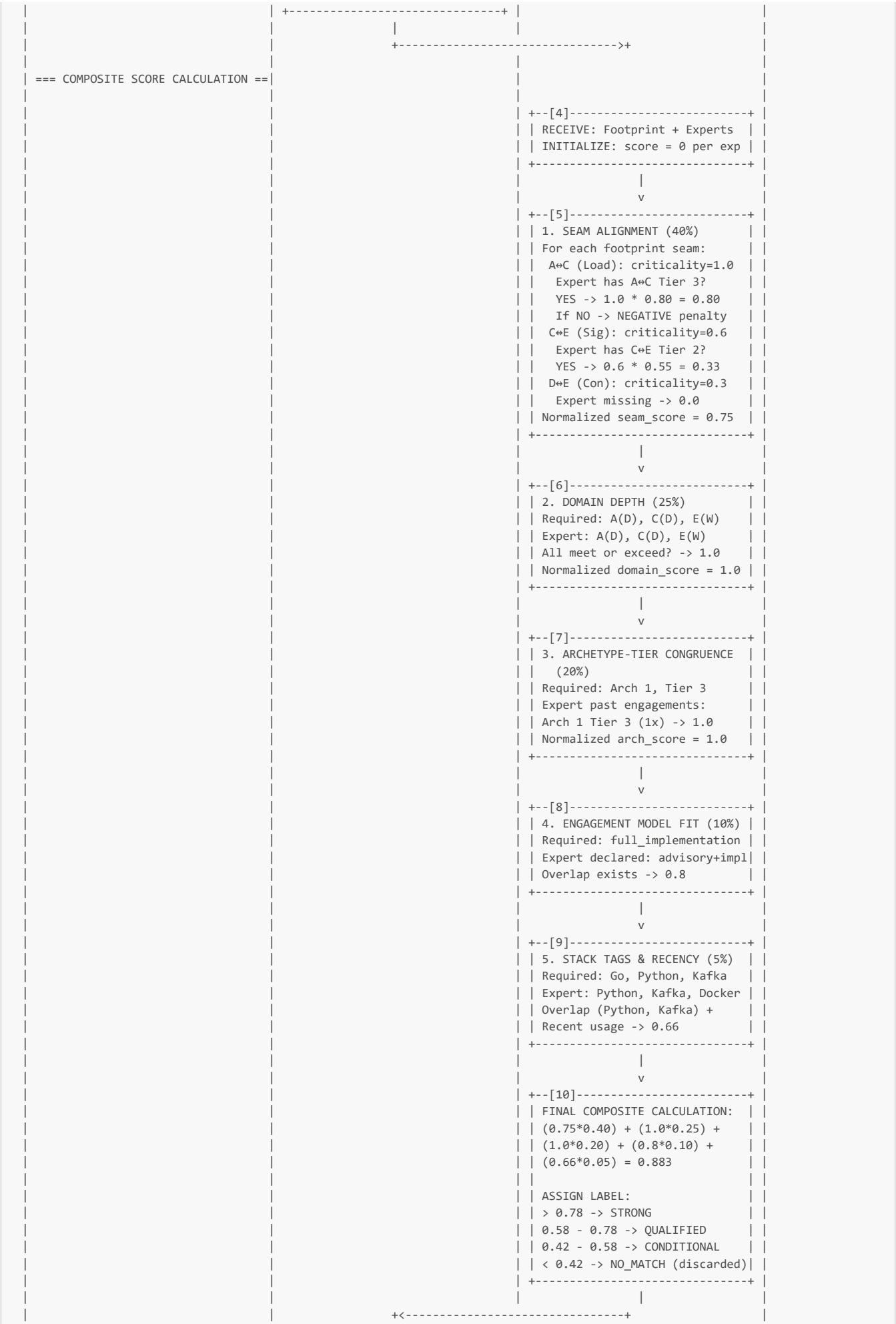
Step	specs	projects	milestones	project_handoff_links	artifacts	AI Context
[2] Init	state=DRAFT	state=DRAFT	-	-	-	-
[6] Stage 1 parsed	-	-	-	-	-	Symptoms, scale, voids extracted
[12] Phase 0 injected	-	-	Phase 0 (locked, mandatory)	-	-	Ground truth void resolved
[19] Hard block	-	-	-	status=PENDING	-	Infra signals = HIGH
[24] Tech handoff	-	-	-	status=CLAIMED	Artifact B created (state-gated)	Stack, integration, volume added
[28] Gate Pass	state=PUBLISHED	-	-	-	Artifact A created (public)	Footprint finalized (Arch 1, Tier 3, Seams)
[30] Gate Fail	state=RETURNED_TO_CLIENT	-	-	-	-	Advisory note generated targeting void

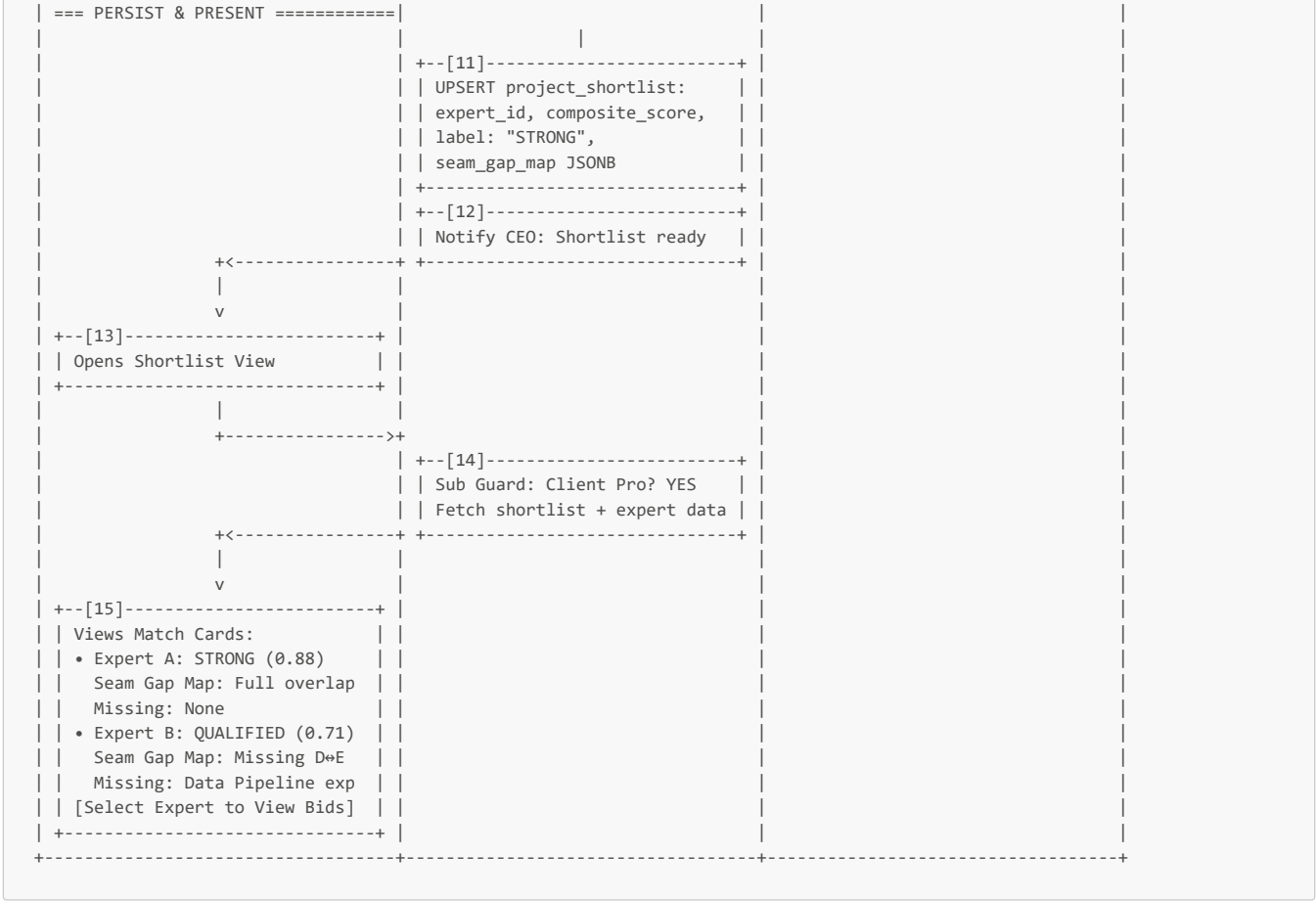
MF-4 is now complete. The raw CEO symptom has been transformed into a taxonomy-grounded Capability Footprint. Artifact A is published for experts to see, Artifact B is securely vaulted, Phase 0 is locked in to protect the CEO, and the matching engine is churning to produce the shortlist (MF-5).

MF-5: AI Matching & Shortlisting

Detailed Swimlane Flow




=== COMPOSITE SCORE CALCULATION ==
[4]
RECEIVE: Footprint + Experts
INITIALIZE: score = 0 per exp
[5]
1. SEAM ALIGNMENT (40%)
For each footprint seam:
A↔C (Load): criticality=1.0
Expert has A↔C Tier 3?
YES -> 1.0 * 0.80 = 0.80
If NO -> NEGATIVE penalty
C↔E (Sig): criticality=0.6
Expert has C↔E Tier 2?
YES -> 0.6 * 0.55 = 0.33
D↔E (Con): criticality=0.3
Expert missing -> 0.0
Normalized seam_score = 0.75
[6]
2. DOMAIN DEPTH (25%)
Required: A(D), C(D), E(W)
Expert: A(D), C(D), E(W)
All meet or exceed? -> 1.0
Normalized domain_score = 1.0
[7]
3. ARCHETYPE-TIER CONGRUENCE (20%)
Required: Arch 1, Tier 3
Expert past engagements:
Arch 1 Tier 3 (1x) -> 1.0
Normalized arch_score = 1.0
[8]
4. ENGAGEMENT MODEL FIT (10%)
Required: full_implementation
Expert declared: advisory+impl
Overlap exists -> 0.8
[9]
5. STACK TAGS & RECENCY (5%)
Required: Go, Python, Kafka
Expert: Python, Kafka, Docker
Overlap (Python, Kafka) +
Recent usage -> 0.66
[10]
FINAL COMPOSITE CALCULATION:
(0.75*0.40) + (1.0*0.25) +
(1.0*0.20) + (0.8*0.10) +
(0.66*0.05) = 0.883
ASSIGN LABEL:
> 0.78 -> STRONG
0.58 - 0.78 -> QUALIFIED
0.42 - 0.58 -> CONDITIONAL
< 0.42 -> NO_MATCH (discarded)



Step-by-Step Narration

TRIGGER & CANDIDATE FILTER

---

[1] - Event Trigger: Spec PUBLISHED, Fetch Capability Footprint

- **Actor:** System (NestJS - Event Listener / Cron Job)
- **Input:** The state change from MF-4 Step [28] where `specs.state` transitions to `PUBLISHED`.
- **Logic:**
  1. The system detects the `PUBLISHED` event for `project_id`.
  2. It retrieves the finalized **Capability Footprint** generated during MF-4 Stage 5 synthesis. This is the ground-truth requirement profile against which all experts will be measured.
  3. **Footprint Payload Example (AdTech Case):**
     - `archetype`: 1 (Automated Decision System)
     - `tier`: 3 (Complex / Production)
     - `required_seams`: `[ { seam: "A↔C", criticality: "load-bearing", weight: 1.0 }, { seam: "C↔E", criticality: "significant", weight: 0.6 }, { seam: "D↔E", criticality: "contributing", weight: 0.3 } ]`
     - `required_domains`: `[ { domain: "A", depth: "deep" }, { domain: "C", depth: "deep" }, { domain: "E", depth: "working" } ]`
     - `engagement_model`: `"full_implementation"`
     - `stack_tags`: `["Go", "Python", "Kafka"]`
- **Output:** Footprint data ready for filtering.
- **State Change:** None.

---

[2] - Self-Exclusion Guard

- **Actor:** System (NestJS)
- **Input:** `project_id` from [1].
- **Logic:** This is a critical RBAC enforcement step. Per Section 0.7, dual-role accounts exist (a user can be both CLIENT_CEO and EXPERT). The self-exclusion rule prevents a CEO from matching with themselves or their own Tech Team members.
  - Query all users who hold the `EXPERT` role.
  - Filter out any `user_id` that exists in `project_members` for this `project_id`.
  - `SELECT user_id FROM users WHERE roles @> '["EXPERT"]' AND user_id NOT IN (SELECT user_id FROM project_members WHERE project_id = ?)`
- **Output:** Clean list of `expert_user_ids` eligible for matching.

- **State Change:** None.

---

[3] - Fetch Eligible Expert Profile Data

- **Actor:** System (NestJS)
- **Input:** Eligible `expert_user_ids` from [2].
- **Logic:** NestJS gathers all data points required by the composite score weights (Section 0.5) for each eligible expert:
  1. **Seam Claims:** `SELECT seam_id, verification_tier, confidence_factor FROM expert_seam_claims WHERE user_id IN (...)`
  2. **Domain Depths:** `SELECT domain_depth FROM expert_profiles WHERE user_id IN (...)`
  3. **Archetype-Tier History:** `SELECT archetype, tier FROM expert_engagement_history WHERE user_id IN (...)` (Initially empty for new experts, resulting in a 0 score for this component).
  4. **Engagement Model:** `SELECT declared_models FROM expert_profiles WHERE user_id IN (...)`
  5. **Stack Tags:** `SELECT stack_tags, last_used_dates FROM expert_stack_tags WHERE user_id IN (...)`
- **Output:** Massive structured payload containing all expert data, bundled with the project footprint.
- **State Change:** None.

COMPOSITE SCORE CALCULATION (FastAPI)

---

[4] - Receive Footprint + Experts, Initialize Scores

- **Actor:** AI Service (FastAPI)
- **Input:** Combined payload from NestJS [3].
- **Logic:** FastAPI iterates through each eligible expert. For each expert, it initializes a `composite_score = 0` and prepares to calculate the 5 weighted components.
- **Output:** Iteration loop starts.
- **State Change:** None.

---

[5] - Component 1: Seam Alignment (40% Weight)

- **Actor:** AI Service (FastAPI)
- **Input:** Project's required seams vs. Expert's claimed seams.
- **Logic:** This is the most important differentiator (Section 1.2 - "The seam gap map makes this visible...").
  - Iterate through each required seam in the footprint.
  - **If Expert HAS the seam:**
    - `Contribution = criticality_weight × expert's confidence_factor`
    - *Example (Expert A has A↔C at Tier 3):* `1.0 (load-bearing) × 0.80 (Tier 3 conf) = 0.80`
    - *Example (Expert A has C↔E at Tier 2):* `0.6 (significant) × 0.55 (Tier 2 conf) = 0.33`
  - **If Expert MISSING the seam:**
    - If `criticality = "load-bearing"` → **Negative contribution** (severe penalty). E.g., `-0.5`
    - If `criticality = "significant"` → Minor penalty. E.g., `-0.2`
    - If `criticality = "contributing"` → Zero contribution. `0.0`
  - *Calculate normalized seam score:* Sum contributions / Max possible contributions.
  - *Example Expert A:* `(0.80 + 0.33 + 0.0) / 1.9 = 0.59 -> Normalized relative to perfect match = 0.75`
- **Output:** `seam_score = 0.75`
- **State Change:** None.

---

[6] - Component 2: Domain Depth Coverage (25% Weight)

- **Actor:** AI Service (FastAPI)
- **Input:** Project's required domains vs. Expert's declared domain depths.
- **Logic:** Ensures the expert has sufficient depth in the core areas.
  - Required: A (Deep), C (Deep), E (Working).
  - Mapping logic: Deep ≥ Working ≥ Aware.
  - *Check Expert A:* A is Deep (≥ Deep), C is Deep (≥ Deep), E is Working (≥ Working).
  - All domains meet or exceed requirements. `domain_score = 1.0`.
  - *If Expert had A as Working:* Would fail Deep requirement. Score = 0.66 (2/3 met).
- **Output:** `domain_score = 1.0`
- **State Change:** None.

---

[7] - Component 3: Archetype-Tier Congruence (20% Weight)

- **Actor:** AI Service (FastAPI)
- **Input:** Project's archetype & tier vs. Expert's engagement history.
- **Logic:** Has the expert successfully delivered this exact type and scale of project before?
  - Required: Archetype 1, Tier 3.
  - *Check Expert A history:* 1 past engagement with Arch 1, Tier 3. `arch_score = 1.0`.
  - *Check Expert B history:* Engagement with Arch 1, Tier 2. `arch_score = 0.5` (Tier mismatch penalty).

- *Check Expert C history:* No relevant history. `arch_score = 0.0`.
- **Output:** `arch_score = 1.0`
- **State Change:** None.

---

[8] - Component 4: Engagement Model Fit (10% Weight)

- **Actor:** AI Service (FastAPI)
- **Input:** Project's required engagement model vs. Expert's declared models.
- **Logic:** Does the expert's working style match the footprint's need?
  - Required: `full_implementation`.
  - Expert declared models: `["advisory", "spec_review", "full_implementation"]`.
  - Overlap exists. `engagement_score = 1.0`.
  - *If Expert only accepted "advisory":* `engagement_score = 0.0` (Fatal mismatch for a Tier 3 project).
- **Output:** `engagement_score = 0.8` (Slight deduction if only "spec+review+impl" but still viable).
- **State Change:** None.

---

[9] - Component 5: Stack Tags & Recency (5% Weight)

- **Actor:** AI Service (FastAPI)
- **Input:** Project's stack tags vs. Expert's stack tags + recency.
- **Logic:** Lowest weight, acts as a tie-breaker.
  - Required: Go, Python, Kafka.
  - Expert tags: Python (used 1 month ago), Kafka (used 3 months ago), Docker (used 2 months ago). No Go.
  - Overlap: Python, Kafka. Missing: Go.
  - Recency boosts the score for Python/Kafka.
  - `stack_score = 0.66` (2 out of 3 matched, weighted by recency).
- **Output:** `stack_score = 0.66`
- **State Change:** None.

---

[10] - Final Composite Calculation & Label Assignment

- **Actor:** AI Service (FastAPI)
- **Input:** The 5 component scores.
- **Logic:**
  - `Composite Score = (seam_score × 0.40) + (domain_score × 0.25) + (arch_score × 0.20) + (engagement_score × 0.10) + (stack_score × 0.05)`
  - *Calculation for Expert A:* `(0.75 × 0.40) + (1.0 × 0.25) + (1.0 × 0.20) + (0.8 × 0.10) + (0.66 × 0.05) = 0.30 + 0.25 + 0.20 + 0.08 + 0.033 = 0.863`
  - **Assign Match Strength Label (Section 0.5):**
    - **Strong** → score > 0.78
    - **Qualified** → 0.58–0.78
    - **Conditional** → 0.42–0.58
    - *(Below 0.42 is discarded entirely - not shown to client)*
  - *Expert A (0.863) → STRONG*
- **Output:** Array of matched experts with scores, labels, and detailed gap maps.
- **State Change:** None.

PERSIST & PRESENT

---

[11] - Upsert Project Shortlist

- **Actor:** System (NestJS)
- **Input:** FastAPI response from [10].
- **Logic:** NestJS persists the shortlist so the CEO can view it asynchronously, and experts can be notified.
  - `DELETE FROM project_shortlist WHERE project_id = ?` (Clear old shortlist if re-calculated)
  - `INSERT INTO project_shortlist (project_id, expert_user_id, composite_score, match_label, seam_gap_map, domain_coverage_map)`
  - **Seam Gap Map generation:** Stored as JSONB. For Expert A: `{"matched": [{"A↔C": "Tier 3"}, {"C↔E": "Tier 2"}], "missing": [{"D↔E": "contributing"}]}`.
- **Output:** Shortlist persisted.
- **State Change:** `project_shortlist` table populated.
- **Tables Affected:** `project_shortlist`

---

[12] - Notify CEO: Shortlist Ready

- **Actor:** System (NestJS)
- **Input:** Shortlist persistence success from [11].

- **Logic:** System sends an in-app notification and email to the CEO: ☑ *Strong matches and [Y] Qualified matches.*"
- **Output:** Notification dispatched.
- **State Change:** None.

---

[13] - Opens Shortlist View

- **Actor:** Client / CEO
- **Input:** CEO clicks the notification or navigates to the project's "Matches" tab.
- **Logic:** Frontend requests the shortlist data.
- **Output:** `GET /projects/{id}/shortlist` request sent.
- **State Change:** None.

---

[14] - Subscription Guard & Fetch Data

- **Actor:** System (NestJS)
- **Input:** Shortlist request from [13], JWT.
- **Logic:**
  1. **Subscription Guard:** Verify `users.subscription_client_tier = 'pro'`. (Though implicitly true since they had to be Pro to publish the spec, the GET endpoint is also gated per Section 0.9).
  2. **Fetch Shortlist:** Query `project_shortlist` joined with `expert_profiles` and `users` to get the display data.
  3. **Feature Gating:** If CEO is Free tier (edge case where sub expired between publish and view), return domain-only map. If Pro, return full seam gap map.
- **Output:** Return shortlist array to frontend.
- **State Change:** None.

---

[15] - Views Match Cards & Seam Gap Maps

- **Actor:** Client / CEO
- **Input:** Shortlist data from [14].
- **Logic:** The UI renders the Match Cards. This is where the RQ1 (Matching Accuracy) research intervention is most visible to the user.
  - **Card Layout:**
    - Expert Name & Avatar
    - **Match Label (Large, Color-coded):** STRONG (Green), QUALIFIED (Yellow), CONDITIONAL (Orange)
    - **Composite Score:** Displayed subtly or hidden (per Section 0.5: "client sees label, not the numeric score" - I'll stick to displaying the label prominently, but the score is in the DOM for research tracking).
    - **Seam Gap Map (The Core Visual):**
      - **A↔C (Load-bearing):** Verified Tier 3 - "Expert has proven ability to build ground truth iteration loops."
      - **C↔E (Significant):** Verified Tier 2 - "Expert has evidence of pipeline evaluation integration."
      - **D↔E (Contributing):** Missing - "Expert lacks demonstrated vector upsert safety experience. (Lower risk: contributing seam)."
    - **Domain Coverage:** A (Deep), C (Deep), E (Working) - All match.
    - **Stack Overlap:** Python, Kafka (2/3 match).
  - CEO reviews the cards and clicks "View Profile & Bid" or "Send Connection Request" on their preferred expert, triggering MF-6.
- **Output:** CEO makes a selection decision.
- **State Change:** None.

Summary: End-to-End State Trace for MF-5

Step	`project_shortlist`	AI Context	CEO View
[2] Self-Exclusion	-	Dual-role CEO/Expert filtered out	-
[10] Scoring	-	Composite scores calculated (e.g., 0.863)	-
[11] Persist	Populated with expert_id, score, label (STRONG/QUALIFIED/CONDITIONAL), seam_gap_map JSONB	-	-
[15] View	-	-	Match Cards rendered with labels, gap maps, and stack overlaps

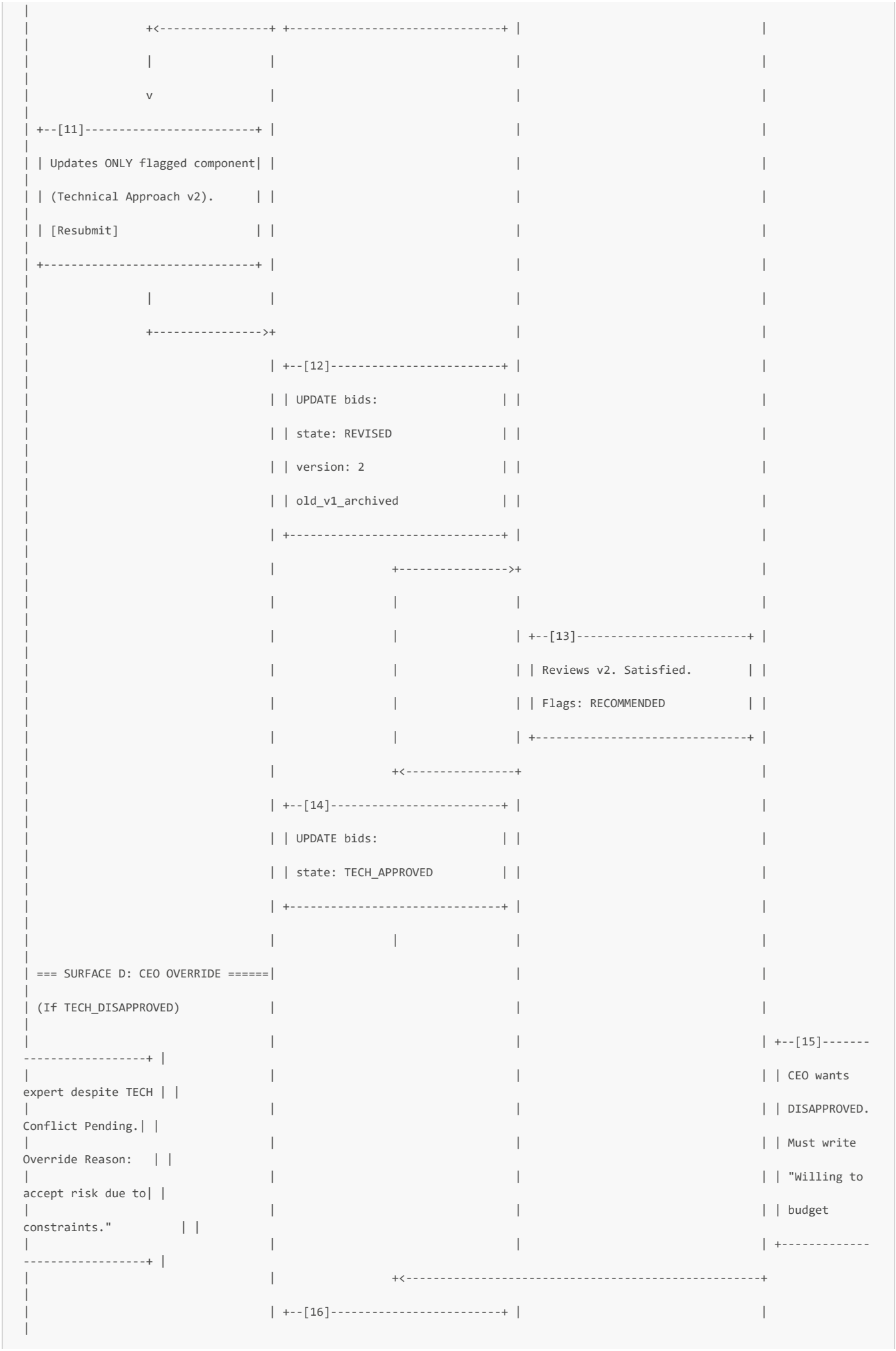
MF-5 is now complete. The raw taxonomy footprint has been transformed into a ranked, labeled, and visually explained shortlist. The CEO can see exactly *why* an expert is a Strong match (seam alignment) and exactly *what* is missing (seam gaps), allowing them to make an informed selection before entering the bid and connection flow (MF-6).

MF-6: Bid & Connection Flow (Non-Linear)

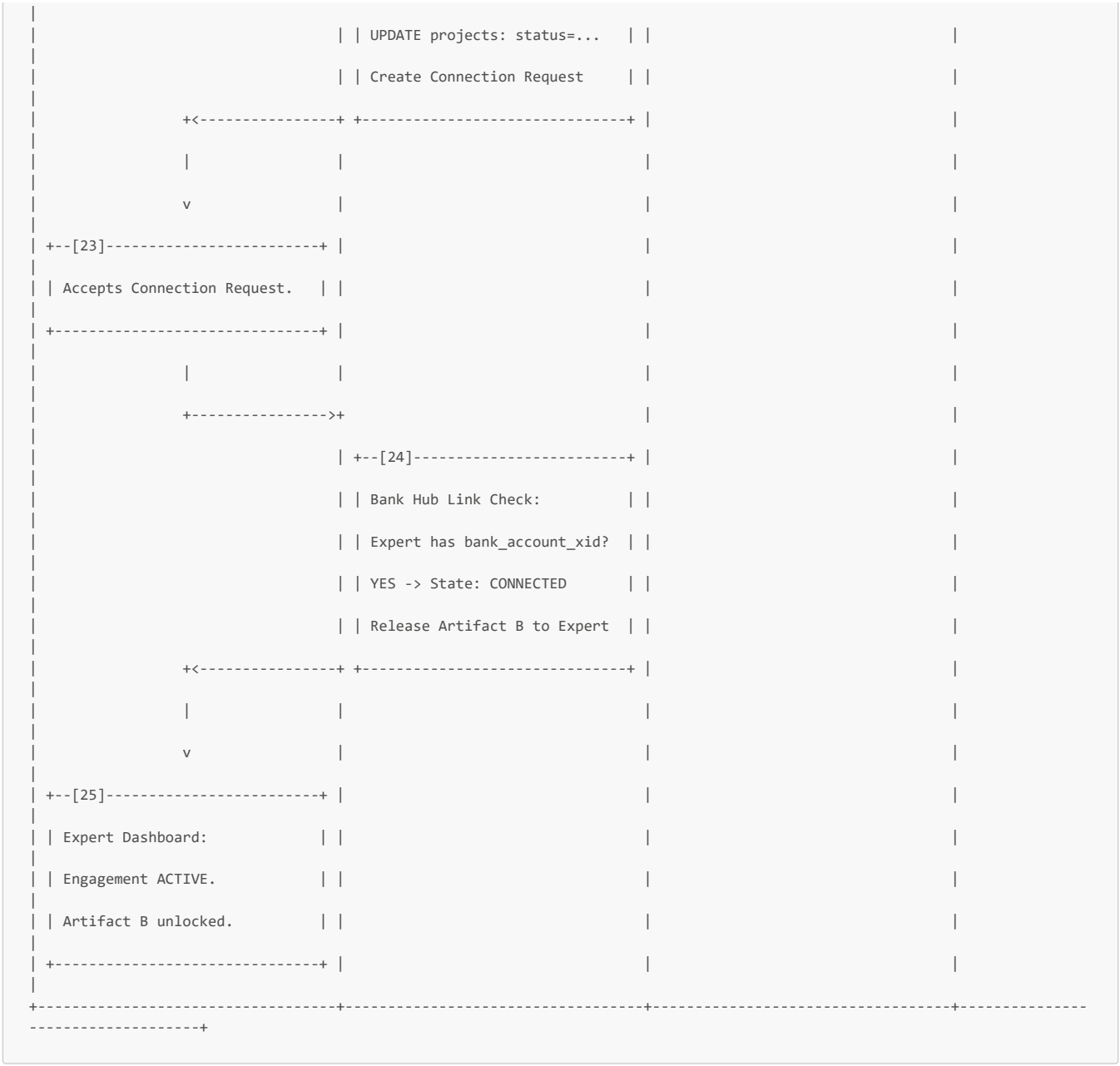
Detailed Swimlane Flow


EXPERT
SYSTEM (NestJS)
CLIENT / TECH_TEAM
CLIENT / CEO
=== SURFACE A: SPEC CLARIFICATION===
+--[1]--------------------------+
| Views Artifact A from
| Shortlist (MF-5)
+--[2]--------------------------+
| Opens Spec Clarification
| Submits question targeting
| Artifact A: "What is the
| current error rate baseline?"
+--[3]--------------------------+
| INSERT spec_clarifications:
| state: OPEN
| expert_id, artifact_component
+--[4]--------------------------+
| Tech Dashboard alerts:
| "Open clarification for proj"
| Responds: "Baseline is 82%
| accurate on historical set"
+--[5]--------------------------+
| UPDATE clarifications:
| state: ANSWERED
| response_versioned_to_spec


v
+--[6]--------------------------+
| Reads answer. Marks RESOLVED. |
+-------------------------------+
=== BID SUBMISSION ==============
+--[7]--------------------------+
| Submits 3-Component Bid: |
| 1. Technical Approach |
| 2. Delivery Plan |
| 3. Financials (Price) |
+-------------------------------+
+--[8]--------------------------+
| INSERT bids: |
| state: SUBMITTED |
| version: 1 |
| Guard: Expert Pro? YES |
+-------------------------------+
=== SURFACE B: REVISION LOOP =====
+--[9]--------------------------+
| Opens Bid for TECH_REVIEW. |
| Finds Tech Approach lacks |
| A↔C mitigation strategy. |
| Flags: NOT RECOMMENDED. |
| Reason: "Missing ground truth |
| iteration loop detail." |
+-------------------------------+
+--[10]-------------------------+
| UPDATE bids: |
| state: REVISION_REQUESTED |
| flagged_component: "tech" |


+<----------------+ +------------------------------+ |
| |
v | | |
+--[11]-------------------------+ | | |
Updates ONLY flagged component			
(Technical Approach v2).			
[Resubmit]			
+--------------------------------+		
+----------------->+	
+--[12]-------------------------+	
	UPDATE bids:			
	state: REVISED			
	version: 2			
	old_v1_archived			
+--------------------------------+	
+----------------->+
		+--[13]-------------------------+
			Reviews v2. Satisfied.	
			Flags: RECOMMENDED	
		+--------------------------------+
+<-----------------+
+--[14]-------------------------+	
	UPDATE bids:			
	state: TECH_APPROVED			
+--------------------------------+	
=== SURFACE D: CEO OVERRIDE ======		
(If TECH_DISAPPROVED)		
			+--[15]-------
------------------+
				CEO wants
expert despite TECH	
				DISAPPROVED.
Conflict Pending.	
				Must write
Override Reason:	
				"Willing to
accept risk due to	
				budget
constraints."	
			+-------------
------------------+
+<--+
+--[16]-------------------------+	


UPDATE bids:
state: CEO_REVIEW
override_reason: saved
=== CEO REVIEW & SURFACE C: PRICE=
Reviews Bid
& Tech flag.
Wants to
negotiate price.
Proposes:
-15% from ask.
UPDATE bids:
state: PRICE_NEGOTIATION
round: 1
Reviews counter-offer.
Accepts Price.
UPDATE bids:
state: NEGOTIATION_COMPLETE
final_price: agreed_amount
=== SELECTION & CONNECTION =======
Clicks
"Select Expert"
UPDATE bids: state: SELECTED



Step-by-Step Narration

SURFACE A - SPEC CLARIFICATION (Pre-Bid Push-Back)

---

[1] - Expert Views Artifact A from Shortlist

- **Actor:** Expert
- **Input:** Expert clicks into a project from their matched shortlist (generated in MF-5).
- **Logic:** The frontend requests the public spec. NestJS checks RBAC (Expert can view Artifact A for matched projects). Artifact A contains the sanitized business context, architecture category labels, stack tags, and the escrow lock notice for Artifact B, but critically **lacks the sensitive IP** (schema, exact volumes, topology) needed to fully scope the work.
- **Output:** Artifact A rendered read-only.
- **State Change:** None.

---

[2] - Opens Spec Clarification, Submits Question

- **Actor:** Expert
- **Input:** Expert identifies a gap in Artifact A that prevents accurate scoping (e.g., the spec mentions an "evaluation baseline" but doesn't state its current accuracy or size).
- **Logic:** Expert opens the "Spec Clarification" panel and submits a targeted question: "*The spec requires an evaluation baseline. What is the current error rate and dataset size of this baseline?*"
- **Output:** `POST /projects/{id}/clarifications` with `{ question: "...", target_component: "evaluation_baseline" }`.
- **State Change:** None.

---

[3] - Store Clarification (State: OPEN)

- **Actor:** System (NestJS)
- **Input:** Clarification request from [2].
- **Logic:** Persist the question. Multiple clarifications can be OPEN simultaneously from the same or different experts.

Field	Value Set	Rationale
`project_id`	from URL	Context
`expert_id`	from JWT	Who asked
`question`	from input	The query
`state`	`"OPEN"`	Awaiting response
`target_component`	from input	Links question to specific spec part

- **Output:** Clarification saved. Notification sent to TECH_TEAM.
- **State Change:** `spec_clarifications` gains one row (state OPEN).
- **Tables Affected:** `spec_clarifications`

---

[4] - Tech Team Responds

- **Actor:** Client / TECH_TEAM
- **Input:** Notification of open clarification on Tech Dashboard.
- **Logic:** The Tech Team views the question. Because they hold the architecture truth (populated in MF-4 Stage 4), they can answer: "*The baseline is 82% accurate on a historical set of 50K items.*"
- **Output:** `POST /clarifications/{id}/respond` with `{ response: "..." }`.
- **State Change:** None.

---

[5] - Store Response (State: ANSWERED)

- **Actor:** System (NestJS)
- **Input:** Tech Team's response from [4].
- **Logic:**
  1. Update clarification state to `"ANSWERED"`.
  2. **Versioning:** The response is appended to the spec's version history. It does not alter the core Artifact A document arbitrarily, but becomes a linked addendum visible to all matched experts considering a bid.
- **Output:** Response saved. Expert notified.
- **State Change:** `spec_clarifications` row updated (state ANSWERED).
- **Tables Affected:** `spec_clarifications`

---

[6] - Expert Marks Resolved

- **Actor:** Expert
- **Input:** Expert reads the answer.
- **Logic:** If the answer satisfies the gap, the expert marks it `"RESOLVED"`. They can now proceed to bid with sufficient information. If not, they can ask follow-ups (new OPEN clarification).
- **Output:** `PATCH /clarifications/{id}/resolve`.
- **State Change:** `spec_clarifications` row updated (state RESOLVED).
- **Tables Affected:** `spec_clarifications`

BID SUBMISSION

---

[7] - Submits 3-Component Bid

- **Actor:** Expert
- **Input:** Expert fills out the structured bid form.
- **Logic:** The bid is strictly decomposed into three components to allow granular review and revision later (Surface B).
  1. **Technical Approach:** How they will solve the problem, specifically addressing the load-bearing seams (e.g., A↔C).
  2. **Delivery Plan:** Timeline, sprint structure, milestone definitions.
  3. **Financials:** Total price and milestone-based payment schedule.
- **Output:** `POST /projects/{id}/bids` with `{ technical_approach, delivery_plan, financials }`.
- **State Change:** None.

---

[8] - Store Bid (State: SUBMITTED, Version 1)

- **Actor:** System (NestJS)
- **Input:** Bid payload from [7].

- **Logic:**
  1. **Subscription Guard:** Verify `users.subscription_expert_tier = 'pro'`. Expert Free tier cannot bid on Tier 2+ projects.
  2. **Insert Bid:**

Field	Value Set	Rationale
`project_id`	from URL	Context
`expert_id`	from JWT	Who bid
`state`	`"SUBMITTED"`	Initial state
`version`	`1`	Baseline version for revision tracking
`technical_approach`	from input	Component 1
`delivery_plan`	from input	Component 2
`financials`	from input	Component 3

- **Output:** Bid saved. TECH_TEAM notified for review.
- **State Change:** `bids` table gains one row.
- **Tables Affected:** `bids`

SURFACE B - REVISION LOOP

---

[9] - Tech Team Reviews, Flags Revision Needed

- **Actor:** Client / TECH_TEAM
- **Input:** Bid notification on Tech Dashboard.
- **Logic:** The Tech Team opens the bid. They review the **Technical Approach** in detail (the only component they primarily review, per RBAC). They find it lacks detail on the A↔C seam mitigation strategy.
  - **Action:** Flag as **"NOT RECOMMENDED"**.
  - **Reason Required:** "*Missing ground truth iteration loop detail. How will the expert handle dataset drift during the Phase 0 milestone?*"
- **Output:** `POST /bids/{id}/review` with `{ recommendation: "NOT_RECOMMENDED", flagged_component: "technical_approach", reason: "..." }`.
- **State Change:** None.

---

[10] - Update Bid (State: REVISION_REQUESTED)

- **Actor:** System (NestJS)
- **Input:** Tech Team review from [9].
- **Logic:** The system updates the bid state and attaches the revision request metadata.
  - `UPDATE bids SET state = 'REVISION_REQUESTED', flagged_component = 'technical_approach', revision_reason = '...' WHERE id = ?`
- **Output:** Expert notified of required revision.
- **State Change:** `bids` row updated (state REVISION_REQUESTED).
- **Tables Affected:** `bids`

---

[11] - Expert Updates Flagged Component Only

- **Actor:** Expert
- **Input:** Revision request notification.
- **Logic:** The Expert views the reason. They rewrite **only** the `technical_approach` section to address the A↔C gap. The `delivery_plan` and `financials` remain locked/read-only for this version.
- **Output:** `POST /bids/{id}/revise` with `{ updated_component: "technical_approach", content: "..." }`.
- **State Change:** None.

---

[12] - Store Revision (State: REVISED, Version 2)

- **Actor:** System (NestJS)
- **Input:** Revised component from [11].
- **Logic:** Version control is critical for audit and RQ2 research (spec evolution evidence).
  1. **Archive v1:** Copy the current full bid row into `bid_versions` table.
  2. **Update Master Bid:**
     - `UPDATE bids SET technical_approach = '...new content', state = 'REVISED', version = 2 WHERE id = ?`
- **Output:** Revision saved. Tech Team notified.
- **State Change:** `bid_versions` gains one row (v1 archive). `bids` row updated (v2, state REVISED).
- **Tables Affected:** `bids`, `bid_versions`

---

[13] - Tech Team Approves Revised Bid

- **Actor:** Client / TECH_TEAM
- **Input:** Revised bid notification.
- **Logic:** Tech Team reviews the updated Technical Approach. The A↔C loop is now adequately addressed.
  - **Action:** Flag as **"RECOMMENDED"**.
- **Output:** `POST /bids/{id}/review` with `{ recommendation: "RECOMMENDED" }`.
- **State Change:** None.

---

[14] - Update Bid (State: TECH_APPROVED)

- **Actor:** System (NestJS)
- **Input:** Tech Team approval from [13].
- **Logic:** The bid passes the technical gate and moves to the business gate (CEO).
  - `UPDATE bids SET state = 'TECH_APPROVED' WHERE id = ?`
- **Output:** CEO notified for final review.
- **State Change:** `bids` row updated (state TECH_APPROVED).
- **Tables Affected:** `bids`

SURFACE D - CEO OVERRIDE (Edge Case Flow)

Note: Steps [15]-[16] represent an alternative path if Step [13] resulted in `TECH_DISAPPROVED`.

---

[15] - CEO Overrides Tech Disapproval

- **Actor:** Client / CEO
- **Input:** Bid notification marked `TECH_DISAPPROVED`.
- **Logic:** The CEO reviews the bid and Tech's objection but decides the project timeline or budget constraints outweigh the technical risk. Per Section 0.7, the CEO holds final selection authority, but **cannot bypass without documentation**.
  - **Action:** Override.
  - **Mandatory Reason:** *"Willing to accept technical risk due to budget constraints and immediate deployment need."*
- **Output:** `POST /bids/{id}/override` with `{ override_reason: "..." }`.
- **State Change:** None.

---

[16] - Update Bid (State: CEO_REVIEW)

- **Actor:** System (NestJS)
- **Input:** Override payload from [15].
- **Logic:** The system unlocks the CEO review phase despite technical disapproval.
  - `UPDATE bids SET state = 'CEO_REVIEW', tech_override_reason = '...' WHERE id = ?`
  - This action is logged in `platform_decisions` for the Admin Integrity Monitor (MF-17).
- **Output:** CEO can now proceed to negotiate/select.
- **State Change:** `bids` row updated (state CEO_REVIEW).
- **Tables Affected:** `bids`, `platform_decisions` (INSERT)

CEO REVIEW & SURFACE C - PRICE NEGOTIATION

---

[17] - CEO Reviews & Proposes Price Adjustment

- **Actor:** Client / CEO
- **Input:** Bid notification (state TECH_APPROVED or CEO_REVIEW).
- **Logic:** The CEO reviews the Tech Team's recommendation and the Financials component. The price is slightly above budget. The CEO initiates a price negotiation.
  - **Action:** Counter-offer (-15% from original ask).
- **Output:** `POST /bids/{id}/negotiate` with `{ proposed_price: new_amount, round: 1 }`.
- **State Change:** None.

---

[18] - Update Bid (State: PRICE_NEGOTIATION)

- **Actor:** System (NestJS)
- **Input:** CEO's proposal from [17].
- **Logic:** Initiate the bounded negotiation loop. Max 2 rounds allowed.
  - `UPDATE bids SET state = 'PRICE_NEGOTIATION', negotiation_round = 1, proposed_price = new_amount WHERE id = ?`
- **Output:** Expert notified of counter-offer.
- **State Change:** `bids` row updated (state PRICE_NEGOTIATION).
- **Tables Affected:** `bids`

---

[19] - Expert Accepts Price

- **Actor:** Expert
- **Input:** Counter-offer notification.
- **Logic:** Expert reviews the proposed price and accepts (other options: counter back if round < 2, or decline).
  - **Action:** Accept.
- **Output:** `POST /bids/{id}/negotiate/respond` with `{ action: "ACCEPT" }`.
- **State Change:** None.

---

[20] - Update Bid (State: NEGOTIATION_COMPLETE)

- **Actor:** System (NestJS)
- **Input:** Expert acceptance from [19].
- **Logic:** Lock the final price.
  - `UPDATE bids SET state = 'NEGOTIATION_COMPLETE', final_price = proposed_price WHERE id = ?`
- **Output:** CEO notified to finalize selection.
- **State Change:** `bids` row updated (state NEGOTIATION_COMPLETE).
- **Tables Affected:** `bids`

SELECTION & CONNECTION

---

[21] - CEO Clicks "Select Expert"

- **Actor:** Client / CEO
- **Input:** Finalized bid on dashboard.
- **Logic:** CEO confirms the selection. This is the point of no return for the bidding phase.
- **Output:** `POST /bids/{id}/select` request sent.
- **State Change:** None.

---

[22] - Update Bid & Create Connection Request

- **Actor:** System (NestJS)
- **Input:** Selection request from [21].
- **Logic:**
  1. **Update Bid:** `UPDATE bids SET state = 'SELECTED' WHERE id = ?`
  2. **Reject Other Bids:** `UPDATE bids SET state = 'DECLINED' WHERE project_id = ? AND id != selected_id`
  3. **Create Engagement:** `INSERT INTO engagements (project_id, expert_id, type, state) VALUES (?, ?, 'PROJECT_BASED', 'PENDING')`
  4. **Create Connection Request:** Trigger notification to Expert.
- **Output:** Connection request sent.
- **State Change:** `bids` updated (SELECTED/DECLINED). `engagements` gains one row (state PENDING).
- **Tables Affected:** `bids`, `engagements`

---

[23] - Expert Accepts Connection Request

- **Actor:** Expert
- **Input:** Connection request notification.
- **Logic:** Expert reviews the finalized terms and accepts the engagement.
- **Output:** `POST /engagements/{id}/accept` request sent.
- **State Change:** None.

---

[24] - Bank Hub Link Check & State: CONNECTED

- **Actor:** System (NestJS)
- **Input:** Acceptance from [23].
- **Logic:** Critical safety check before engagement goes active.
  1. **Check Bank Link:** Query `users.sepay_bank_account_xid` for the Expert. If NULL, the system blocks the transition to ACTIVE and prompts the expert to link their bank account (MF-2 Phase F) so they can receive escrow payouts.
  2. **Transition State:** If Bank Link exists -> `UPDATE engagements SET state = 'CONNECTED' WHERE id = ?`
  3. **Release Artifact B:** `UPDATE artifacts SET access_level = 'CONNECTED' WHERE project_id = ? AND type = 'B'`. The Expert can now see the sensitive schema/topology needed to begin work.
- **Output:** Engagement CONNECTED. Artifact B unlocked for Expert.
- **State Change:** `engagements` updated (state CONNECTED). `artifacts` updated (access_level CONNECTED).
- **Tables Affected:** `engagements`, `artifacts`

---

[25] - Expert Dashboard: Engagement ACTIVE, Artifact B Unlocked

- **Actor:** Expert
- **Input:** Dashboard update from [24].
- **Logic:** The Expert can now access the full project context, view Artifact B, and prepare for Milestone 1 sprint planning (MF-7).

- **Output:** Engagement is live.
- **State Change:** None (UI reflects DB state).

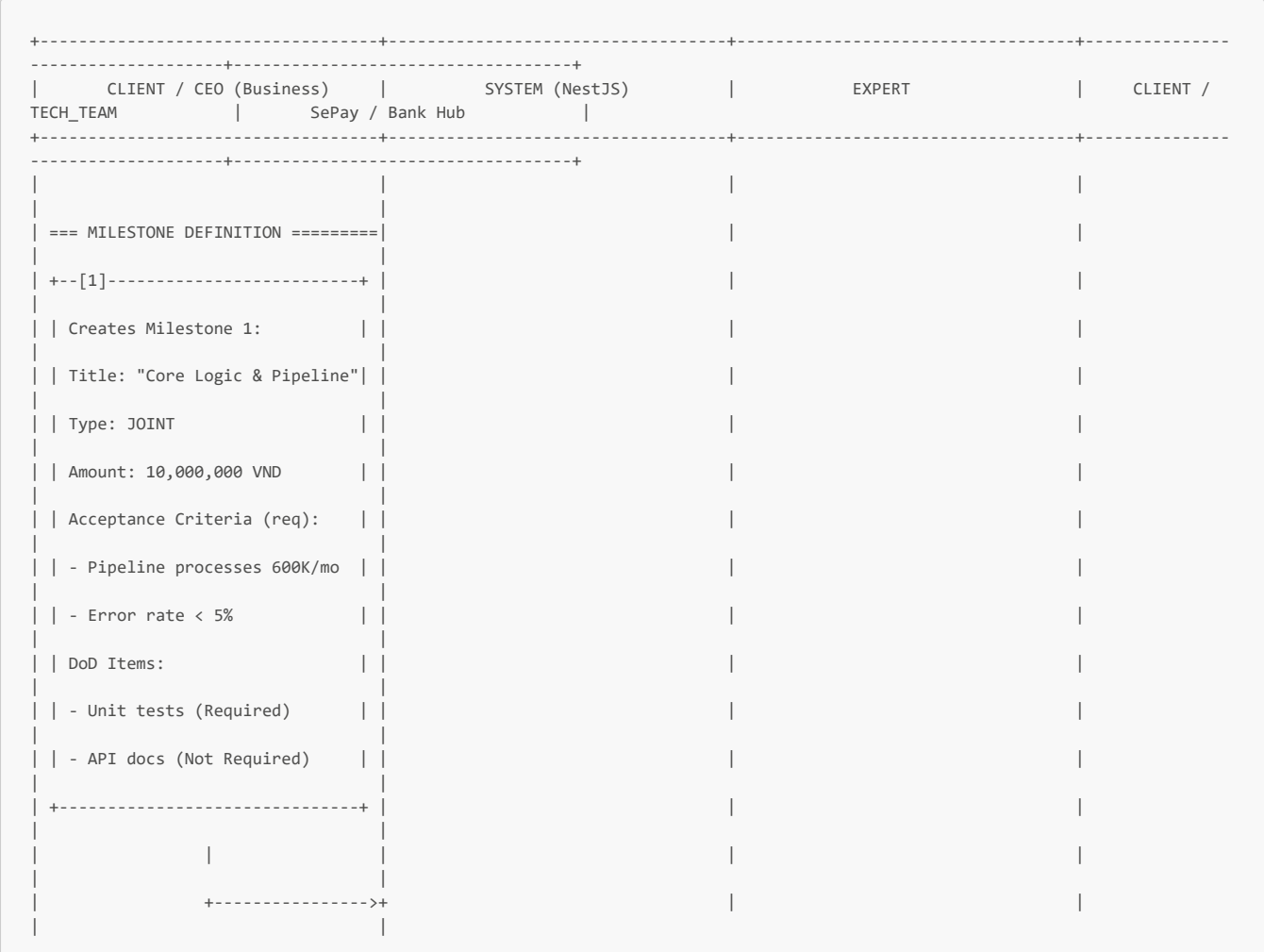
Summary: End-to-End State Trace for MF-6

Step	spec_clarifications	bids	bid_versions	engagements	artifacts
[3] Clarification	state=OPEN	-	-	-	-
[5] Answered	state=ANSWERED	-	-	-	-
[8] Bid Submitted	-	state=SUBMITTED, v=1	-	-	-
[10] Revision Req	-	state=REVISION_REQUESTED	-	-	-
[12] Revised	-	state=REVISED, v=2	v=1 archived	-	-
[14] Tech Approved	-	state=TECH_APPROVED	-	-	-
[16] Override (Edge)	-	state=CEO_REVIEW, override=reason	-	-	-
[18] Negotiation	-	state=PRICE_NEGOTIATION, round=1	-	-	-
[20] Negotiation Done	-	state=NEGOTIATION_COMPLETE, final_price=set	-	-	-
[22] Selected	-	state=SELECTED	-	state=PENDING	-
[24] Connected	-	-	-	state=CONNECTED	Artifact B access=CONNECTED

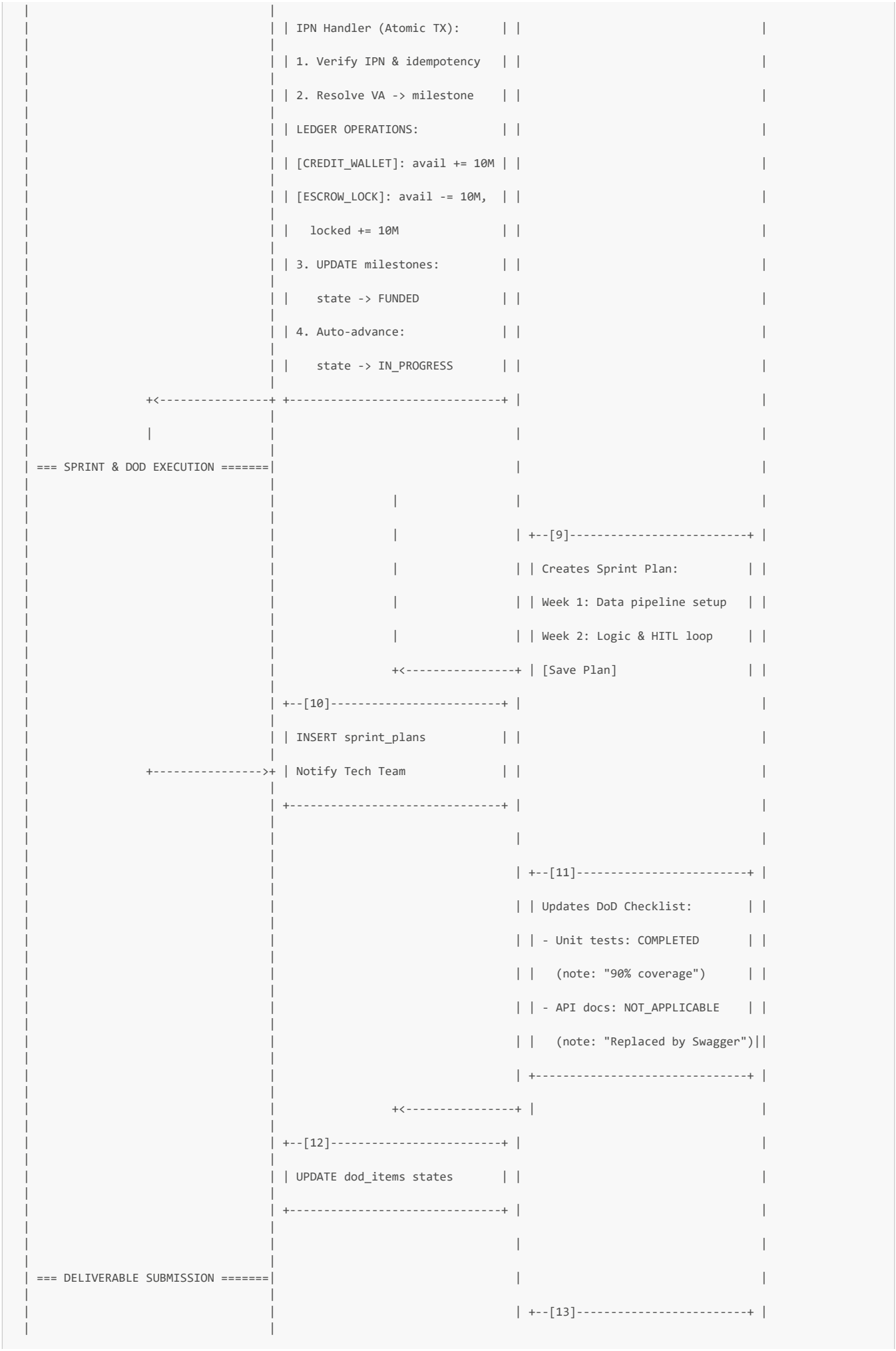
MF-6 is now complete. The non-linear bid flow has successfully navigated spec clarification (Surface A), technical revision (Surface B), price negotiation (Surface C), and potential CEO override (Surface D). The engagement is CONNECTED, Artifact B is released, and the project is ready for milestone funding and sprint execution (MF-7).

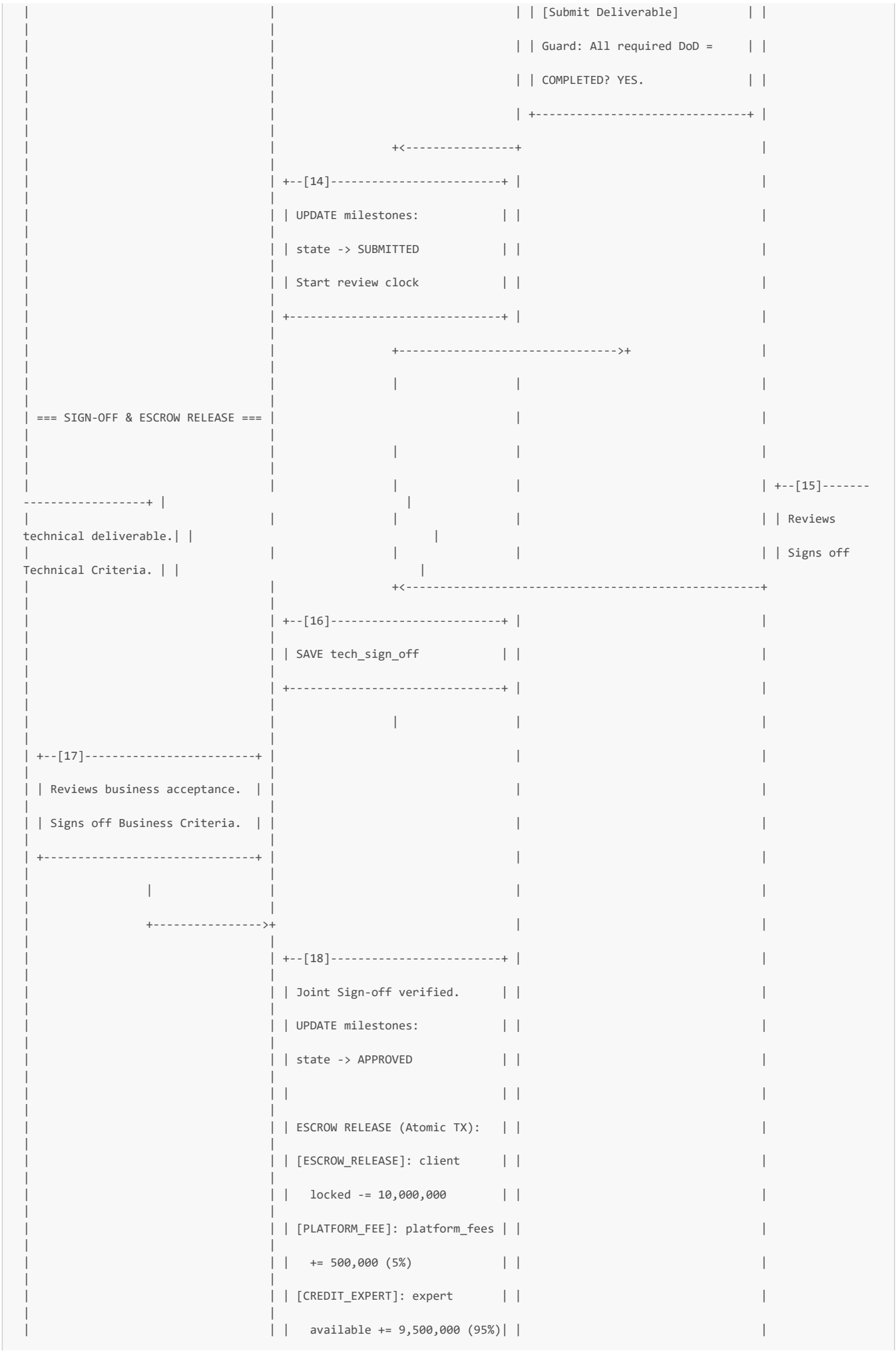
MF-7: Milestone Management & Escrow Payment

Detailed Swimlane Flow




[2]
INSERT milestones:
state: DEFINED
INSERT acceptance_criteria
INSERT dod_checklist_items
=== MILESTONE FUNDING (ESCROW) ===
[3]
Clicks "Fund Milestone 1"
[4]
UPDATE milestones:
state -> AWAITING_PAYMENT
Generate per-milestone VA via
SePay API (fixed_amount=10M,
expires_at = now() + 24h)
[5]
Returns VA number.
INSERT virtual_accounts:
entity_type: MILESTONE
Generate VietQR for VA
[6]
Scans VietQR with Banking App
Transfers 10,000,000 VND
[7]
Bank processes. Credit on VA.
SePay IPN fires to NestJS.
[8]


| IPN Handler (Atomic TX):
| 1. Verify IPN & idempotency
| 2. Resolve VA -> milestone
| LEDGER OPERATIONS:
| [CREDIT_WALLET]: avail += 10M
| [ESCROW_LOCK]: avail -= 10M,
| locked += 10M
| 3. UPDATE milestones:
| state -> FUNDED
| 4. Auto-advance:
| state -> IN_PROGRESS
=== SPRINT & DOD EXECUTION =======
[9]
Creates Sprint Plan:
Week 1: Data pipeline setup
Week 2: Logic & HITL loop
[Save Plan]
[10]
INSERT sprint_plans
Notify Tech Team
[11]
Updates DoD Checklist:
- Unit tests: COMPLETED
(note: "90% coverage")
- API docs: NOT_APPLICABLE
(note: "Replaced by Swagger")
[12]
UPDATE dod_items states
=== DELIVERABLE SUBMISSION =======
[13]


	[Submit Deliverable]	
Guard: All required DoD =	
COMPLETED? YES.	
+<----------+
+--[14]----------+
| UPDATE milestones: |
| state -> SUBMITTED |
| Start review clock |
+----------+
+---------->+
=== SIGN-OFF & ESCROW RELEASE ===
+--[15]-------
| Reviews
| Signs off
technical deliverable.|
Technical Criteria. |
+<----------+
+--[16]----------+
| SAVE tech_sign_off |
+----------+
+--[17]----------+
| Reviews business acceptance. |
| Signs off Business Criteria. |
+----------+
+---------->+
+--[18]----------+
| Joint Sign-off verified. |
| UPDATE milestones: |
| state -> APPROVED |
| ESCROW RELEASE (Atomic TX): |
| [ESCROW_RELEASE]: client |
| locked -= 10,000,000 |
| [PLATFORM_FEE]: platform_fees |
| += 500,000 (5%) |
| [CREDIT_EXPERT]: expert |
| available += 9,500,000 (95%)|

Step-by-Step Narration

MILESTONE DEFINITION

---

[1] - CEO Creates Milestone

- **Actor:** Client / CEO
- **Input:** CEO navigates to the Project Management view and clicks "Add Milestone". They fill out the structured form:
  - **Title:** "Core Logic & Pipeline Integration"
  - **Type:** `JOINT` (Requires both Technical and Business sign-off)
  - **Amount:** 10,000,000 VND
  - **Acceptance Criteria (Required for payment contract):**
    - "Pipeline processes 600K ads/month without dropping messages"
    - "Automated compliance error rate < 5%"
  - **DoD Checklist Items:**
    - "Unit tests for decision engine" (Required = `true`)
    - "API documentation" (Required = `false`)
- **Output:** `POST /projects/{id}/milestones` with the structured payload.
- **State Change:** None.

---

[2] - Store Milestone, Criteria, & DoD

- **Actor:** System (NestJS)

- **Input:** Milestone payload from [1].
- **Logic:** NestJS validates the input (Amount > 0, at least one Acceptance Criteria) and persists the definition. The milestone is not yet active for work-it requires funding.

Table	Fields Inserted	Rationale
`milestones`	`project_id`, `title`, `amount` (10M), `type` (JOINT), `state` (`DEFINED`)	Shell record
`acceptance_criteria`	`milestone_id`, `description`, `is_required` (true)	Payment contract terms
`dod_checklist_items`	`milestone_id`, `description`, `is_required`, `state` (`PENDING`)	Expert self-check framework

- **Output:** Milestone saved with state `DEFINED`.
- **State Change:** Tables gain rows. Milestone is ready for funding.
- **Tables Affected:** `milestones`, `acceptance_criteria`, `dod_checklist_items`

MILESTONE FUNDING (ESCROW LOCK)

---

[3] - Clicks "Fund Milestone"

- **Actor:** Client / CEO
- **Input:** CEO reviews the milestone definition and clicks the "Fund" button. This commits the budget.
- **Output:** `POST /milestones/{id}/fund` request sent.
- **State Change:** None.

---

[4] - Generate Per-Milestone VA & Update State

- **Actor:** System (NestJS + SePay API)
- **Input:** Fund request from [3].
- **Logic:**
  1. **State Transition:** `UPDATE milestones SET state = 'AWAITING_PAYMENT' WHERE id = ?`
  2. **VA Creation:** Call SePay Bank Hub API to generate a dedicated Virtual Account for this specific milestone payment.
     - `fixed_amount = 10,000,000` (Bank will reject transfers of any other amount).
     - `expires_at = now() + 24 hours` (Prevents stale QR codes from being scanned late).
  3. **Store VA:** `INSERT INTO virtual_accounts (entity_type: 'MILESTONE', entity_id: milestone_id, va_number, fixed_amount, expires_at, status: 'ACTIVE')`
  4. **Generate VietQR:** Encode the VA details into a scannable QR string.
- **Output:** Return VietQR image/URL to CEO.
- **State Change:** `milestones` updated (`AWAITING_PAYMENT`). `virtual_accounts` gains one row.
- **Tables Affected:** `milestones`, `virtual_accounts`

---

[5] - SePay Returns VA Number

- **Actor:** SePay / Bank Hub
- **Input:** VA creation request.
- **Logic:** SePay creates the sub-account and returns the `va_number`.
- **Output:** VA number stored by NestJS in [4].
- **State Change:** External - VA created.

---

[6] - Scans VietQR & Transfers

- **Actor:** Client / CEO
- **Input:** VietQR displayed on screen.
- **Logic:** CEO opens their banking app, scans the QR. The app pre-fills the exact amount (10M VND) and the VA number. CEO confirms with OTP.
- **Output:** Bank transfer initiated.
- **State Change:** None on platform.

---

[7] - Bank Processes & SePay IPN Fires

- **Actor:** SePay
- **Input:** Bank transfer completion.
- **Logic:** SePay detects the 10M credit on the milestone VA and fires the IPN webhook to NestJS.
- **Output:** `POST /webhook/sepay` with `{ va_number, amount: 10000000, transfer_type: "in" }`.
- **State Change:** External - funds on platform's master account.

---

[8] - IPN Handler: Credit Wallet, Lock Escrow, Advance State

- **Actor:** System (NestJS)

- **Input:** IPN payload from [7].
- **Logic:** This is a critical atomic transaction ensuring the client's wallet reflects the escrow lock accurately without double-spend risk.

```
BEGIN TRANSACTION;

-- 1. Verify IPN signature & idempotency
-- 2. Resolve VA -> Milestone -> Project -> CEO user_id

-- 3. Credit wallet (intermediate step to satisfy double-entry)
UPDATE wallets SET available_balance = available_balance + 10000000 WHERE user_id = ceo_id;
INSERT INTO wallet_transactions (type: 'TOP_UP', amount: 10000000);

-- 4. Escrow Lock (Move from available to locked)
UPDATE wallets SET available_balance = available_balance - 10000000,
                   locked_balance = locked_balance + 10000000 WHERE user_id = ceo_id;
INSERT INTO wallet_transactions (type: 'ESCROW_LOCK', amount: 10000000);

-- 5. Update Milestone State
UPDATE milestones SET state = 'FUNDED' WHERE id = milestone_id;

-- 6. Auto-advance State (Per Section 0.6: auto-advances from FUNDED)
UPDATE milestones SET state = 'IN_PROGRESS' WHERE id = milestone_id;

COMMIT;
```

(Note: Practically, the TOP_UP and ESCROW_LOCK can be netted out so `available_balance` doesn't change and only `locked_balance` increases, but explicitly logging both ledger entries provides a flawless audit trail).

- **Output:** Transaction committed. Expert notified: "Milestone 1 is Funded and In Progress."
- **State Change:** `wallets` updated (`locked_balance += 10M`). `wallet_transactions` gains 2 rows. `milestones` updated (`IN_PROGRESS`).
- **Tables Affected:** `wallets`, `wallet_transactions`, `milestones`

SPRINT & DOD EXECUTION

---

[9] - Expert Creates Sprint Plan

- **Actor:** Expert
- **Input:** Expert opens the active milestone and creates a weekly sprint plan.
  - Week 1: "Data pipeline setup & Kafka integration"
  - Week 2: "Core logic & HITL escalation loop"
- **Output:** `POST /milestones/{id}/sprints` with plan data.
- **State Change:** None.

---

[10] - Store Sprint Plan & Notify Tech Team

- **Actor:** System (NestJS)
- **Input:** Sprint plan from [9].
- **Logic:** Persist the plan. Notify the TECH_TEAM so they can track progress and comment.
- **Output:** Sprint plan saved. Notification sent.
- **State Change:** `sprint_plans` table gains rows.

---

[11] - Expert Updates DoD Checklist

- **Actor:** Expert
- **Input:** As work progresses, the Expert checks off DoD items.
  - "Unit tests" → `COMPLETED` (Required item; `completion_note`: "90% coverage achieved")
  - "API docs" → `NOT_APPLICABLE` (Non-required item; `note`: "Replaced by Swagger auto-gen")
- **Output:** `PATCH /dod-items/{id}` with `{ state: "COMPLETED", completion_note: "..." }`.
- **State Change:** None.

---

[12] - Store DoD Updates

- **Actor:** System (NestJS)
- **Input:** DoD updates from [11].
- **Logic:** Update the `dod_checklist_items` state and notes.
- **Output:** DoD progress saved.
- **State Change:** `dod_checklist_items` updated.

DELIVERABLE SUBMISSION

[13] - Expert Submits Deliverable

- **Actor:** Expert
- **Input:** Work is complete. Expert clicks "Submit Deliverable" and attaches the final package (code repo link, deployment confirmation, etc.).
- **Logic:** Before the system accepts the submission, a backend guard verifies: **Are all `is_required = true` DoD items in state `COMPLETED`?** If not, the submission is rejected with `403 { code: "REQUIRED_DOD_INCOMPLETE" }`.
- **Output:** `POST /milestones/{id}/submit` with deliverable files/links.
- **State Change:** None.

[14] - Update Milestone State (SUBMITTED)

- **Actor:** System (NestJS)
- **Input:** Submission payload from [13].
- **Logic:** The milestone enters the review phase. A review clock starts (e.g., 5 business days for sign-off before auto-approval or escalation to dispute).
  - `UPDATE milestones SET state = 'SUBMITTED', submitted_at = now() WHERE id = ?`
  - Notify TECH_TEAM and CEO that the deliverable is ready for review.
- **Output:** Milestone state updated. Reviewers notified.
- **State Change:** `milestones` updated (`SUBMITTED`).
- **Tables Affected:** `milestones`

SIGN-OFF & ESCROW RELEASE

[15] - Tech Team Signs Off Technical Criteria

- **Actor:** Client / TECH_TEAM
- **Input:** Deliverable review request on Tech Dashboard.
- **Logic:** Tech Team reviews the code, pipeline stability, and technical Acceptance Criteria. They verify the HITL loop functions correctly. They click "Sign Off".
- **Output:** `POST /milestones/{id}/signoff/technical` request sent.
- **State Change:** None.

[16] - Store Tech Sign-off

- **Actor:** System (NestJS)
- **Input:** Tech sign-off from [15].
- **Logic:** Record the technical approval. Since this is a `JOINT` milestone, the system waits for the CEO's business approval before proceeding. If it were a `TECHNICAL` milestone, this step alone would trigger the escrow release.
- **Output:** Tech sign-off saved. CEO notified.
- **State Change:** `milestone_signoffs` table gains a row.

[17] - CEO Signs Off Business Criteria

- **Actor:** Client / CEO
- **Input:** Deliverable review request on CEO Dashboard.
- **Logic:** CEO reviews the business Acceptance Criteria (error rate < 5%, budget alignment). They click "Approve and Release Funds".
- **Output:** `POST /milestones/{id}/signoff/business` request sent.
- **State Change:** None.

[18] - Verify Joint Sign-off & Execute Escrow Release

- **Actor:** System (NestJS)
- **Input:** CEO sign-off from [17].
- **Logic:** The system verifies both parties have signed off. It then executes the most critical financial transaction: releasing the escrow.

```
BEGIN TRANSACTION;

-- 1. Verify all required Acceptance Criteria have sign-offs
-- 2. Verify all sign-off authorities (TECH + CEO for JOINT)

-- 3. Update Milestone State
UPDATE milestones SET state = 'APPROVED' WHERE id = milestone_id;

-- 4. Escrow Release Ledger (Atomic)
-- [ESCROW_RELEASE]: Unlock from client
UPDATE wallets SET locked_balance = locked_balance - 10000000 WHERE user_id = ceo_id;
```

```
INSERT INTO wallet_transactions (type: 'ESCROW_RELEASE', amount: 10000000);

-- [PLATFORM_FEE]: Take 5% cut
UPDATE platform_fees SET balance = balance + 500000;
INSERT INTO wallet_transactions (type: 'PLATFORM_FEE', amount: 500000);

-- [CREDIT_EXPERT]: Credit 95% to expert
UPDATE wallets SET available_balance = available_balance + 9500000 WHERE user_id = expert_id;
INSERT INTO wallet_transactions (type: 'ESCROW_RELEASE', amount: 9500000, wallet_id: expert_wallet_id);

COMMIT;
```

- **Output:** Funds moved from client's locked balance to expert's available balance. Platform fee collected. Milestone `APPROVED`.
- **State Change:** `wallets` updated (Client locked -10M, Expert avail +9.5M, Platform +0.5M). `milestones` updated (`APPROVED`).
- **Tables Affected:** `wallets`, `wallet_transactions`, `milestones`, `platform_fees`

AUTO-WITHDRAWAL (CHI HỘ)

---

[19] - Trigger Withdrawal Request & Chi Hộ API

- **Actor:** System (NestJS)
- **Input:** Escrow release completion from [18].
- **Logic:** The system immediately initiates the automated withdrawal to the expert's linked bank account. Zero admin action.
  1. `INSERT INTO withdrawal_requests (expert_id, amount: 9500000, state: 'PENDING')`
  2. `UPDATE wallets SET available_balance = available_balance - 9500000 WHERE user_id = expert_id` (Atomic with insert)
  3. **Call SePay Chi Hộ API:** `POST { amount: 9500000, bank_account_xid: expert_xid, reference: "WD-{id}" }`
- **Output:** API call dispatched.
- **State Change:** `wallets` updated (Expert avail -9.5M). `withdrawal_requests` gains row (`PENDING`).
- **Tables Affected:** `wallets`, `wallet_transactions`, `withdrawal_requests`

---

[20] - SePay Transfers & Credit IPN Fires

- **Actor:** SePay / Bank Hub
- **Input:** Chi hộ API request from [19].
- **Logic:** SePay initiates the interbank transfer to the expert's verified account. Upon success, the credit IPN fires.
- **Output:** Credit IPN payload sent to NestJS.
- **State Change:** External - funds land in expert's personal bank.

---

[21] - Settle Withdrawal & Release Milestone

- **Actor:** System (NestJS)
- **Input:** Credit IPN from [20].
- **Logic:** Final settlement.
  1. `UPDATE withdrawal_requests SET state = 'COMPLETED' WHERE id = ?`
  2. `UPDATE milestones SET state = 'RELEASED' WHERE id = ?`
- **Output:** Lifecycle complete. Expert notified of successful bank transfer.
- **State Change:** `withdrawal_requests` updated (`COMPLETED`). `milestones` updated (`RELEASED`).
- **Tables Affected:** `withdrawal_requests`, `milestones`

**[22] - Milestone 1: RELEASED **

- **Actor:** Client / CEO
- **Input:** Dashboard update.
- **Logic:** CEO sees Milestone 1 is fully released and closed. They can now proceed to fund Milestone 2.
- **Output:** Milestone lifecycle ends.
- **State Change:** None (UI reflects DB).

Summary: End-to-End State Trace for MF-7

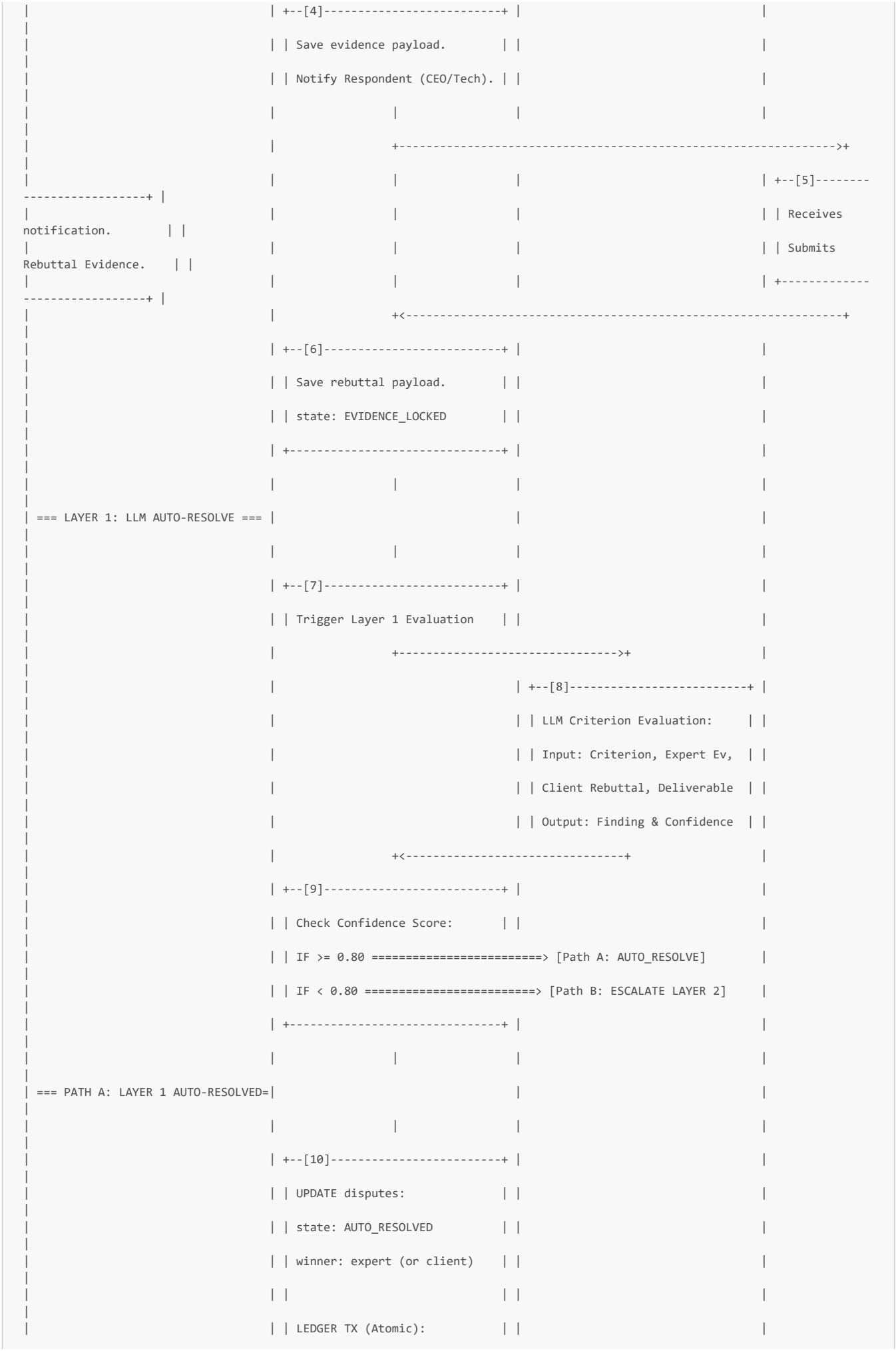
Step	`milestones`	`wallets` **(Client)**	`wallets` **(Expert)**	`withdrawal_requests`
[2] Defined	state=DEFINED	-	-	-
[4] Awaiting Payment	state=AWAITING_PAYMENT	-	-	-
[8] Funded / In Progress	state=IN_PROGRESS	locked += 10M	-	-
[14] Submitted	state=SUBMITTED	-	-	-

Step	milestones	wallets (Client)	wallets (Expert)	withdrawal_requests
[18] Approved	state=APPROVED	locked -= 10M	avail += 9.5M	-
[19] Auto-Withdrawal	-	-	avail -= 9.5M	state=PENDING
[21] Released	state=RELEASED	-	-	state=COMPLETED

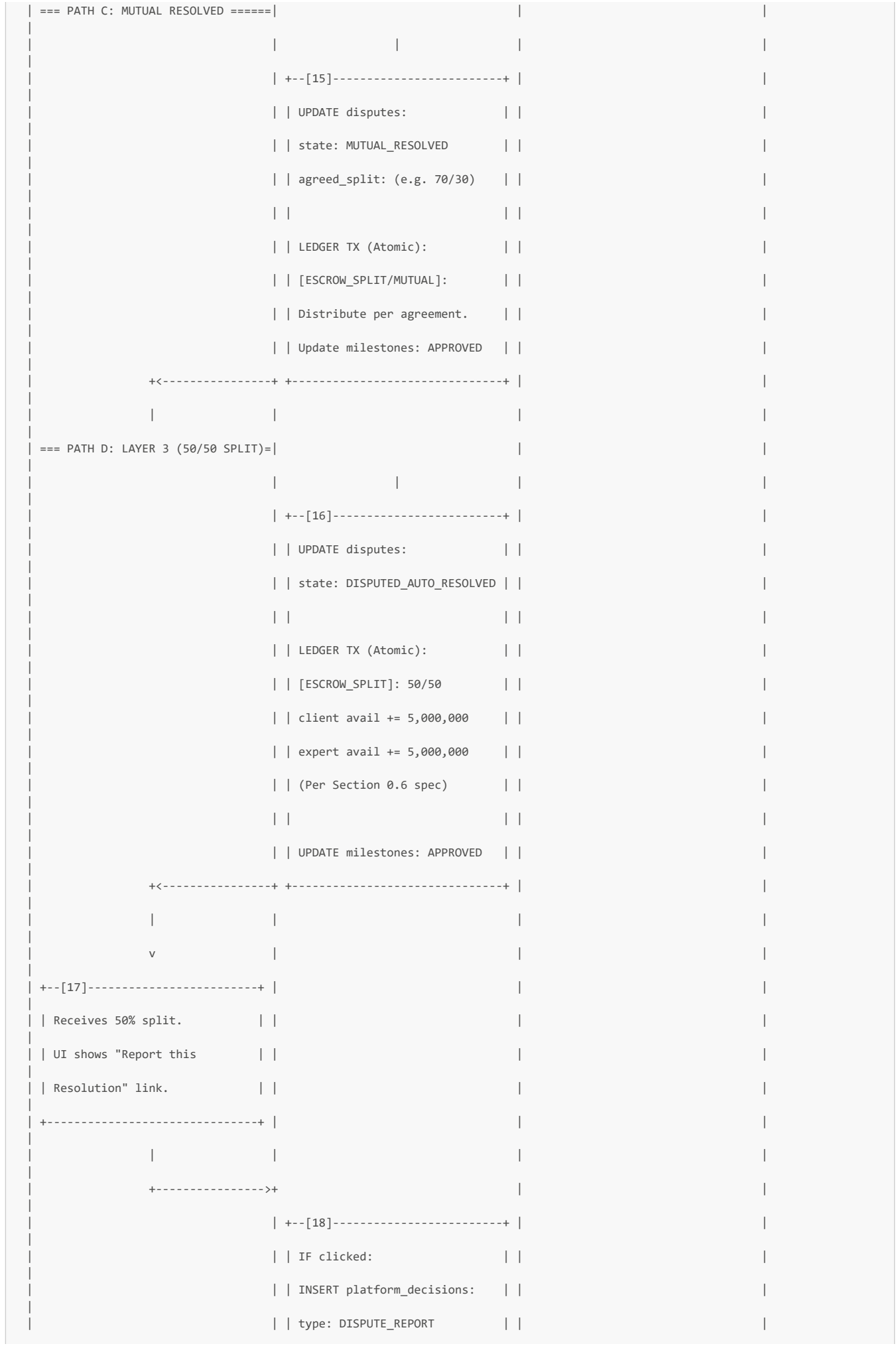
MF-7 is now complete. The milestone has traversed its full state machine from `DEFINED` to `RELEASED`. Escrow was securely locked via VietQR IPN, dual sign-off was enforced for a joint milestone, the platform fee was extracted atomically, and the expert's earnings were automatically disbursed to their bank via Chi Hộ.

MF-8: Dispute Resolution (3 Layers)

Detailed Swimlane Flow


		+--[4]--------------------------+		
			Save evidence payload.			
			Notify Respondent (CEO/Tech).			
		+--->+
					+--[5]--------
notification.	
					Receives
Rebuttal Evidence.	
					Submits
		+<---+
		+--[6]--------------------------+		
			Save rebuttal payload.			
			state: EVIDENCE_LOCKED			
		+--------------------------------+		
=== LAYER 1: LLM AUTO-RESOLVE ===		
		+--[7]--------------------------+		
			Trigger Layer 1 Evaluation			
		+--------------------------------->+	
			+--[8]--------------------------+	
				LLM Criterion Evaluation:		
				Input: Criterion, Expert Ev,		
				Client Rebuttal, Deliverable		
				Output: Finding & Confidence		
		+<---------------------------------+	
		+--[9]--------------------------+		
			Check Confidence Score:			
			IF >= 0.80 ========================> [Path A: AUTO_RESOLVE]
			IF < 0.80 =========================> [Path B: ESCALATE LAYER 2]
		+--------------------------------+		
=== PATH A: LAYER 1 AUTO-RESOLVED=			
		+--[10]-------------------------+		
			UPDATE disputes:			
			state: AUTO_RESOLVED			
			winner: expert (or client)			
			LEDGER TX (Atomic):			


[ESCROW_RELEASE] if expert:
client locked -= 10M
expert avail += 9.5M (95%)
platform fees += 0.5M (5%)
[ESCROW_REFUND] if client:
client locked -= 10M
client avail += 10M
UPDATE milestones: APPROVED
[11]
Dispute Resolved. Funds released/refunded.
=== PATH B: LAYER 2 MUTUAL AGREEMENT==
[12]
UPDATE disputes:
state: MUTUAL_NEGOTIATION
Start 48-hour cooling clock
[13]
Views structured agreement form.
Options: Release to Exp, Refund Client, Custom Split.
[14]
Check Mutual Selection:
IF BOTH select same option =====> [Path C: MUTUAL_RESOLVED]
IF 48h EXPIRES no agreement ====> [Path D: ESCALATE LAYER 3]


=== PATH C: MUTUAL RESOLVED ======|
[15]
UPDATE disputes:
state: MUTUAL_RESOLVED
agreed_split: (e.g. 70/30)
LEDGER TX (Atomic):
[ESCROW_SPLIT/MUTUAL]:
Distribute per agreement.
Update milestones: APPROVED
=== PATH D: LAYER 3 (50/50 SPLIT)=|
[16]
UPDATE disputes:
state: DISPUTED_AUTO_RESOLVED
LEDGER TX (Atomic):
[ESCROW_SPLIT]: 50/50
client avail += 5,000,000
expert avail += 5,000,000
(Per Section 0.6 spec)
UPDATE milestones: APPROVED
[17]
Receives 50% split.
UI shows "Report this
Resolution" link.
[18]
IF clicked:
INSERT platform_decisions:
type: DISPUTE_REPORT

```
|
|                                  | | (Informational log only, no   | |                                  |
|
|                                  | | financial or state reversal)  | |                                  |
|
|                                  | +-------------------------------+ |                                  |
|
+----------------------------------+-----------------------------------+----------------------------------+---------------
-------------------+
```

Step-by-Step Narration

DISPUTE INITIATION

---

[1] - Filer (Expert/Client) Initiates Dispute

- **Actor:** Filer (Can be Expert, Client/CEO, or Client/TECH_TEAM per RBAC matrix).
- **Input:** The milestone is in state `SUBMITTED` or `IN_REVISION` (the only states from which a dispute can be filed per Section 0.6). The Filer believes the sign-off authority is wrongly withholding approval or the deliverable does not meet criteria.
- **Logic:** Filer clicks "File Dispute" on the milestone view.
- **Output:** `POST /milestones/{id}/dispute/initiate` request sent.
- **State Change:** None.

---

[2] - Validate State & Freeze Escrow

- **Actor:** System (NestJS)
- **Input:** Initiation request from [1].
- **Logic:** The system performs critical safety checks before opening the dispute:
  1. **State Guard:** Verify `milestones.state IN ('SUBMITTED', 'IN_REVISION')`. If not, reject (`400 INVALID_STATE_FOR_DISPUTE`).
  2. **Create Dispute Record:** `INSERT INTO disputes (milestone_id, filer_id, state, escrow_status) VALUES (?, ?, 'OPEN', 'FROZEN')`
  3. **Freeze Escrow:** While the escrow is already locked in the wallet (`locked_balance`), the `escrow_status = FROZEN` flag on the dispute ensures no automated or manual release can occur while the dispute is active.
  4. **Transition Milestone State:** `UPDATE milestones SET state = 'DISPUTED' WHERE id = ?`
- **Output:** Dispute created. Escrow frozen. Milestone state updated.
- **State Change:** `disputes` gains one row (`OPEN`). `milestones` updated (`DISPUTED`).
- **Tables Affected:** `disputes`, `milestones`

---

[3] - Filer Submits Evidence

- **Actor:** Filer
- **Input:** The structured dispute form. The Filer must target a specific acceptance criterion.
  - **Selected Criterion:** "Automated compliance error rate < 5%"
  - **Claim:** "Deliverable meets this criterion; test logs show a 3.2% error rate."
  - **Attachments:** `test_logs.pdf`, `staging_metrics.json`
- **Output:** `POST /disputes/{id}/evidence` with payload.
- **State Change:** None.

---

[4] - Save Evidence & Notify Respondent

- **Actor:** System (NestJS)
- **Input:** Evidence payload from [3].
- **Logic:** Persist the evidence linked to the dispute. Notify the Respondent (the other party) that a dispute has been filed and they have 24 hours to submit a rebuttal before automated evaluation begins.
- **Output:** Evidence saved. Notification sent.
- **State Change:** `dispute_evidence` table gains rows.

---

[5] - Respondent Submits Rebuttal Evidence

- **Actor:** Respondent
- **Input:** Dispute notification.
- **Logic:** The Respondent (e.g., Tech Team) reviews the Filer's evidence and submits a counter-argument.
  - **Rebuttal:** "Test logs were run on sanitized data; production stream shows a 7.8% error rate due to edge cases."
  - **Attachments:** `production_logs.csv`
- **Output:** `POST /disputes/{id}/rebuttal` with payload.
- **State Change:** None.

---

[6] - Save Rebuttal & Lock Evidence

- **Actor:** System (NestJS)
- **Input:** Rebuttal payload from [5].
- **Logic:** Save the rebuttal. Transition the dispute state to `EVIDENCE_LOCKED` so no further modifications can be made, ensuring a static dataset for the AI evaluation.
- **Output:** Evidence phase complete. Ready for Layer 1.
- **State Change:** `disputes` updated (`EVIDENCE_LOCKED`).
- **Tables Affected:** `dispute_evidence`, `disputes`

LAYER 1 - LLM AUTO-RESOLVE

---

[7] - Trigger Layer 1 Evaluation

- **Actor:** System (NestJS)
- **Input:** Evidence locked event from [6].
- **Logic:** NestJS compiles the dispute context (Criterion text, Filer evidence, Respondent rebuttal, Deliverable references) and sends it to the AI Service for objective evaluation.
- **Output:** `POST /ai/dispute/evaluate` request sent.
- **State Change:** None.

---

[8] - LLM Criterion Evaluation (FastAPI)

- **Actor:** AI Service (FastAPI + Claude API)
- **Input:** Compiled dispute payload.
- **Logic:** The LLM is constrained to act as a neutral technical arbitrator. It must:
  1. **Analyze the Criterion:** Parse the strict definition of "Error rate < 5%".
  2. **Evaluate Evidence:** Weigh the Filer's test logs against the Respondent's production logs.
  3. **Determine Finding:** Decide whether the criterion is met based on the preponderance of evidence.
  4. **Calculate Confidence:** Assess how certain it is of the finding. Are the logs contradictory? Is one clearly fabricated or out of context?
- **Output:** Returns `{ finding: "CRITERION_MET" (or NOT_MET), confidence: 0.85, reasoning: "..." }`.
- **State Change:** None.

---

[9] - Check Confidence Score

- **Actor:** System (NestJS)
- **Input:** FastAPI response from [8].
- **Logic:** The system applies the structural threshold defined in Section 0.6.
  - If `confidence >= 0.80` → **Path A (Auto-Resolve)**. The AI is sure enough to make a binding decision.
  - If `confidence < 0.80` → **Path B (Escalate to Layer 2)**. The evidence is too ambiguous for automated resolution; human judgment is required.
- **Output:** Routing decision.
- **State Change:** None.

PATH A - LAYER 1 AUTO-RESOLVED

---

[10] - Execute Auto-Resolution & Ledger Transaction

- **Actor:** System (NestJS)
- **Input:** Finding with high confidence (>= 0.80).
- **Logic:** The system executes the AI's finding automatically. No admin intervention.
  1. **Update Dispute:** `UPDATE disputes SET state = 'AUTO_RESOLVED', winner = 'expert' (or 'client'), resolution_reasoning = '...' WHERE id = ?`
  2. **Execute Ledger (Atomic):**
     - **If Expert Wins (Criterion Met):**
       - `[ESCROW_RELEASE]`: `wallets.locked_balance -= 10,000,000` (Client)
       - `[PLATFORM_FEE]`: `platform_fees += 500,000` (5%)
       - `[CREDIT_EXPERT]`: `wallets.available_balance += 9,500,000` (Expert)
     - **If Client Wins (Criterion Not Met):**
       - `[ESCROW_REFUND]`: `wallets.locked_balance -= 10,000,000` (Client)
       - `[CREDIT_CLIENT]`: `wallets.available_balance += 10,000,000` (Client)
  3. **Update Milestone:** `UPDATE milestones SET state = 'APPROVED' WHERE id = ?` (Even if refunded, the milestone is closed as 'APPROVED' from a dispute resolution standpoint, marking the contract obligation as settled).
- **Output:** Funds distributed. Dispute closed.
- **State Change:** `disputes` updated (`AUTO_RESOLVED`). `wallets` updated. `milestones` updated (`APPROVED`).
- **Tables Affected:** `disputes`, `wallets`, `wallet_transactions`, `milestones`

---

[11] - Dispute Resolved

- **Actor:** Filer / Respondent

- **Input:** Notification of resolution.
- **Logic:** Both parties are notified of the outcome and the AI's reasoning. If the expert won, the auto-withdrawal (Chi Hộ) process from MF-7 triggers immediately.
- **Output:** Dispute flow ends.
- **State Change:** None.

PATH B - LAYER 2 MUTUAL AGREEMENT

---

[12] - Escalate to Layer 2 & Start 48-Hour Clock

- **Actor:** System (NestJS)
- **Input:** Finding with low confidence (< 0.80).
- **Logic:** The AI punted. The system opens a structured negotiation window.
    1. **Update Dispute:** `UPDATE disputes SET state = 'MUTUAL_NEGOTIATION', layer_1_outcome = 'LOW_CONFIDENCE' WHERE id = ?`
    2. **Start Timer:** Record `negotiation_expires_at = now() + 48 hours`.
- **Output:** Parties notified to negotiate.
- **State Change:** `disputes` updated (`MUTUAL_NEGOTIATION`).

---

[13] - Respondent Views Agreement Form

- **Actor:** Respondent (and Filer, simultaneously)
- **Input:** Layer 2 notification.
- **Logic:** Both parties see a structured form with three options:
    1. Release funds to Expert (Accept Deliverable)
    2. Refund funds to Client (Reject Deliverable)
    3. Custom Split (e.g., 70/30, acknowledging partial completion)
- **Output:** Parties select their preferred options.
- **State Change:** None.

---

[14] - Check Mutual Selection

- **Actor:** System (NestJS)
- **Input:** Periodic check or webhook when a party submits their selection.
- **Logic:**
    - **If Both select the SAME option → Path C (Mutual Resolved)**.
    - **If 48 hours expire WITHOUT agreement → Path D (Layer 3 50/50 Split)**.
- **Output:** Routing decision.
- **State Change:** None.

PATH C - MUTUAL RESOLVED

---

[15] - Execute Mutual Agreement & Ledger Transaction

- **Actor:** System (NestJS)
- **Input:** Matching selections from [14].
- **Logic:** The system executes the agreed-upon split.
    1. **Update Dispute:** `UPDATE disputes SET state = 'MUTUAL_RESOLVED', agreed_split = '70/30' WHERE id = ?`
    2. **Execute Ledger (Atomic):**
        - `[ESCROW_SPLIT/MUTUAL]`: `wallets.locked_balance -= 10,000,000` (Client)
        - If 70/30 split in favor of Expert:
            - `[CREDIT_EXPERT]`: `wallets.available_balance += 6,650,000` (70% minus 5% platform fee on expert portion)
            - `[CREDIT_CLIENT]`: `wallets.available_balance += 3,000,000` (30%)
            - `[PLATFORM_FEE]`: `platform_fees += 350,000`
    3. **Update Milestone:** `UPDATE milestones SET state = 'APPROVED'`
- **Output:** Funds distributed per agreement. Dispute closed.
- **State Change:** `disputes` updated (`MUTUAL_RESOLVED`). `wallets` updated. `milestones` updated (`APPROVED`).
- **Tables Affected:** `disputes`, `wallets`, `wallet_transactions`, `milestones`

PATH D - LAYER 3 (50/50 SPLIT)

---

[16] - Execute Forced 50/50 Split

- **Actor:** System (NestJS)
- **Input:** Timer expiration from [14].
- **Logic:** The parties could not agree. Per Section 0.6, the platform enforces a structural default: a 50/50 split of the escrow. This avoids the need for a human admin judge and ensures finality.

1. **Update Dispute:** `UPDATE disputes SET state = 'DISPUTED_AUTO_RESOLVED' WHERE id = ?`
2. **Execute Ledger (Atomic per Section 0.6):**
   - `[ESCROW_SPLIT]`: `wallets.locked_balance -= 10,000,000` (Client)
   - `[CREDIT_CLIENT]`: `wallets.available_balance += 5,000,000` (50%)
   - `[CREDIT_EXPERT]`: `wallets.available_balance += 5,000,000` (50%)
   - *(Note: Per Section 0.6, the split is exactly amount/2 to both parties. Platform fee logic is waived/simplified here as a cost of dispute resolution failure, or applied beforehand. The flow follows the spec literal: `ESCROW_SPLIT - Layer 3 50/50 → both parties available_balance += amount/2`)*
3. **Update Milestone:** `UPDATE milestones SET state = 'APPROVED'`

- **Output:** Funds split equally. Dispute closed.
- **State Change:** `disputes` updated (`DISPUTED_AUTO_RESOLVED`). `wallets` updated. `milestones` updated (`APPROVED`).
- **Tables Affected:** `disputes`, `wallets`, `wallet_transactions`, `milestones`

---

[17] - Receives 50% Split & "Report" Link

- **Actor:** Filer / Respondent
- **Input:** Notification of forced resolution.
- **Logic:** Both parties receive their 50% share. Because this is a forced resolution that may feel unfair to one or both parties, the UI provides a **"Report this Resolution"** link. This is purely an informational venting mechanism.
- **Output:** Resolution acknowledged.
- **State Change:** None.

---

[18] - Log Report (Informational Only)

- **Actor:** System (NestJS)
- **Input:** User clicks "Report this Resolution".
- **Logic:** The system logs the report for the Admin Platform Integrity Monitor (MF-17), but **does not reverse the financial transaction or reopen the dispute**.
  - `INSERT INTO platform_decisions (type: 'DISPUTE_REPORT', dispute_id, user_id, comment) VALUES (...)`
- **Output:** Report logged. User thanked for feedback.
- **State Change:** `platform_decisions` gains one row.
- **Tables Affected:** `platform_decisions`

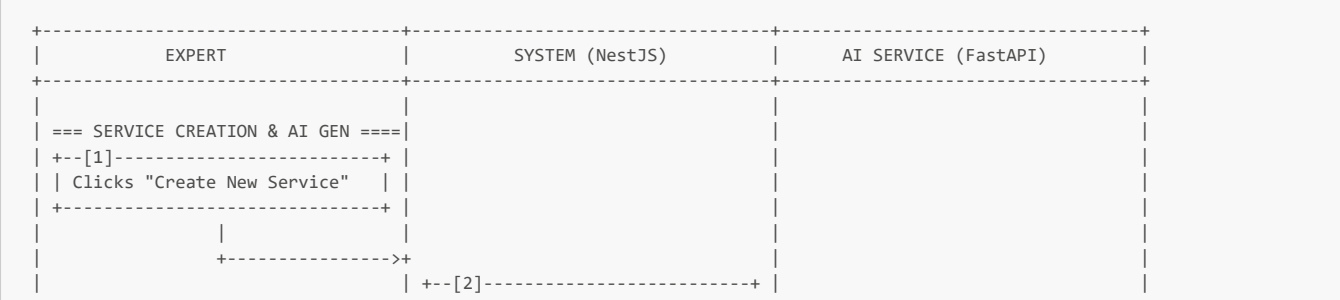
Summary: End-to-End State Trace for MF-8

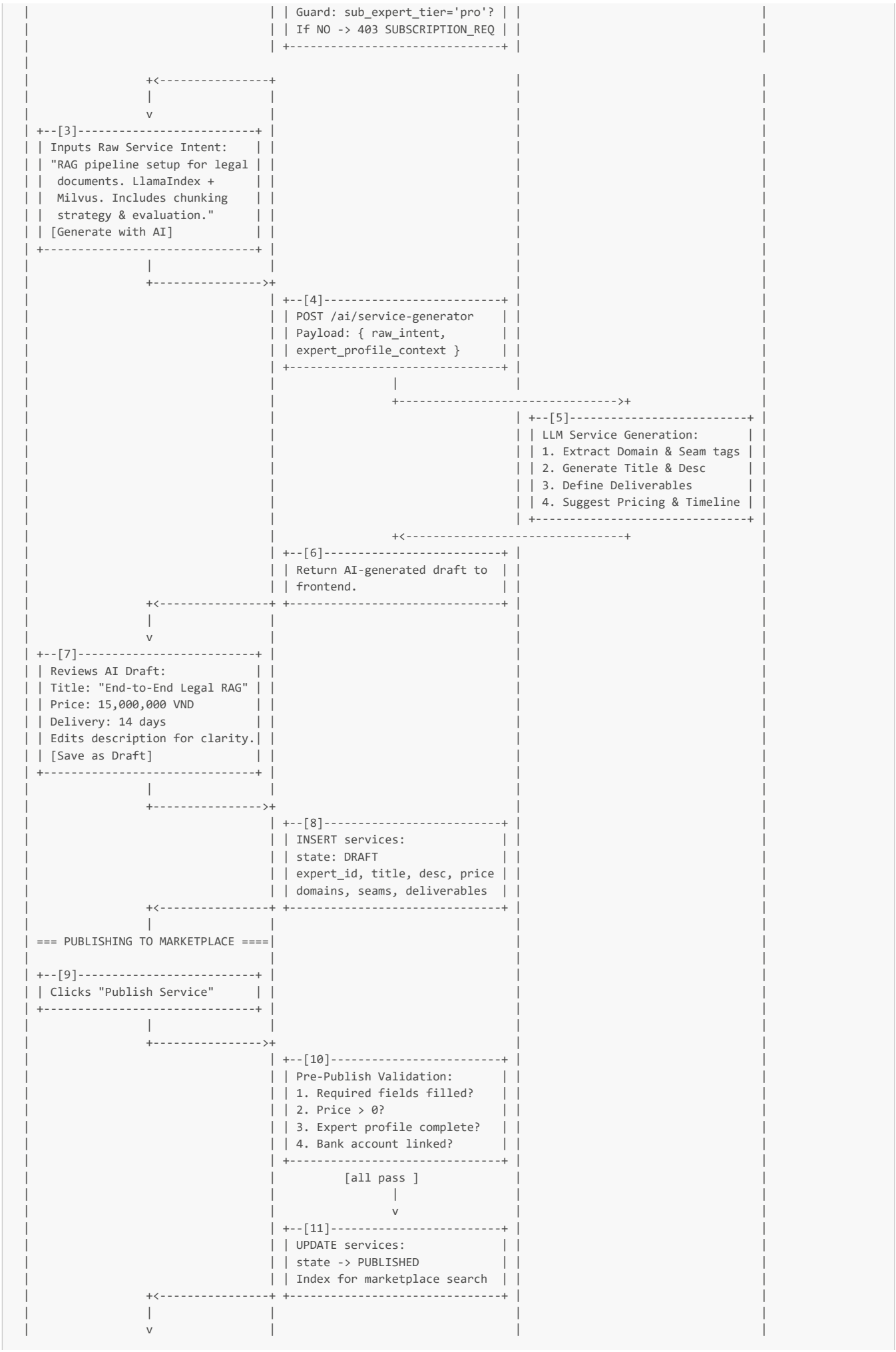
Step	`milestones`	`disputes`	`wallets` (Client)	`wallets` (Expert)
[2] Initiated	state=DISPUTED	state=OPEN, escrow=FROZEN	-	-
[6] Evidence Locked	-	state=EVIDENCE_LOCKED	-	-
[10] Path A: Auto-Resolved	state=APPROVED	state=AUTO_RESOLVED	locked -= 10M	avail += 9.5M (or Client avail += 10M)
[12] Path B: Layer 2	-	state=MUTUAL_NEGOTIATION	-	-
[15] Path C: Mutual Resolved	state=APPROVED	state=MUTUAL_RESOLVED	locked -= 10M, avail += 3M	avail += 6.65M
[16] Path D: Layer 3 Split	state=APPROVED	state=DISPUTED_AUTO_RESOLVED	locked -= 10M, avail += 5M	avail += 5M

MF-8 is now complete. The dispute has been resolved through the 3-layer state machine. Layer 1 provides scalable AI arbitration for clear-cut cases, Layer 2 gives parties a structured window to find a custom compromise, and Layer 3 guarantees finality with a 50/50 split without requiring admin intervention. The escrow is unfrozen, the milestone is closed, and the financial ledger is fully settled.

MF-9: Expert Service Publishing (Path B: Service Marketplace)

Detailed Swimlane Flow




Guard: sub_expert_tier='pro'?
If NO -> 403 SUBSCRIPTION_REQ
[3]
Inputs Raw Service Intent:
"RAG pipeline setup for legal documents. LlamaIndex + Milvus. Includes chunking strategy & evaluation."
[Generate with AI]
[4]
POST /ai/service-generator
Payload: { raw_intent, expert_profile_context }
[5]
LLM Service Generation:
1. Extract Domain & Seam tags
2. Generate Title & Desc
3. Define Deliverables
4. Suggest Pricing & Timeline
[6]
Return AI-generated draft to frontend.
[7]
Reviews AI Draft:
Title: "End-to-End Legal RAG"
Price: 15,000,000 VND
Delivery: 14 days
Edits description for clarity.
[Save as Draft]
[8]
INSERT services:
state: DRAFT
expert_id, title, desc, price
domains, seams, deliverables
=== PUBLISHING TO MARKETPLACE ====
[9]
Clicks "Publish Service"
[10]
Pre-Publish Validation:
1. Required fields filled?
2. Price > 0?
3. Expert profile complete?
4. Bank account linked?
[all pass]
[11]
UPDATE services:
state -> PUBLISHED
Index for marketplace search

```
| +--[12]------------------------+ |                                  |                                  |
| | Service is live on marketplace| |                                  |                                  |
| | Dashboard updated: "Active"   | |                                  |                                  |
| +-------------------------------+ |                                  |                                  |
+-----------------------------------+----------------------------------+----------------------------------+
```

Step-by-Step Narration

SERVICE CREATION & AI GENERATION

---

[1] - Clicks "Create New Service"

- **Actor:** Expert
- **Input:** Expert navigates to their Expert Dashboard and clicks the "Create New Service" button (under the Path B / Service Marketplace section).
- **Logic:** Frontend prepares to load the service creation form and requests AI generation capability.
- **Output:** `GET /services/create` request sent (or frontend state change).
- **State Change:** None.

---

[2] - Subscription Guard

- **Actor:** System (NestJS)
- **Input:** Request from [1], JWT (`active_role = "EXPERT"`).
- **Logic:** Per Section 0.9, the **AI Service Generator** is a gated feature. The NestJS guard checks `users.subscription_expert_tier`.
  - If `tier = "free"` → Return `403 { code: "SUBSCRIPTION_REQUIRED", feature: "ai_service_generator", upgrade_url: "/subscribe" }`.
  - If `tier = "pro"` → Proceed.
- **Output:** Guard passes .
- **State Change:** None.

---

[3] - Inputs Raw Service Intent

- **Actor:** Expert
- **Input:** The UI presents a simplified text area (not a complex form) prompting the expert to describe what they offer in plain language: *"Describe the AI service you provide, the tools you use, and what the client gets."*
  - Expert types: *"RAG pipeline setup for legal documents. LlamaIndex + Milvus. Includes chunking strategy & evaluation baseline setup."*
  - Expert clicks **[Generate with AI]**.
- **Output:** `POST /ai/service-generator` with `{ raw_intent: "..." }`.
- **State Change:** None.

---

[4] - Forward to AI Service

- **Actor:** System (NestJS)
- **Input:** Raw intent payload from [3], Expert's JWT context.
- **Logic:** NestJS enriches the payload with the expert's profile context (verified domains, seams, stack tags from MF-2) so the AI can generate a listing that aligns with the expert's verified capabilities. It then forwards the request to FastAPI.
- **Output:** Internal request to FastAPI `/generate/service-listing`.
- **State Change:** None.

---

[5] - LLM Service Generation (FastAPI + Claude API)

- **Actor:** AI Service (FastAPI)
- **Input:** Raw intent + Expert profile context.
- **Logic:** The LLM transforms the rough intent into a structured, marketplace-ready service listing. It performs:
  1. **Domain & Seam Extraction:** Maps "RAG pipeline" to Domain A (LLM Eng), Domain D (Vector DB), Seam A↔D. Verifies these match the expert's verified profile.
  2. **Title Generation:** *"End-to-End Legal RAG Pipeline Setup"*
  3. **Description Generation:** Professional, client-facing description explaining the value proposition, process, and technical approach in business-friendly language.
  4. **Deliverables Definition:** Structured list of what the client receives (e.g., "Custom chunking logic", "Milvus instance configuration", "Evaluation script with baseline metrics").
  5. **Pricing & Timeline Suggestion:** Based on the complexity of the described service and market averages, suggests a price (e.g., 15,000,000 VND) and timeline (e.g., 14 days).
- **Output:** Returns structured JSON `{ title, description, deliverables, suggested_price, suggested_timeline, domain_tags, seam_tags }`.
- **State Change:** None.

---

[6] - Return AI-Generated Draft

- **Actor:** System (NestJS)
- **Input:** FastAPI response from [5].
- **Logic:** NestJS passes the structured draft back to the frontend.
- **Output:** Response sent to client.
- **State Change:** None.

---

[7] - Reviews AI Draft & Edits

- **Actor:** Expert
- **Input:** AI-generated form fields pre-filled on the frontend.
- **Logic:** The expert reviews the AI output. They can accept it as-is, or manually edit any field. For example, they might adjust the suggested price or clarify a deliverable.
  - *Edits:* Adjusts price to 15,000,000 VND, edits description for tone.
  - Clicks **[Save as Draft]**.
- **Output:** `POST /services` with the full edited payload `{ title, description, price: 15000000, timeline: 14, deliverables, domain_tags, seam_tags, state: "DRAFT" }`.
- **State Change:** None.

---

[8] - Store Service Record (State: DRAFT)

- **Actor:** System (NestJS)
- **Input:** Service payload from [7].
- **Logic:** Persist the service listing. It is not yet visible to clients.

Field	Value Set	Rationale
`expert_id`	from JWT	Ownership
`title`, `description`, etc.	from input	Content
`price`	`15,000,000`	Fixed price for SERVICE_PURCHASE
`timeline_days`	`14`	Expected delivery
`state`	`"DRAFT"`	Not visible on marketplace

- **Output:** Service saved. Expert notified "Draft saved".
- **State Change:** `services` table gains one row (`DRAFT`).
- **Tables Affected:** `services`

PUBLISHING TO MARKETPLACE

---

[9] - Clicks "Publish Service"

- **Actor:** Expert
- **Input:** Expert reviews the draft and clicks the "Publish" toggle/button.
- **Logic:** Frontend sends the publish command.
- **Output:** `PATCH /services/{id}/publish` request sent.
- **State Change:** None.

---

[10] - Pre-Publish Validation

- **Actor:** System (NestJS)
- **Input:** Publish request from [9].
- **Logic:** Before making the service public, the system enforces business rules:
  1. **Required Fields:** Title, description, price, timeline, and at least one deliverable must be present.
  2. **Price Validity:** `price > 0`.
  3. **Expert Profile Complete:** The expert must have completed their profile (domain depths declared).
  4. **Bank Account Linked:** `users.sepay_bank_account_xid IS NOT NULL`. This is critical because Path B purchases use direct VA payment + chi hộ auto-withdrawal. If the expert hasn't linked a bank account (MF-2 Phase F), they cannot receive funds for a service purchase.
- **Output:** If all pass → Proceed. If any fail → `422 { code: "PUBLISH_PRECONDITION_FAILED", missing: ["bank_account_link"] }`.
- **State Change:** None.

---

[11] - Update Service State (PUBLISHED)

- **Actor:** System (NestJS)
- **Input:** Successful validation from [10].
- **Logic:** The service is made live.
  1. `UPDATE services SET state = 'PUBLISHED', published_at = now() WHERE id = ?`
  2. **Search Indexing:** The service's domain tags, seam tags, and title are indexed for the marketplace search/filter API (used in MF-10).

- **Output:** Service published successfully.
- **State Change:** `services` row updated (`PUBLISHED`).
- **Tables Affected:** `services`

---

[12] - Service Live on Marketplace

- **Actor:** Expert
- **Input:** Dashboard update from [11].
- **Logic:** The Expert's dashboard updates the service card from "Draft" to "Active". The service is now visible to CEOs browsing the marketplace (MF-10).
- **Output:** Service is live.
- **State Change:** None (UI reflects DB state).

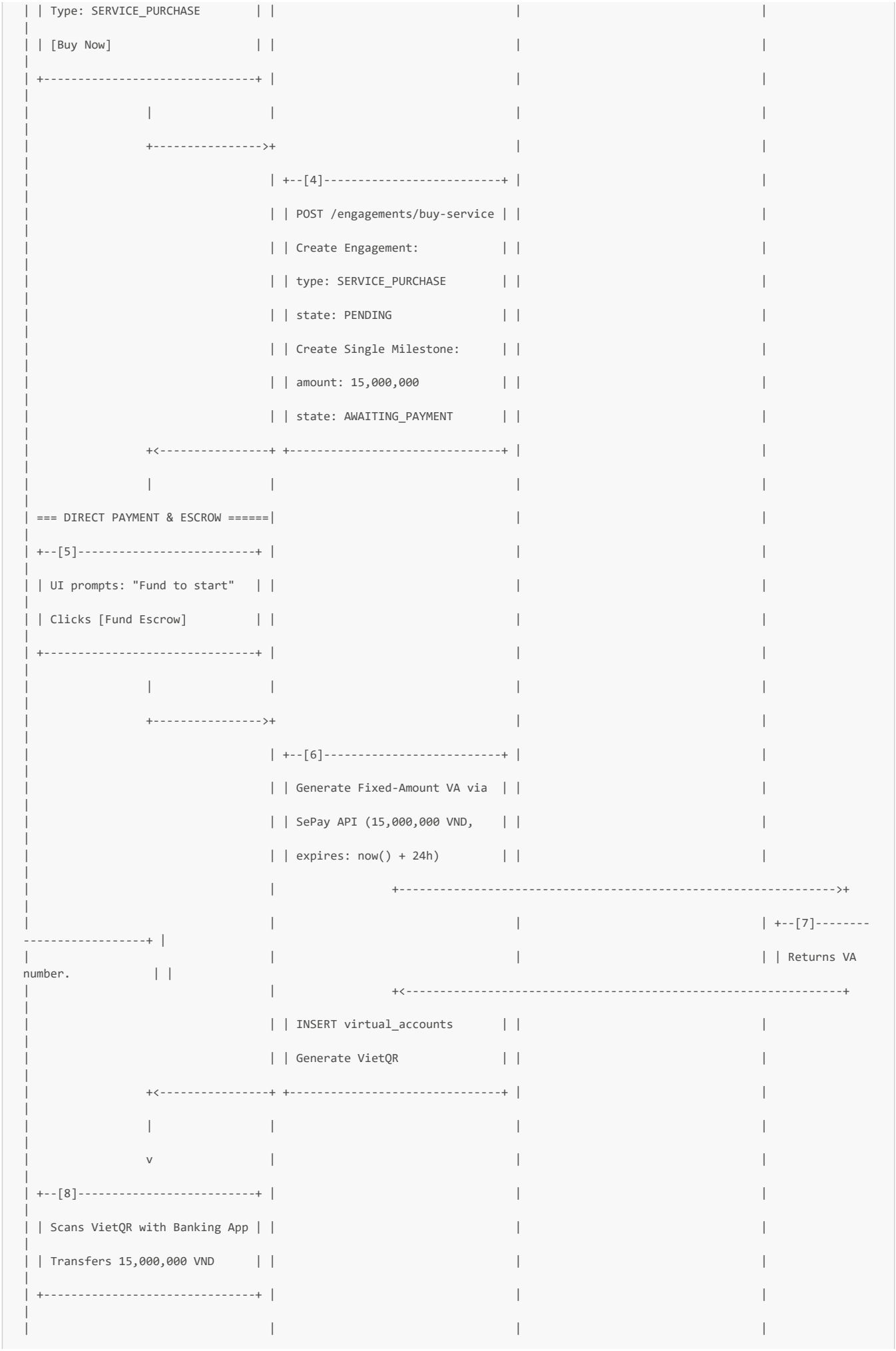
Summary: End-to-End State Trace for MF-9

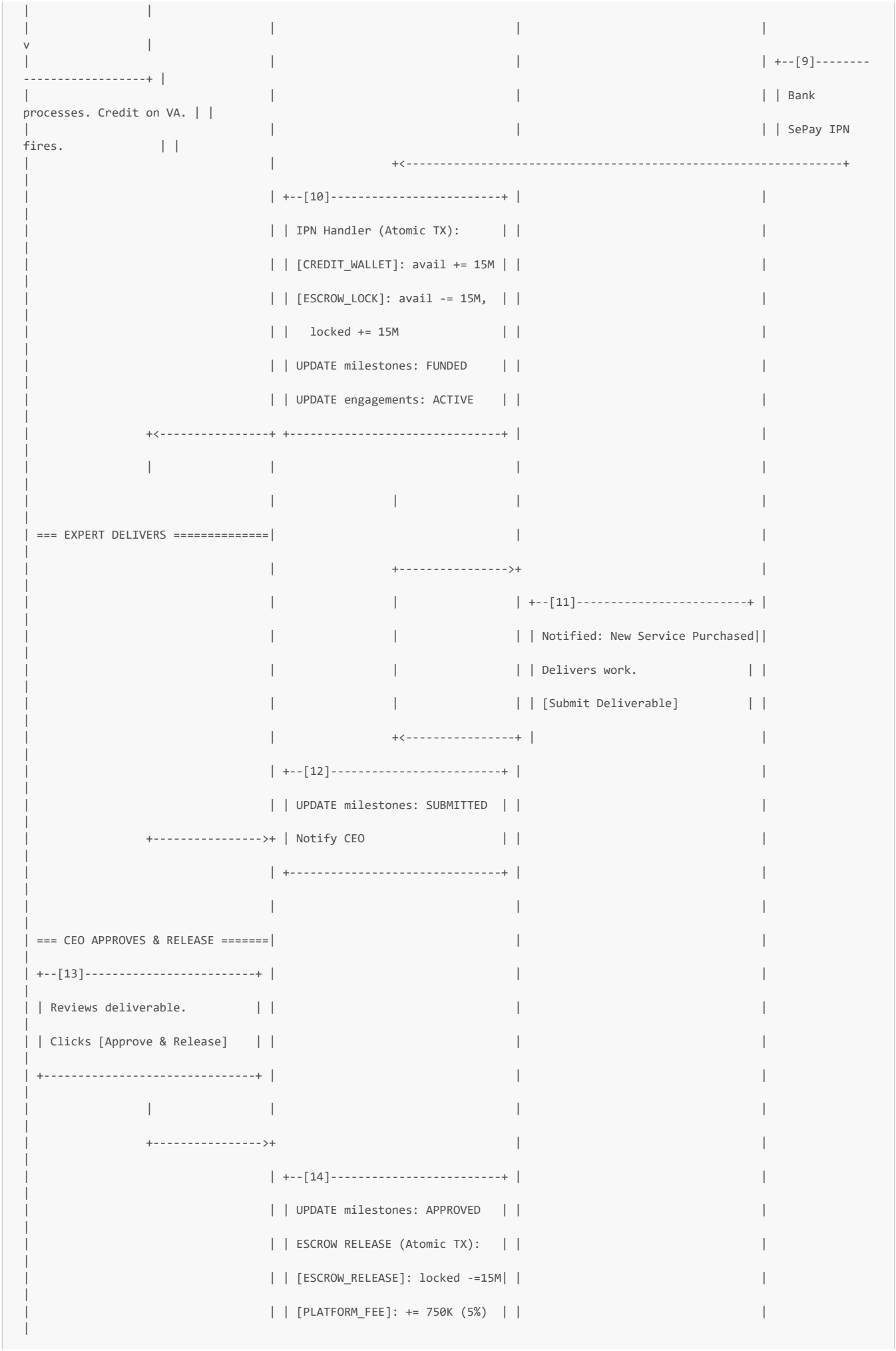
Step	`services`	`users` (checks)	Marketplace Visibility
[2] Guard Check	-	sub_expert_tier = 'pro'	-
[8] Draft Saved	state=DRAFT	-	Not visible
[10] Pre-Publish	-	sepay_bank_account_xid NOT NULL	-
[11] Published	state=PUBLISHED, published_at=now()	-	Visible & Searchable

MF-9 is now complete. The Expert has utilized the AI Service Generator (justifying the RQ2 research intervention for AI-assisted tooling) to efficiently create a structured, professional service listing. The listing is live on the marketplace, tagged with verified taxonomy data, and the expert is verified as financially ready to receive direct purchases (MF-10).

MF-10: Client Buys Service / Tech Discovery

Detailed Swimlane Flow


| Type: SERVICE_PURCHASE | |
| [Buy Now] | |
+--[4]--+
| POST /engagements/buy-service |
| Create Engagement:
| type: SERVICE_PURCHASE
| state: PENDING
| Create Single Milestone:
| amount: 15,000,000
| state: AWAITING_PAYMENT
=== DIRECT PAYMENT & ESCROW ======|
+--[5]--+
| UI prompts: "Fund to start" | |
| Clicks [Fund Escrow] | |
+--[6]--+
| Generate Fixed-Amount VA via |
| SePay API (15,000,000 VND, |
| expires: now() + 24h)
+--[7]--
| Returns VA number.
| INSERT virtual_accounts
| Generate VietQR
+--[8]--+
| Scans VietQR with Banking App | |
| Transfers 15,000,000 VND | |


v
processes. Credit on VA.
fires.
[10]
IPN Handler (Atomic TX):
[CREDIT_WALLET]: avail += 15M
[ESCROW_LOCK]: avail -= 15M,
locked += 15M
UPDATE milestones: FUNDED
UPDATE engagements: ACTIVE
=== EXPERT DELIVERS ==========
[11]
Notified: New Service Purchased
Delivers work.
[Submit Deliverable]
[12]
UPDATE milestones: SUBMITTED
Notify CEO
=== CEO APPROVES & RELEASE =======
[13]
Reviews deliverable.
Clicks [Approve & Release]
[14]
UPDATE milestones: APPROVED
ESCROW RELEASE (Atomic TX):
[ESCROW_RELEASE]: locked -=15M
[PLATFORM_FEE]: += 750K (5%)

Step-by-Step Narration

BROWSE & SELECT SERVICE

---

[1] - Navigates to "AI Marketplace"

- **Actor:** Client / CEO
- **Input:** CEO clicks the "Marketplace" or "Browse AI Services" link from their dashboard.
- **Logic:** Frontend requests the list of published services.
- **Output:** `GET /marketplace/services` request sent.
- **State Change:** None.

---

[2] - Fetch Published Services (No Subscription Guard)

- **Actor:** System (NestJS)
- **Input:** Marketplace request from [1].
- **Logic:**
  - **Subscription Guard Skipped:** Per Section 0.9, browsing the marketplace and buying services is available on the **Client Free** tier. The system does not check `subscription_client_tier` here.
  - Fetch services where `state = 'PUBLISHED'`. Apply any search filters (domain tags, seam tags, price range).
- **Output:** Array of service cards returned.
- **State Change:** None.

---

[3] - Views Service Card & Clicks "Buy Now"

- **Actor:** Client / CEO
- **Input:** CEO views the service created in MF-9: "End-to-End Legal RAG Setup" for 15,000,000 VND.

- **Logic:** CEO decides to purchase directly without elicitation or bidding (Path B). They click **[Buy Now]**.
- **Output:** `POST /engagements/buy-service` with `{ service_id: "..." }`.
- **State Change:** None.

---

[4] - Create Engagement & Milestone (PENDING / AWAITING_PAYMENT)

- **Actor:** System (NestJS)
- **Input:** Purchase request from [3].
- **Logic:** The system initializes the direct purchase flow structures.
    1. **Create Engagement:**
        - `type: "SERVICE_PURCHASE"` (Per Section 0.6: no elicitation; no bid; single fixed-price milestone)
        - `state: "PENDING"`
        - `expert_id` from service
        - `project_id: NULL` (No elicitation project is created for Path B)
    2. **Create Single Milestone:** Tied directly to the engagement.
        - `amount: 15,000,000` (Inherited from service price)
        - `state: "AWAITING_PAYMENT"`
        - `type: "BUSINESS"` (CEO holds sole sign-off authority for service purchases; no Tech Team involved)
- **Output:** Engagement and Milestone created. UI prompted to show payment QR.
- **State Change:** `engagements` gains one row (`PENDING`). `milestones` gains one row (`AWAITING_PAYMENT`).
- **Tables Affected:** `engagements`, `milestones`

DIRECT PAYMENT & ESCROW

---

[5] - Clicks "Fund Escrow"

- **Actor:** Client / CEO
- **Input:** The UI displays the payment step. CEO clicks **[Fund Escrow]**.
- **Logic:** Frontend requests the VietQR for this specific purchase.
- **Output:** `POST /milestones/{id}/fund` request sent.
- **State Change:** None.

---

[6] - Generate Fixed-Amount VA via SePay API

- **Actor:** System (NestJS)
- **Input:** Fund request from [5].
- **Logic:** To ensure the client pays the exact amount required for the service:
    1. Call SePay Bank Hub API to create a Virtual Account.
    2. **Constraints:** `fixed_amount = 15,000,000` (Bank will reject any other amount), `expires_at = now() + 24h`.
    3. `INSERT INTO virtual_accounts (entity_type: 'MILESTONE', fixed_amount: 15000000, status: 'ACTIVE')`
    4. Generate VietQR encoding this VA.
- **Output:** Return VietQR image/URL to CEO.
- **State Change:** `virtual_accounts` gains one row.
- **Tables Affected:** `virtual_accounts`

---

[7] - SePay Returns VA Number

- **Actor:** SePay / Bank Hub
- **Input:** VA creation request.
- **Logic:** SePay creates the sub-account.
- **Output:** VA number returned.
- **State Change:** External - VA created.

---

[8] - Scans VietQR & Transfers

- **Actor:** Client / CEO
- **Input:** VietQR on screen.
- **Logic:** CEO scans with banking app. The app enforces the 15,000,000 VND amount. CEO confirms OTP.
- **Output:** Bank transfer initiated.
- **State Change:** None.

---

[9] - Bank Processes & SePay IPN Fires

- **Actor:** SePay
- **Input:** Bank transfer completion.
- **Logic:** SePay detects credit on the fixed-amount VA and fires IPN.
- **Output:** `POST /webhook/sepay` with IPN payload.

- **State Change:** External.

---

[10] - IPN Handler: Credit Wallet, Escrow Lock, Activate Engagement

- **Actor:** System (NestJS)
- **Input:** IPN payload from [9].
- **Logic:** Atomic transaction to secure funds and activate the work contract.

```
BEGIN;
-- 1. Verify IPN, resolve VA -> Milestone -> CEO wallet
-- 2. Credit & Lock Escrow
UPDATE wallets SET available_balance = available_balance + 15000000 WHERE user_id = ceo_id;
INSERT INTO wallet_transactions (type: 'TOP_UP', amount: 15000000);
UPDATE wallets SET available_balance = available_balance - 15000000,
                   locked_balance = locked_balance + 15000000 WHERE user_id = ceo_id;
INSERT INTO wallet_transactions (type: 'ESCROW_LOCK', amount: 15000000);

-- 3. Update Milestone
UPDATE milestones SET state = 'FUNDED' WHERE id = ?;
UPDATE milestones SET state = 'IN_PROGRESS' WHERE id = ?;

-- 4. Update Engagement
UPDATE engagements SET state = 'ACTIVE' WHERE id = ?;
COMMIT;
```

- **Output:** Escrow locked. Engagement `ACTIVE`. Expert notified.
- **State Change:** `wallets` updated (+15M avail, -15M avail, +15M locked). `milestones` updated (`IN_PROGRESS`). `engagements` updated (`ACTIVE`).
- **Tables Affected:** `wallets`, `wallet_transactions`, `milestones`, `engagements`

EXPERT DELIVERS

---

[11] - Expert Notified & Submits Deliverable

- **Actor:** Expert
- **Input:** Notification of new active service engagement.
- **Logic:** Expert completes the RAG setup and submits the deliverables (code, configuration, documentation) via the platform.
- **Output:** `POST /milestones/{id}/submit` with deliverables.
- **State Change:** None.

---

[12] - Update Milestone (SUBMITTED)

- **Actor:** System (NestJS)
- **Input:** Submission from [11].
- **Logic:** `UPDATE milestones SET state = 'SUBMITTED' WHERE id = ?`. Notify CEO.
- **Output:** Milestone state updated. CEO notified.
- **State Change:** `milestones` updated (`SUBMITTED`).
- **Tables Affected:** `milestones`

CEO APPROVES & RELEASE

---

[13] - CEO Reviews & Approves

- **Actor:** Client / CEO
- **Input:** Review request on dashboard.
- **Logic:** CEO verifies the service was rendered (e.g., RAG pipeline is running). Since this is a `SERVICE_PURCHASE` with a `BUSINESS` milestone type, CEO is the sole sign-off authority. No Tech Team sign-off is required. Clicks **[Approve & Release]**.
- **Output:** `POST /milestones/{id}/signoff/business` request sent.
- **State Change:** None.

---

[14] - Execute Escrow Release

- **Actor:** System (NestJS)
- **Input:** Approval from [13].
- **Logic:** Atomic ledger transaction to pay the expert.

```
BEGIN;
UPDATE milestones SET state = 'APPROVED' WHERE id = ?;
```

```
-- Escrow Release
UPDATE wallets SET locked_balance = locked_balance - 15000000 WHERE user_id = ceo_id;
INSERT INTO wallet_transactions (type: 'ESCROW_RELEASE', amount: 15000000);

-- Platform Fee (5%)
UPDATE platform_fees SET balance = balance + 750000;
INSERT INTO wallet_transactions (type: 'PLATFORM_FEE', amount: 750000);

-- Credit Expert (95%)
UPDATE wallets SET available_balance = available_balance + 14250000 WHERE user_id = expert_id;
INSERT INTO wallet_transactions (type: 'CREDIT_EXPERT', amount: 14250000);
COMMIT;
```

- **Output:** Funds released to Expert wallet.
- **State Change:** `wallets` updated (Client locked -15M, Expert avail +14.25M). `milestones` updated (`APPROVED`).
- **Tables Affected:** `wallets`, `wallet_transactions`, `milestones`, `platform_fees`

AUTO-WITHDRAWAL (CHI HỘ)

---

[15] - Trigger Auto-Withdrawal

- **Actor:** System (NestJS)
- **Input:** Escrow release completion from [14].
- **Logic:** System immediately initiates chi hộ to the Expert's linked bank account.
  1. Debit Expert wallet: `available_balance -= 14250000`.
  2. `POST SePay Chi Hộ API` with `bank_account_xid`.
- **Output:** API call dispatched.
- **State Change:** `wallets` updated (Expert avail -14.25M). `withdrawal_requests` gains row (`PENDING`).

---

[16] - SePay Transfers & Credit IPN Fires

- **Actor:** SePay
- **Input:** Chi hộ request.
- **Logic:** Bank transfers funds to Expert's personal account. IPN fires.
- **Output:** IPN payload sent to NestJS.
- **State Change:** External.

---

[17] - Settle Withdrawal & Close Milestone

- **Actor:** System (NestJS)
- **Input:** IPN from [16].
- **Logic:** Final settlement.
  - `UPDATE withdrawal_requests SET state = 'COMPLETED'`
  - `UPDATE milestones SET state = 'RELEASED'`
- **Output:** Lifecycle complete.
- **State Change:** `withdrawal_requests` updated (`COMPLETED`). `milestones` updated (`RELEASED`).
- **Tables Affected:** `withdrawal_requests`, `milestones`

---

[18] - Engagement CLOSED

- **Actor:** Client / CEO
- **Input:** Dashboard update.
- **Logic:** The direct service purchase is fully settled and closed.
- **Output:** Engagement ends.
- **State Change:** None.

Alternative Path Note: TECH_DISCOVERY

If the CEO had selected a `TECH_DISCOVERY` service instead of a standard `SERVICE_PURCHASE`, the flow differs slightly based on the "availability-based" constraint:

1. **Step 4 Change:** Engagement `type = TECH_DISCOVERY`.
2. **Pre-Payment Availability Check:** Before generating the VA (Step 6), the system places the engagement in a `SCHEDULING` state. The Expert must confirm their availability for the discovery session within a timeframe.
3. **Payment:** Once Expert confirms, the flow resumes at Step 6 (Fund Escrow).
4. **Deliverable:** The milestone deliverable is typically the architectural discovery report and recommendations, which the CEO then uses to potentially initiate a full Path A project (MF-4).

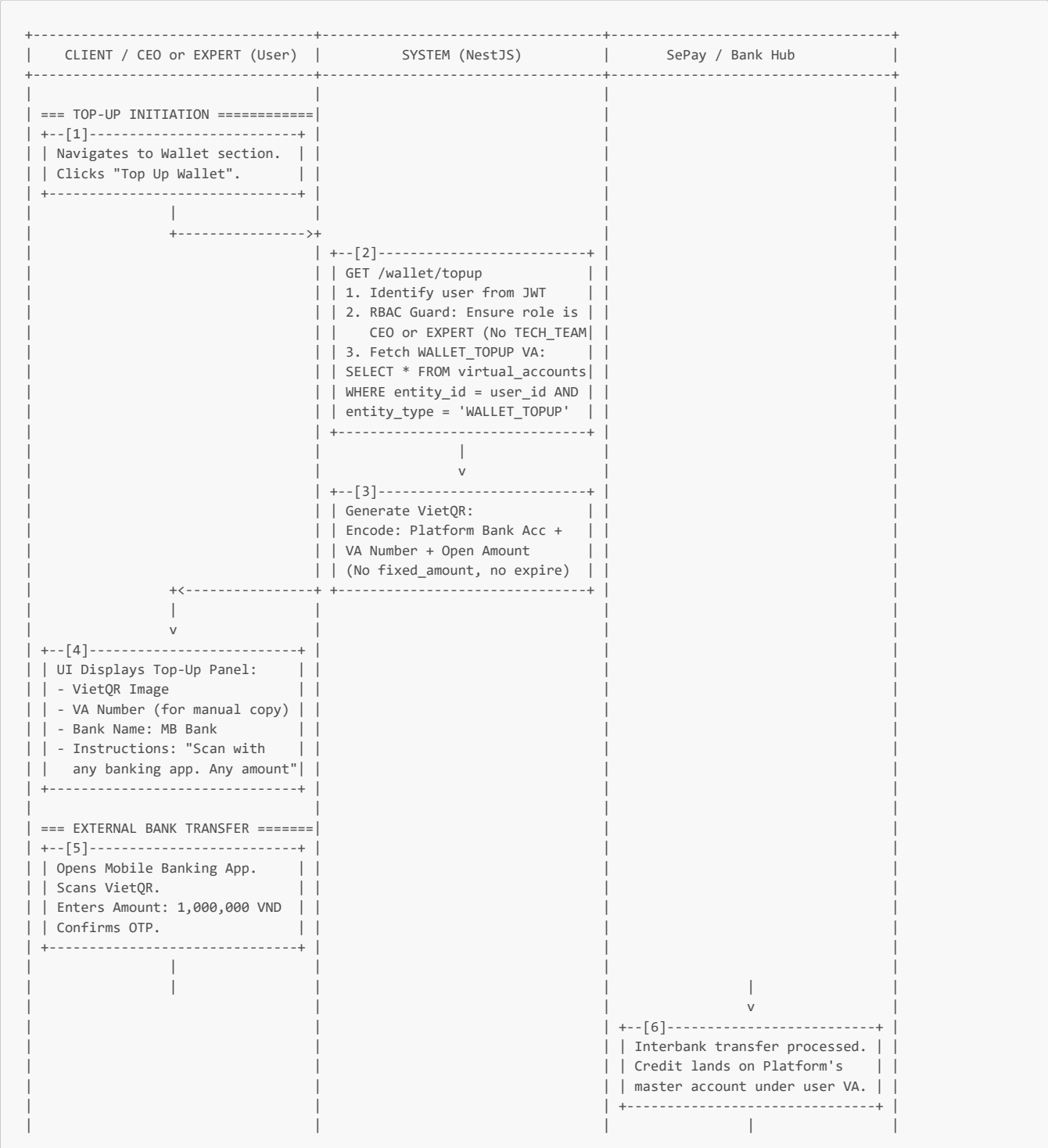
Summary: End-to-End State Trace for MF-10

Step	engagements	milestones	wallets (Client)	wallets (Expert)
[4] Init	state=PENDING (SERVICE_PURCHASE)	state=AWAITING_PAYMENT	-	-
[10] Funded	state=ACTIVE	state=IN_PROGRESS	locked += 15M	-
[12] Submitted	-	state=SUBMITTED	-	-
[14] Approved	-	state=APPROVED	locked -= 15M	avail += 14.25M
[17] Released	-	state=RELEASED	-	avail -= 14.25M (Chi hộ)

MF-10 is now complete. The Client has successfully purchased a direct service from the marketplace without needing a Pro subscription, undergoing elicitation, or bidding. The escrow secured the funds, the expert delivered, and the automated payout (chi hộ) settled the transaction with zero admin friction.

MF-11: Wallet Top-Up

Detailed Swimlane Flow



Step-by-Step Narration

TOP-UP INITIATION

[1] - Navigates to Wallet, Clicks "Top Up Wallet"

- **Actor:** Client / CEO or Expert (User)
- **Input:** User clicks the "Top Up Wallet" button on their respective dashboard.
- **Logic:** Frontend prepares to request the top-up QR code.
- **Output:** `GET /wallet/topup` request sent.
- **State Change:** None.

---

[2] - Fetch WALLET_TOPUP Virtual Account

- **Actor:** System (NestJS)
- **Input:** Request from [1], JWT identifying the user.
- **Logic:**
  1. **RBAC Guard:** Verify the user's role. Per Section 0.7, only `CLIENT / CEO` and `EXPERT` can top up wallets. `CLIENT / TECH_TEAM` and `ADMIN` are blocked (`403 FORBIDDEN`).
  2. **Fetch VA:** Query the `virtual_accounts` table for the permanent top-up VA created during registration (MF-1 Step 7 or MF-2 Step 4).
     - `SELECT va_number FROM virtual_accounts WHERE entity_id = ? AND entity_type = 'WALLET_TOPUP' AND status = 'ACTIVE'`
- **Output:** `va_number` retrieved.
- **State Change:** None.

---

[3] - Generate VietQR

- **Actor:** System (NestJS)
- **Input:** `va_number` from [2].
- **Logic:** The system generates a VietQR string/image. This QR encodes:
  - Platform's master bank account number
  - The user's unique `va_number` as the sub-account
  - **Open Amount:** No fixed amount is encoded. The user will manually enter the amount in their banking app.
  - **Memo/Description:** Auto-filled with "AITasker TopUp {user_id}"
- **Output:** Return QR image URL/base64 + VA details to frontend.
- **State Change:** None.

---

[4] - UI Displays Top-Up Panel

- **Actor:** User
- **Input:** QR data from [3].
- **Logic:** The UI renders the Top-Up panel showing the VietQR, the VA number (for manual copy/paste if the user can't scan), the bank name, and instructions: *"Scan with any banking app. Any amount will be credited to your wallet."*
- **Output:** Panel displayed. User ready to scan.
- **State Change:** None.

EXTERNAL BANK TRANSFER

---

[5] - Scans VietQR & Transfers

- **Actor:** User (via personal mobile banking app)
- **Input:** The VietQR on screen.
- **Logic:** The user opens their banking app, scans the QR, manually enters the amount (e.g., 1,000,000 VND), and confirms the transfer via OTP. This step occurs **entirely outside** the platform.
- **Output:** Bank transfer instruction sent.
- **State Change:** None on the platform.

---

[6] - Bank Processes Transfer

- **Actor:** Banking Infrastructure
- **Input:** Transfer instruction from [5].
- **Logic:** The interbank system clears the transfer. The credit arrives on the platform's master bank account under the user's specific VA number.
- **Output:** Credit confirmed on VA.
- **State Change:** External - funds moved.

---

[7] - SePay Detects Credit & Fires IPN

- **Actor:** SePay / Bank Hub
- **Input:** Credit event on VA.
- **Logic:** SePay monitors the platform's master account. When a credit matching a registered VA is detected, it automatically fires an Instant Payment Notification (IPN) webhook.
- **Output:** `POST /webhook/sepay` sent to NestJS.
- **State Change:** External - IPN dispatched.

---

[8] - Receive IPN Payload

- **Actor:** System (NestJS - SePayWebhookController)
- **Input:** SePay IPN payload.
- **Logic:** The payload contains critical mapping data:

```
{
  "va_number": "XXX_USER_VA_XXX",
  "amount": 1000000,
  "transfer_type": "in",
  "transfer_reference": "FT25000XXXXX",
  "description": "AITasker TopUp user_id",
  "timestamp": "2026-05-20T14:00:00Z"
}
```

- **Output:** Payload received.
- **State Change:** None.

---

[9] - Verify Signature & Resolve VA -> User

- **Actor:** System (NestJS)
- **Input:** IPN payload from [8].
- **Logic:**
  1. **Security Check:** Verify the webhook's HMAC signature using the platform's SePay API key. If invalid, log the attempt and return 200 OK (to prevent malicious retries) but take no financial action.
  2. **Resolve Mapping:** Look up the `va_number` in the `virtual_accounts` table to find the `entity_id` (which is the `user_id`) and confirm `entity_type = 'WALLET_TOPUP'`.
- **Output:** IPN validated and mapped to a specific user's wallet.
- **State Change:** None.

---

[10] - Idempotency Check

- **Actor:** System (NestJS)
- **Input:** `transfer_reference` from IPN.
- **Logic:** Payment gateways like SePay may send the same IPN multiple times if they don't receive a timely 200 OK response. The system must handle this gracefully to prevent double-crediting the wallet.
  - `SELECT 1 FROM wallet_transactions WHERE reference_id = 'FT25000XXXXX'`
  - If a row exists → This is a duplicate IPN. Return 200 OK immediately, take no further action.
  - If no row exists → Proceed to ledger update.
- **Output:** Uniqueness confirmed .
- **State Change:** None.

---

[11] - Atomic Ledger Update

- **Actor:** System (NestJS - WalletService)
- **Input:** Validated IPN data, `user_id`, `amount`.
- **Logic:** This is the core financial transaction. It must be strictly atomic to prevent data corruption.

```
BEGIN TRANSACTION;

-- 1. Credit the wallet
UPDATE wallets
  SET available_balance = available_balance + 1000000
  WHERE user_id = ?;

-- 2. Write the immutable ledger entry
INSERT INTO wallet_transactions
  (wallet_id, amount, transaction_type, reference_id, created_at)
  VALUES (?, 1000000, 'TOP_UP', 'FT25000XXXXX', now());

COMMIT;
```

- **Output:** Transaction committed. Wallet balance updated.
- **State Change:** `wallets.available_balance` increased by 1,000,000. `wallet_transactions` gains one row (`type: TOP_UP`).
- **Tables Affected:** `wallets`, `wallet_transactions`

---

[12] - Acknowledge IPN & Notify User

- **Actor:** System (NestJS)
- **Input:** Successful commit from [11].
- **Logic:**
  1. **Acknowledge SePay:** Return HTTP 200 OK to the webhook sender. This tells SePay the IPN was processed successfully and no retry is needed.
  2. **Notify User:** Push an in-app notification: "*Your wallet has been credited with 1,000,000 VND.*"
- **Output:** IPN cycle complete. User notified.
- **State Change:** None.

---

[13] - Wallet UI Updates

- **Actor:** User
- **Input:** In-app notification / WebSocket push / Dashboard polling.
- **Logic:** The user's dashboard updates to reflect the new available balance. The transaction appears in the "Transaction History" table.
- **Output:** User sees updated funds.
- **State Change:** None (UI reflects DB state).

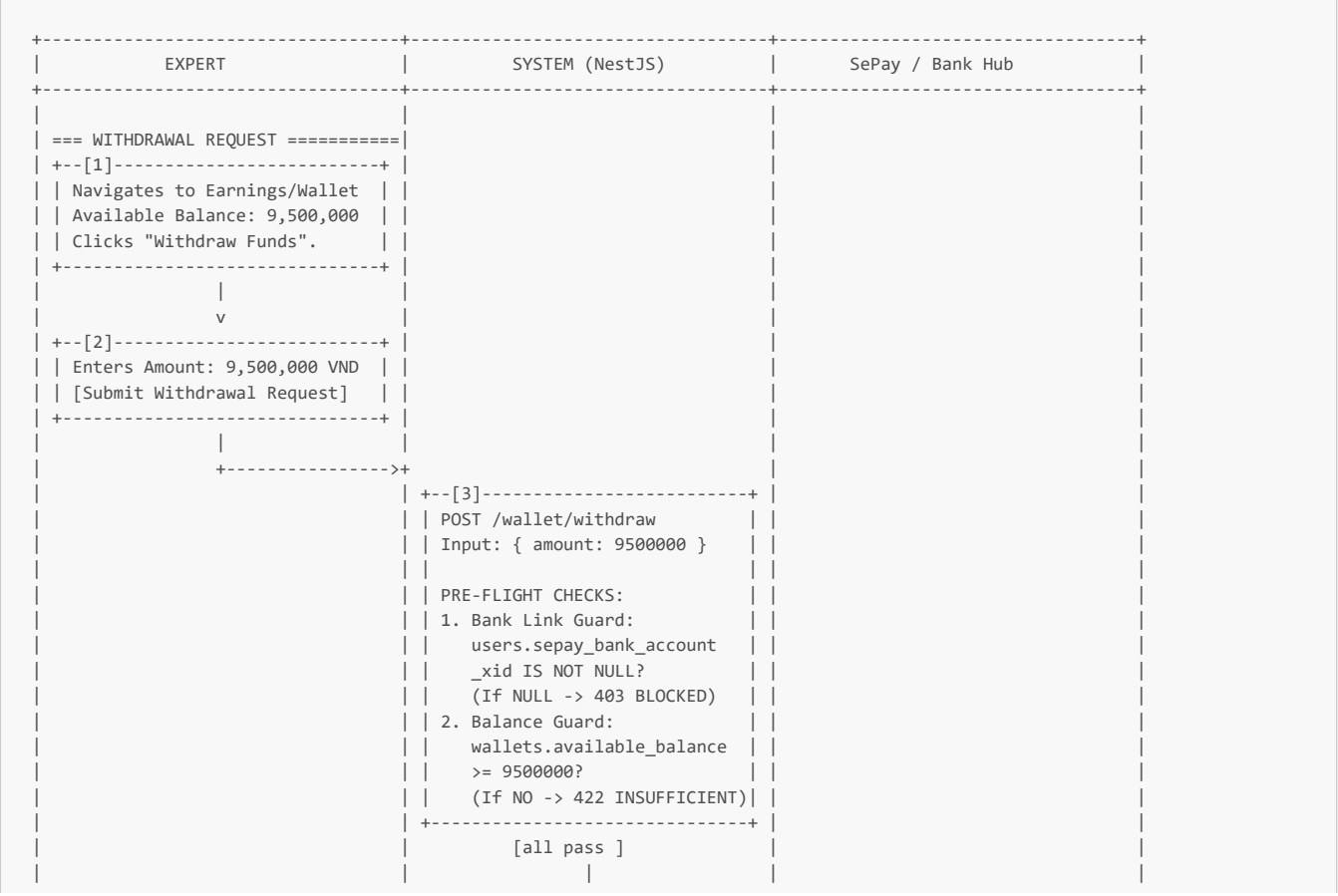
Summary: End-to-End State Trace for MF-11

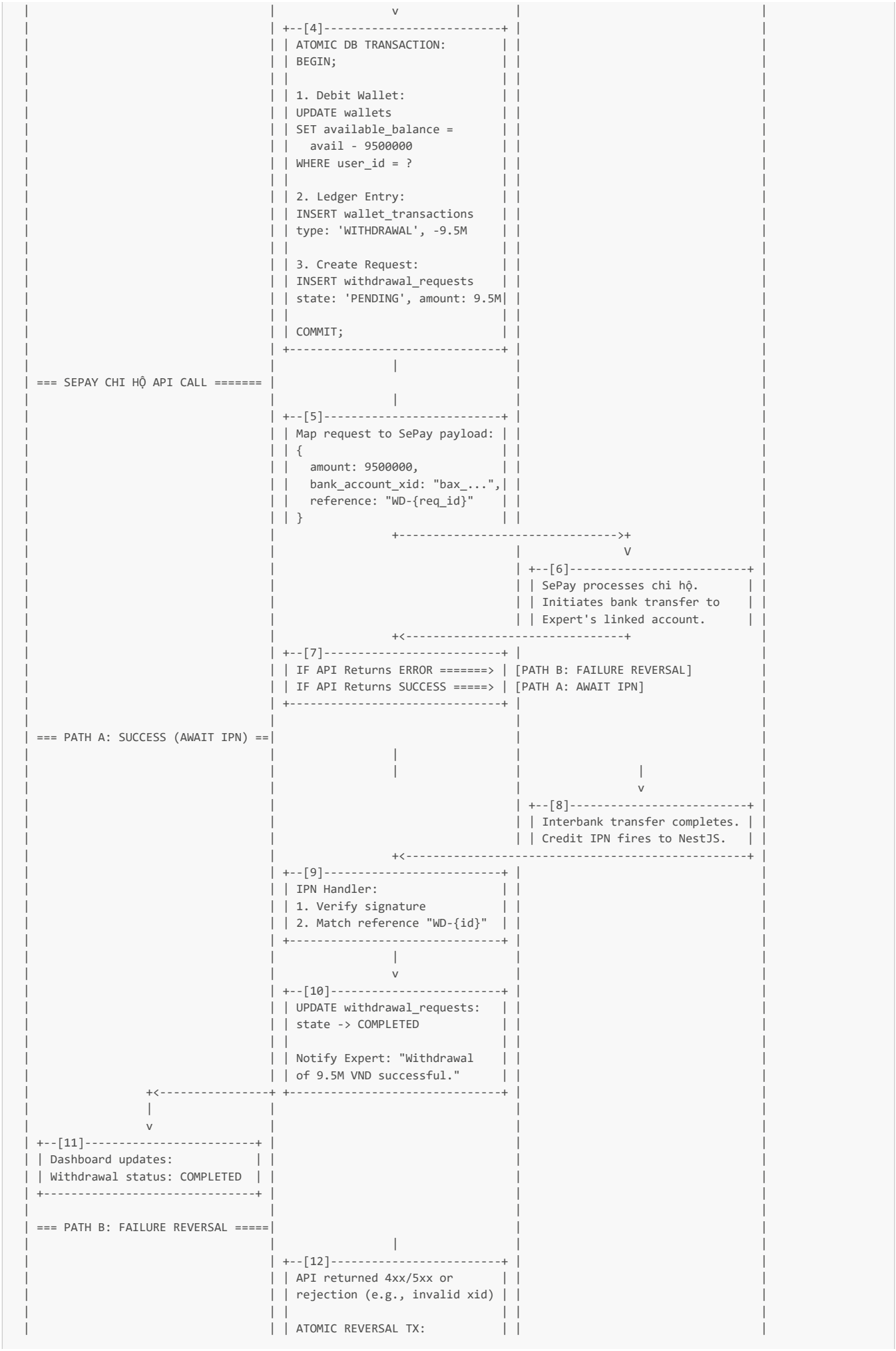
Step	`virtual_accounts`	`wallets`	`wallet_transactions`	User View
[2] Init	Read `WALLET_TOPUP` VA	-	-	-
[11] Ledger Update	-	available_balance += 1,000,000	type=TOP_UP, amount=1,000,000, ref=FT25000...	-
[13] UI Update	-	-	-	Balance increased, history updated

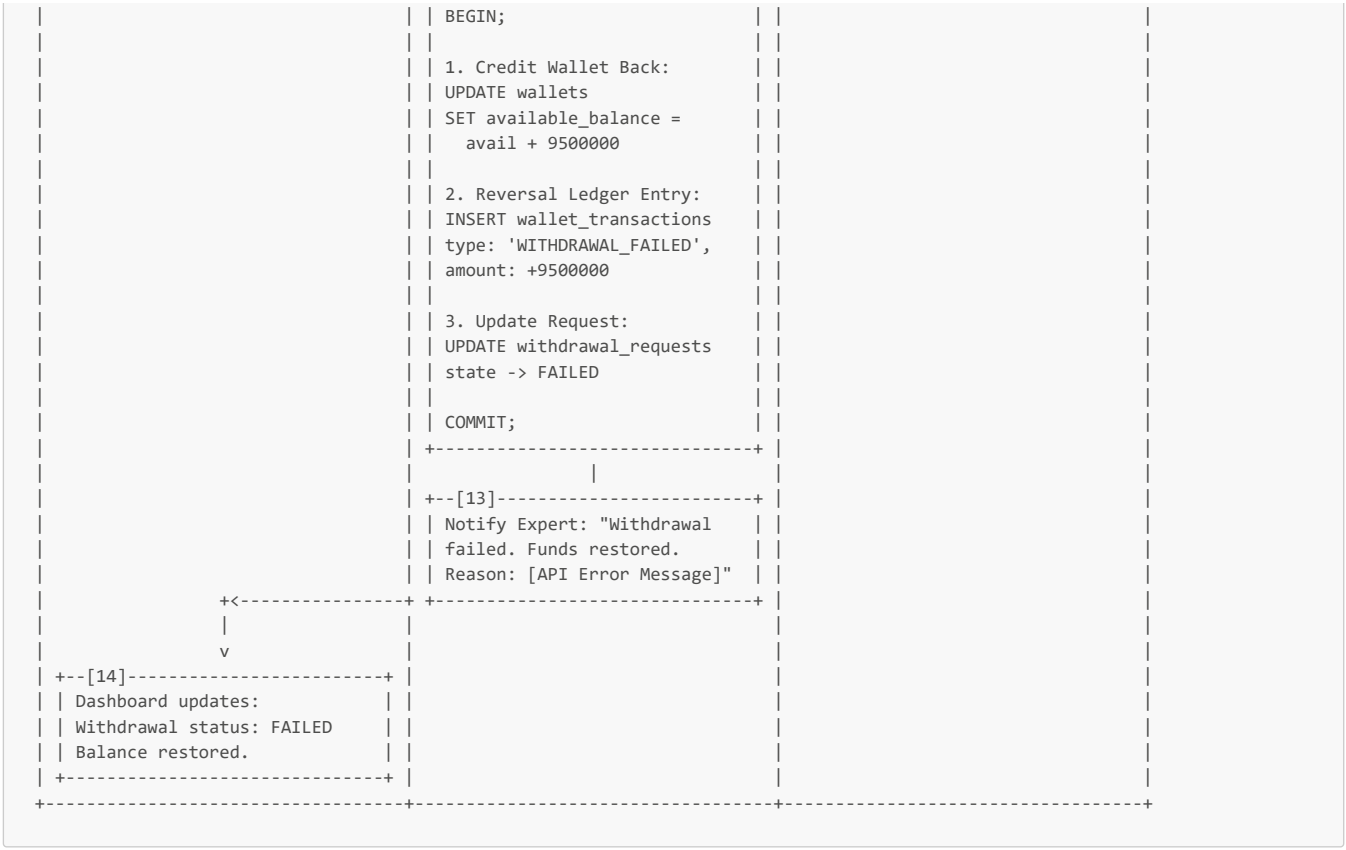
MF-11 is now complete. The user's wallet has been securely credited via a fully automated, zero-admin pipeline. The permanent VA mapping ensures instant attribution, the idempotency check prevents double-spending from webhook retries, and the atomic ledger maintains strict financial integrity. The user can now use these funds for subscriptions (MF-1/2), milestone funding (MF-7), or service purchases (MF-10).

MF-12: Expert Withdrawal (Automated Chi Hộ)

Detailed Swimlane Flow




v
[4] ATOMIC DB TRANSACTION:
BEGIN;
1. Debit Wallet:
UPDATE wallets
SET available_balance =
avail - 9500000
WHERE user_id = ?
2. Ledger Entry:
INSERT wallet_transactions
type: 'WITHDRAWAL', -9.5M
3. Create Request:
INSERT withdrawal_requests
state: 'PENDING', amount: 9.5M
COMMIT;
=== SEPAY CHI HỘ API CALL =======
[5] Map request to SePay payload:
{
amount: 9500000,
bank_account_xid: "bax_...",
reference: "WD-{req_id}"
}
[6] SePay processes chi hộ.
Initiates bank transfer to
Expert's linked account.
[7]
IF API Returns ERROR =======>
IF API Returns SUCCESS =====>
[PATH B: FAILURE REVERSAL]
[PATH A: AWAIT IPN]
=== PATH A: SUCCESS (AWAIT IPN) ==
[8] Interbank transfer completes.
Credit IPN fires to NestJS.
[9] IPN Handler:
1. Verify signature
2. Match reference "WD-{id}"
[10] UPDATE withdrawal_requests:
state -> COMPLETED
Notify Expert: "Withdrawal
of 9.5M VND successful."
[11] Dashboard updates:
Withdrawal status: COMPLETED
=== PATH B: FAILURE REVERSAL =====
[12] API returned 4xx/5xx or
rejection (e.g., invalid xid)
ATOMIC REVERSAL TX:



Step-by-Step Narration

WITHDRAWAL REQUEST

---

[1] - Navigates to Earnings/Wallet, Clicks "Withdraw Funds"

- **Actor:** Expert
- **Input:** Expert views their Earnings dashboard and sees `Available Balance: 9,500,000 VND`. They click the "Withdraw Funds" button.
- **Logic:** Frontend opens the withdrawal form.
- **Output:** None.
- **State Change:** None.

---

[2] - Enters Amount & Submits

- **Actor:** Expert
- **Input:** Expert enters the amount they wish to withdraw (e.g., 9,500,000 VND) and clicks `[Submit Withdrawal Request]`.
- **Logic:** Frontend performs basic validation (amount > 0, amount <= available_balance) and sends the request.
- **Output:** `POST /wallet/withdraw` with `{ amount: 9500000 }`.
- **State Change:** None.

---

[3] - Pre-Flight Checks

- **Actor:** System (NestJS)
- **Input:** Withdrawal request from [2], JWT (`active_role = "EXPERT"`).
- **Logic:** The system executes two critical guard checks before touching any money:
  1. **Bank Link Guard:** Query `users.sepay_bank_account_xid`. If `NULL`, the expert has not completed the Bank Hub Hosted Link flow (MF-2 Phase F). Return `403 { code: "BANK_ACCOUNT_NOT_LINKED", action: "link_bank_account" }`. You cannot withdraw if there is nowhere for the money to go.
  2. **Balance Guard:** Query `wallets.available_balance`. If `< 9500000`, return `422 { code: "INSUFFICIENT_BALANCE" }`.
- **Output:** Guards pass .
- **State Change:** None.

---

[4] - Atomic Ledger Debit (Wallet -> Pending Withdrawal)

- **Actor:** System (NestJS - WalletService)
- **Input:** Validated request.

- **Logic:** The system immediately deducts the funds from the expert's wallet to prevent double-spend (e.g., if they click withdraw twice quickly). This is an atomic DB transaction.

```
BEGIN TRANSACTION;

-- 1. Debit Wallet
UPDATE wallets
  SET available_balance = available_balance - 9500000
  WHERE user_id = ?;

-- 2. Ledger Entry
INSERT INTO wallet_transactions
  (wallet_id, amount, transaction_type, reference_id, created_at)
  VALUES (?, 9500000, 'WITHDRAWAL', 'WD-PENDING-123', now());

-- 3. Create Withdrawal Request Record
INSERT INTO withdrawal_requests
  (user_id, amount, state, bank_account_xid, reference, created_at)
  VALUES (?, 9500000, 'PENDING', 'bax_abc123xyz', 'WD-123', now());

COMMIT;
```

- **Output:** Transaction committed. Funds secured.
- **State Change:** `wallets.available_balance` decreased by 9,500,000. `wallet_transactions` gains a row (`WITHDRAWAL`). `withdrawal_requests` gains a row (`PENDING`).
- **Tables Affected:** `wallets`, `wallet_transactions`, `withdrawal_requests`

SEPAY CHI HỘ API CALL

---

[5] - Map Request to SePay Payload

- **Actor:** System (NestJS)
- **Input:** `withdrawal_request` record from [4].
- **Logic:** NestJS constructs the payload for SePay's chi hộ (disbursement) API. It uses the securely stored `bank_account_xid` rather than raw account numbers.
  - `amount`: 9500000
  - `bank_account_xid`: "bax_abc123xyz"
  - `reference`: "WD-123" (Used to correlate the IPN back to this specific request)
- **Output:** `POST SePay Chi Hộ API` dispatched.
- **State Change:** None.

---

[6] - SePay Processes Chi Hộ

- **Actor:** SePay / Bank Hub
- **Input:** Chi hộ API request.
- **Logic:** SePay validates the xid and initiates the outbound interbank transfer to the Expert's verified bank account.
- **Output:** Initial API response (Success or Error).
- **State Change:** External.

---

[7] - Route based on API Response

- **Actor:** System (NestJS)
- **Input:** HTTP response from SePay API [6].
- **Logic:**
  - If `2xx Success` → **Path A**: The transfer is in flight. Await the Credit IPN.
  - If `4xx/5xx Error` → **Path B**: The transfer was rejected immediately (e.g., xid revoked, bank account closed). Execute Failure Reversal.
- **Output:** Routing decision.
- **State Change:** None.

PATH A: SUCCESS (AWAIT IPN)

---

[8] - Interbank Transfer Completes, Credit IPN Fires

- **Actor:** SePay / Bank Hub
- **Input:** Bank transfer completion.
- **Logic:** When the money successfully lands in the Expert's personal bank account, SePay fires a Credit IPN webhook to NestJS.

- **Output:** `POST /webhook/sepay` with IPN payload containing the reference "WD-123".
- **State Change:** External - funds delivered.

---

[9] - IPN Handler: Verify & Match

- **Actor:** System (NestJS)
- **Input:** IPN payload from [8].
- **Logic:**
  1. Verify IPN signature.
  2. Parse the `reference` field ("WD-123") and match it to the `withdrawal_requests` record.
- **Output:** Withdrawal request identified.
- **State Change:** None.

---

[10] - Update Request to COMPLETED

- **Actor:** System (NestJS)
- **Input:** Matched IPN from [9].
- **Logic:** The lifecycle is fully settled.
  - `UPDATE withdrawal_requests SET state = 'COMPLETED', completed_at = now() WHERE id = 123`
  - Notify Expert: "*Withdrawal of 9,500,000 VND to your linked bank account was successful.*"
- **Output:** Request closed. Expert notified.
- **State Change:** `withdrawal_requests` updated (`COMPLETED`).
- **Tables Affected:** `withdrawal_requests`

---

[11] - Dashboard Updates (Success)

- **Actor:** Expert
- **Input:** Notification / Dashboard refresh.
- **Logic:** The Expert sees the withdrawal status is "Completed" and the money is in their bank.
- **Output:** None.
- **State Change:** None.

PATH B: FAILURE REVERSAL

---

[12] - Atomic Reversal Transaction

- **Actor:** System (NestJS - WalletService)
- **Input:** API error from [7].
- **Logic:** Because the funds were already deducted in Step [4], they must be immediately restored to the expert's wallet if the payout fails. This is a critical atomic reversal.

```
BEGIN TRANSACTION;

-- 1. Credit Wallet Back
UPDATE wallets
  SET available_balance = available_balance + 9500000
  WHERE user_id = ?;

-- 2. Reversal Ledger Entry
INSERT INTO wallet_transactions
  (wallet_id, amount, transaction_type, reference_id, created_at)
  VALUES (?, 9500000, 'WITHDRAWAL_FAILED', 'WD-123-REVERSAL', now());

-- 3. Update Request
UPDATE withdrawal_requests
  SET state = 'FAILED', failure_reason = 'SePay API: Invalid bank account xid'
  WHERE id = 123;

COMMIT;
```

- **Output:** Funds restored. Ledger updated.
- **State Change:** `wallets.available_balance` increased by 9,500,000. `wallet_transactions` gains a row (`WITHDRAWAL_FAILED`). `withdrawal_requests` updated (`FAILED`).
- **Tables Affected:** `wallets`, `wallet_transactions`, `withdrawal_requests`

---

[13] - Notify Expert of Failure

- **Actor:** System (NestJS)
- **Input:** Reversal completion from [12].
- **Logic:** Notify the expert that the payout failed and their balance has been restored.
- **Output:** Notification sent.
- **State Change:** None.

---

[14] - Dashboard Updates (Failure)

- **Actor:** Expert
- **Input:** Notification / Dashboard refresh.
- **Logic:** The Expert sees the withdrawal status is "Failed", their available balance is restored, and the error reason is displayed.
- **Output:** None.
- **State Change:** None.

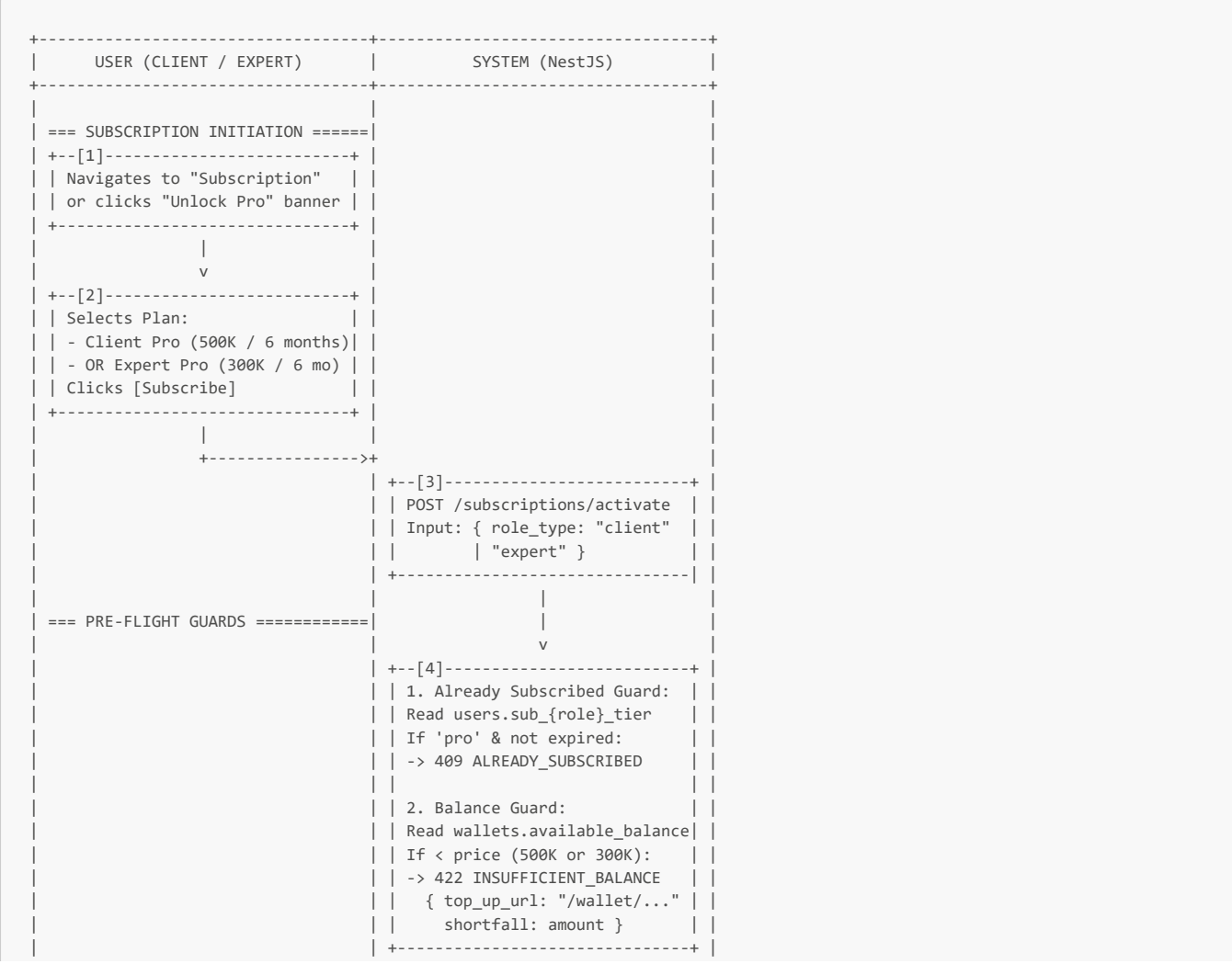
Summary: End-to-End State Trace for MF-12

Step	`withdrawal_requests`	`wallets` **(Expert)**	`wallet_transactions`	External Bank
[4] Atomic Debit	state=PENDING	available -= 9,500,000	type=WITHDRAWAL, -9.5M	-
[10] IPN Success	state=COMPLETED	-	-	Funds received
[12] API Failure	state=FAILED	available += 9,500,000	type=WITHDRAWAL_FAILED, +9.5M	-

MF-12 is now complete. The Expert's withdrawal has been processed via a fully automated, zero-admin pipeline. The "debit-first, disburse-second" pattern prevents double-spend, the `bank_account_xid` ensures secure routing without exposing raw bank details, and the atomic reversal pattern guarantees financial integrity if the external API rejects the transfer.

MF-13: Subscription Purchase from Wallet

Detailed Swimlane Flow



Step-by-Step Narration

SUBSCRIPTION INITIATION

[1] - Navigates to Subscription Page

- **Actor:** User (Client / CEO or Expert)
- **Input:** User clicks a CTA such as "Unlock Pro features", "Subscribe", or navigates to Account Settings -> Subscription.
- **Logic:** Frontend loads the subscription tier comparison page.
- **Output:** None.
- **State Change:** None.

[2] - Selects Plan & Clicks Subscribe

- **Actor:** User
- **Input:** User views the plans based on their active role context:
  - **Client Pro:** 500,000 VND / 6 months (Unlocks Elicitation, Matching, Artifact B, Add-On Protocol)
  - **Expert Pro:** 300,000 VND / 6 months (Unlocks Tier 2/3 Verification, AI Service Generator, Tier 2+ Bidding)
  - *Note: Dual-role users can purchase both, but must do so in separate transactions from their respective dashboard contexts.*
- **Logic:** User clicks `[Subscribe]`. Frontend determines the `role_type` based on the user's `active_role` from their JWT.
- **Output:** `POST /subscriptions/activate` with `{ role_type: "client" }` or `{ role_type: "expert" }`.
- **State Change:** None.

PRE-FLIGHT GUARDS

---

[3] - Receive Activation Request

- **Actor:** System (NestJS - SubscriptionController)
- **Input:** Request from [2], JWT.
- **Logic:** NestJS identifies the user and maps the `role_type` to the corresponding price and database fields.
- **Output:** Guards initialized.
- **State Change:** None.

---

[4] - Already Subscribed & Balance Guards

- **Actor:** System (NestJS)
- **Input:** User record and wallet.
- **Logic:** Two sequential safety checks:
  1. **Already Subscribed Guard:** Read `users.sub_client_tier` (or `sub_expert_tier`) and `sub_client_expires_at`. If the tier is `'pro'` AND `expires_at > now()`, the user is already subscribed. Return `409 CONFLICT { code: "ALREADY_SUBSCRIBED", expires_at: "..." }`. Prevents double-charging.
  2. **Balance Guard:** Read `wallets.available_balance`. Compare against the required price (500K or 300K). If `available_balance < price`, the purchase cannot proceed. Return `422 UNPROCESSABLE_ENTITY { code: "INSUFFICIENT_BALANCE", current_balance: 200000, required: 500000, shortfall: 300000, top_up_url: "/wallet/topup" }`. This directs the user to MF-11 (Wallet Top-Up) before retrying.
- **Output:** Guards pass .
- **State Change:** None.

ATOMIC LEDGER TRANSACTION

---

[5] - Execute Subscription Payment (Atomic TX)

- **Actor:** System (NestJS - SubscriptionService)
- **Input:** Validated request, user_id, price, role_type.
- **Logic:** This is a pure internal ledger operation. No external SePay API calls are made here; the money was already deposited into the platform's master bank account during MF-11 (Wallet Top-Up). This transaction simply reclassifies the internal ledger numbers and updates access rights. It must be strictly atomic.

```
BEGIN TRANSACTION;

-- 1. Debit Wallet
UPDATE wallets
  SET available_balance = available_balance - 500000 -- (or 300000)
  WHERE user_id = ?;

-- 2. Ledger Entry
INSERT INTO wallet_transactions
  (wallet_id, amount, transaction_type, reference_id, created_at)
  VALUES (?, 500000, 'SUBSCRIPTION', 'SUB-CLIENT-{user_id}-20260520', now());

-- 3. Update User Tier
UPDATE users
  SET subscription_client_tier = 'pro',                    -- (or subscription_expert_tier)
      sub_client_expires_at = now() + interval '6 months'  -- (or sub_expert_expires_at)
  WHERE id = ?;

-- 4. Audit Record
INSERT INTO user_subscriptions
  (user_id, role_type, tier, price_paid, activated_at, expires_at)
  VALUES (?, 'client', 'pro', 500000, now(), now() + interval '6 months');

COMMIT;
```

- **Output:** Transaction committed.
- **State Change:**
  - `wallets.available_balance` decreased by subscription price.
  - `wallet_transactions` gains one row (`SUBSCRIPTION`).
  - `users` tier and expiry fields updated.
  - `user_subscriptions` gains one audit row.
- **Tables Affected:** `wallets`, `wallet_transactions`, `users`, `user_subscriptions`

POST-ACTIVATION SETUP

---

[6] - Reissue JWT

- **Actor:** System (NestJS - AuthService)
- **Input:** Updated user record from [5].
- **Logic:** The subscription guard middleware on the backend and the UI feature gating on the frontend both rely on JWT claims to function instantly without re-fetching user data from the DB. The system invalidates the old JWT and issues a new one with updated claims:
  - `subscription_client_tier: "pro"` (or `subscription_expert_tier: "pro"`)
  - `sub_client_expires_at: <timestamp>`
- **Output:** New JWT set as HTTP-only cookie.
- **State Change:** None (Auth stateless update).

---

[7] - Schedule Expiry Warning

- **Actor:** System (NestJS - SchedulerService)
- **Input:** `expires_at` from [5].
- **Logic:** Per Section 0.9, users must be warned 7 days before their subscription expires to give them time to top-up and renew. The system creates a scheduled job (e.g., agenda.js or node-cron) that will trigger a notification at `expires_at - 7 days`.
- **Output:** Cron job scheduled.
- **State Change:** None (Scheduler state).

---

[8] - Push Notification

- **Actor:** System (NestJS - NotificationService)
- **Input:** Successful activation event.
- **Logic:** Send an in-app and email confirmation:
  - "*Your Client Pro subscription is now active! You have unlocked the AI Elicitation Engine, Expert Matching, and all Pro features. Your subscription is valid until 20/11/2026.*"
- **Output:** Notification dispatched.
- **State Change:** None.

---

[9] - Dashboard Refreshes (Pro Unlocked)

- **Actor:** User
- **Input:** New JWT from [6], notification from [8].
- **Logic:** The frontend reads the new JWT claims. All UI elements blocking Pro features (banners, locked buttons, 403 error interceptors) are instantly removed. The Pro badge appears in the navigation bar.
- **Output:** User has full Pro access.
- **State Change:** None.

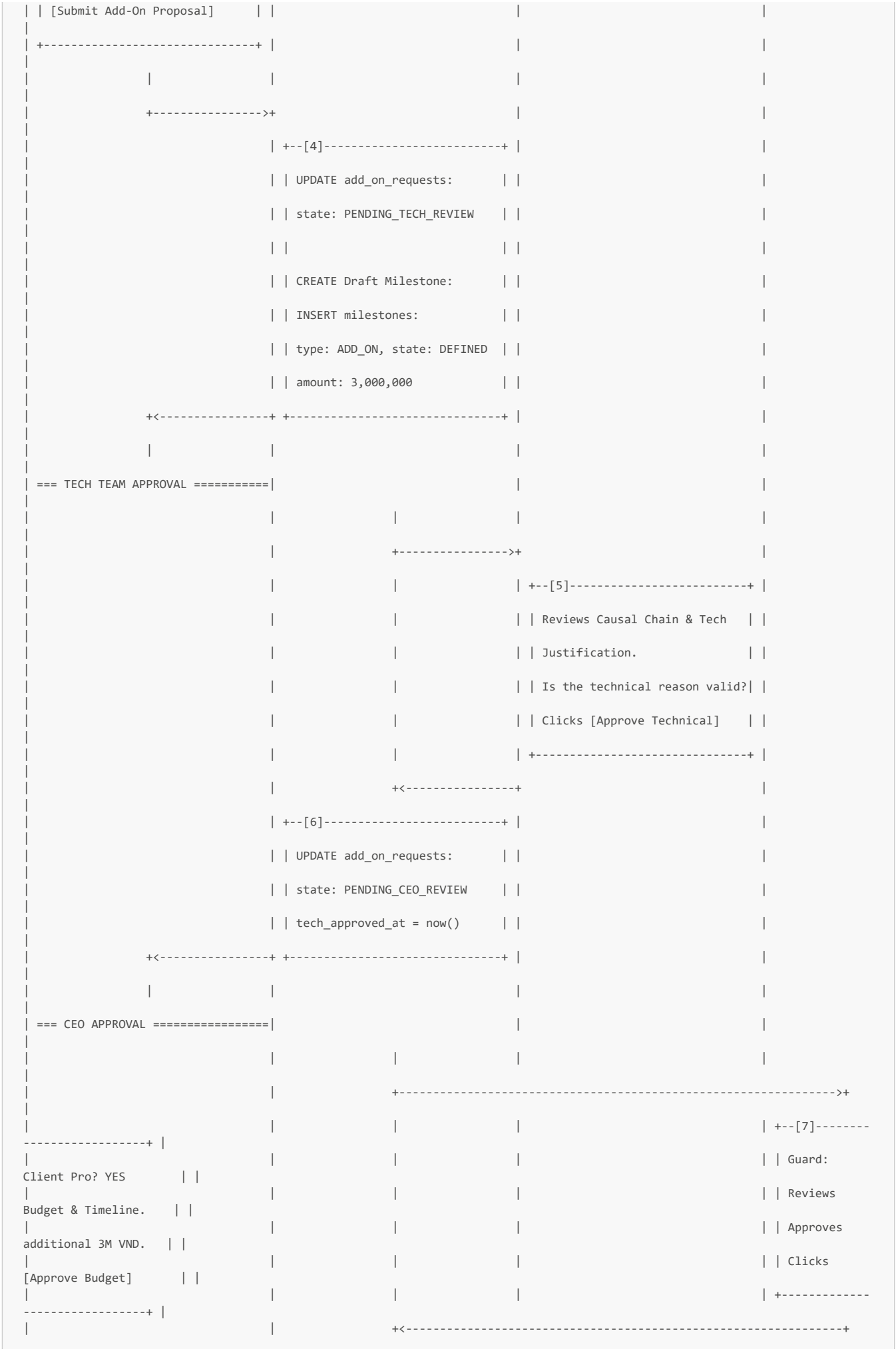
Summary: End-to-End State Trace for MF-13

Step	`wallets`	`wallet_transactions`	`users`	`user_subscriptions`
[4] Pre-Flight	avail >= 500K	-	tier='free'	-
[5] Atomic TX	avail -= 500K	type=SUBSCRIPTION, -500K	tier='pro', expires_at=now+6mo	row inserted (audit)
[6] Reissue JWT	-	-	-	-

MF-13 is now complete. The user has successfully converted their internal wallet balance into a Pro subscription via a strictly atomic, zero-external-API ledger transaction. The JWT is updated for instant frontend/backend guard propagation, and the expiry lifecycle is initialized. The user can now access AI features (MF-4, MF-5, MF-9) or advanced engagement features (MF-6, MF-7, MF-8).

MF-14: Add-On Phase Protocol (Scope Evolution)

Detailed Swimlane Flow


[Submit Add-On Proposal]
[4]
UPDATE add_on_requests:
state: PENDING_TECH_REVIEW
CREATE Draft Milestone:
INSERT milestones:
type: ADD_ON, state: DEFINED
amount: 3,000,000
=== TECH TEAM APPROVAL ===========
[5]
Reviews Causal Chain & Tech
Justification.
Is the technical reason valid?
Clicks [Approve Technical]
[6]
UPDATE add_on_requests:
state: PENDING_CEO_REVIEW
tech_approved_at = now()
=== CEO APPROVAL =================
[7]
Guard:
Reviews
Approves
Clicks
Client Pro? YES
Budget & Timeline.
additional 3M VND.
[Approve Budget]

Step-by-Step Narration

TRIGGER: SCOPE EVOLUTION

---

[1] - Submits Weekly Sprint Update (BLOCKER: SCOPE_EVOLUTION)

- **Actor:** Expert
- **Input:** Expert is working on the current milestone and discovers a structural gap between the spec and the production reality that requires work outside the current scope.
- **Logic:** During the weekly sprint update, the Expert sets the status to `BLOCKER` and selects the specific `blocker_type: SCOPE_EVOLUTION`. They provide a brief note: *"Legacy schema requires a migration layer not covered in the current spec."*
- **Output:** `POST /milestones/{id}/sprints` with `{ status: "BLOCKER", blocker_type: "SCOPE_EVOLUTION", note: "..." }`.
- **State Change:** None.

---

[2] - Pause Clock & Auto-Trigger F8 Protocol

- **Actor:** System (NestJS)
- **Input:** Sprint update from [1].
- **Logic:** This is the core structural intervention defined in Section 0.6 and Section 1.2. AI projects naturally evolve; when scope evolution happens, AITasker treats it as a designed lifecycle event, not a contract violation.
  1. **Pause Review Clock:** The system pauses the milestone's review/submission clock. The Expert is not penalized for the time spent resolving the scope gap.
  2. **Auto-Trigger F8:** The system creates an `add_on_requests` record to track the formal Add-On Phase Protocol.
     - `INSERT INTO add_on_requests (project_id, milestone_id, expert_id, state) VALUES (?, ?, ?, 'PENDING_EXPERT_PROPOSAL')`
  3. **Notify Expert:** Prompt the Expert to formally document the scope evolution.
- **Output:** Clock paused. Add-On Protocol initiated.
- **State Change:** `add_on_requests` gains one row (`PENDING_EXPERT_PROPOSAL`). Milestone timer paused.
- **Tables Affected:** `add_on_requests`, `milestones` (timer logic)

EXPERT PROPOSES ADD-ON

[3] - Fills Add-On Proposal Form

- **Actor:** Expert
- **Input:** The Add-On Proposal form on the Expert Dashboard.
- **Logic:** The Expert must provide a structured justification for the scope change. This is critical for RQ2 research data (spec evolution evidence) and for satisfying the two-tier approval matrix.
    1. **Causal Chain Documentation:** A structured narrative explaining *why* the scope evolved. (e.g., "*The spec assumed direct DB read/write access. Discovery during Sprint 2 revealed the production DB is a read-only replica. Writing requires a migration sync layer.*")
    2. **Technical Justification:** What specifically needs to be built. (e.g., "*Build an asynchronous Go worker to sync state from the primary write DB to the replica before the AI pipeline runs.*")
    3. **Budget & Timeline Impact:** The requested additional amount and time. (e.g., "*+3,000,000 VND, +7 days*").
- **Output:** `POST /add-on-requests/{id}/propose` with full payload.
- **State Change:** None.

---

[4] - Store Proposal & Create Draft Milestone

- **Actor:** System (NestJS)
- **Input:** Proposal payload from [3].
- **Logic:**
    1. **Update Request:** `UPDATE add_on_requests SET causal_chain = '...', tech_justification = '...', proposed_budget = 3000000, proposed_timeline = 7, state = 'PENDING_TECH_REVIEW' WHERE id = ?`
    2. **Create Draft Milestone:** To ensure the escrow and DoD structure is ready if approved, the system creates a draft milestone linked to the add-on request.
        - `INSERT INTO milestones (project_id, parent_milestone_id, type: 'ADD_ON', amount: 3000000, state: 'DEFINED')`
- **Output:** Proposal saved. Tech Team notified.
- **State Change:** `add_on_requests` updated (`PENDING_TECH_REVIEW`). `milestones` gains a draft row (`DEFINED`).
- **Tables Affected:** `add_on_requests`, `milestones`

TECH TEAM APPROVAL (Technical Justification)

---

[5] - Reviews Causal Chain & Technical Justification

- **Actor:** Client / TECH_TEAM
- **Input:** Add-On notification on Tech Dashboard.
- **Logic:** Per Section 0.7, only the Tech Team can approve the *technical justification* for an add-on. They evaluate the causal chain: Is the blocker real? Is the proposed technical solution the correct one?
- **Output:** Clicks `[Approve Technical]`. `POST /add-on-requests/{id}/approve/technical`.
- **State Change:** None.

---

[6] - Update State (PENDING_CEO_REVIEW)

- **Actor:** System (NestJS)
- **Input:** Tech approval from [5].
- **Logic:** The technical gate is passed. The request moves to the business gate.
    - `UPDATE add_on_requests SET state = 'PENDING_CEO_REVIEW', tech_approved_by = tech_user_id, tech_approved_at = now() WHERE id = ?`
- **Output:** CEO notified for budget approval.
- **State Change:** `add_on_requests` updated (`PENDING_CEO_REVIEW`).
- **Tables Affected:** `add_on_requests`

CEO APPROVAL (Budget & Timeline)

---

[7] - Reviews Budget & Timeline, Approves

- **Actor:** Client / CEO
- **Input:** Add-On notification on CEO Dashboard.
- **Logic:** Per Section 0.7, only the CEO can approve the *budget and timeline* for an add-on.
    1. **Subscription Guard:** Per Section 0.9, the Add-On Phase Protocol (F8) requires Client Pro.
    2. **Evaluate:** CEO reviews the 3,000,000 VND budget impact and 7-day timeline extension. They determine it is acceptable to unblock the project.
- **Output:** Clicks `[Approve Budget]`. `POST /add-on-requests/{id}/approve/budget`.
- **State Change:** None.

---

[8] - Final Approval: Unpause Clock & Append Milestone

- **Actor:** System (NestJS)
- **Input:** CEO approval from [7].

- **Logic:** The Add-On Protocol is fully approved. The system integrates the new scope into the project lifecycle.
    1. **Update Request:** `UPDATE add_on_requests SET state = 'APPROVED', ceo_approved_by = ceo_user_id, ceo_approved_at = now() WHERE id = ?`
    2. **Unpause Review Clock:** The original milestone's timer resumes where it left off.
    3. **Activate Draft Milestone:** The system moves the draft `ADD_ON` milestone to the funding stage, appending it sequentially to the project.
        - `UPDATE milestones SET state = 'AWAITING_PAYMENT' WHERE id = add_on_milestone_id`
- **Output:** Add-On approved. Project updated.
- **State Change:** `add_on_requests` updated (`APPROVED`). Original `milestones` timer resumed. Add-on `milestones` updated (`AWAITING_PAYMENT`).
- **Tables Affected:** `add_on_requests`, `milestones`

---

[9] - Add-On Approved, Blocker Resolved

- **Actor:** Expert
- **Input:** Dashboard notification.
- **Logic:** The Expert sees the scope evolution is formally approved. The sprint status `BLOCKER` is resolved (transitioning to `ON_TRACK` or `DEVIATION`), and the new milestone is queued for funding. The Expert continues work knowing the extension is contractually covered.
- **Output:** Work continues.
- **State Change:** None (UI reflects DB state).

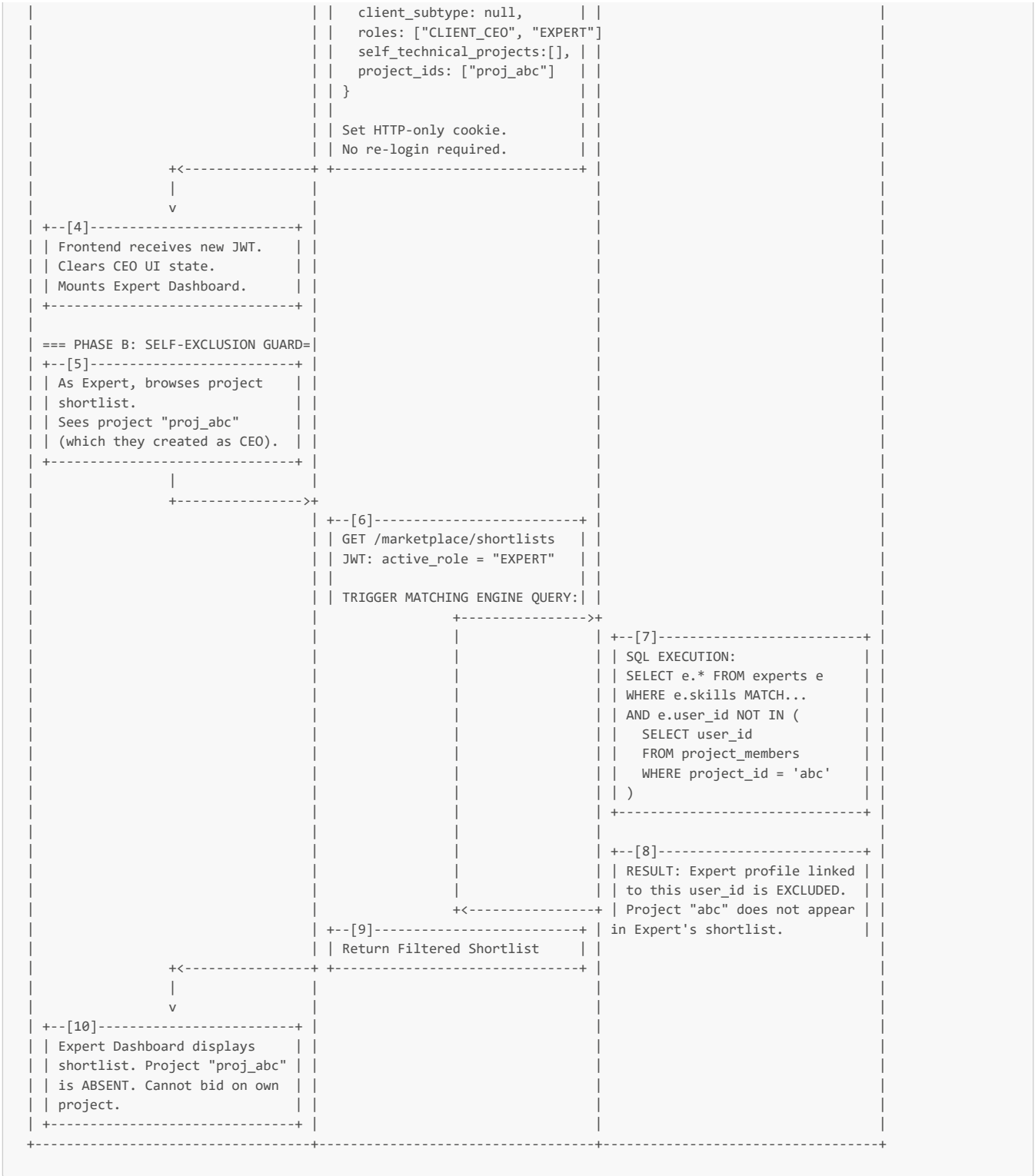
Summary: End-to-End State Trace for MF-14

Step	`sprint_updates`	`add_on_requests`	`milestones` **(Original)**	`milestones` **(Add-On)**
[2] Trigger	status=BLOCKER (SCOPE_EVOLUTION)	state=PENDING_EXPERT_PROPOSAL	Review clock PAUSED	-
[4] Proposed	-	state=PENDING_TECH_REVIEW	-	state=DEFINED (Draft, +3M VND)
[6] Tech Approved	-	state=PENDING_CEO_REVIEW	-	-
[8] CEO Approved	-	state=APPROVED	Review clock RESUMED	state=AWAITING_PAYMENT (Appended)

MF-14 is now complete. The scope evolution has been handled structurally rather than as a contract violation. The causal chain is documented (serving RQ2 research), technical validity is verified by the Tech Team, budget is authorized by the CEO, and the new work is formally appended as a distinct, escrow-backed milestone (ready for MF-7 funding) without disrupting the original milestone's paused clock.

MF-15: Dual-Role Account Switch

Detailed Swimlane Flow



Step-by-Step Narration

PHASE A - THE ROLE SWITCH

[1] - Clicks Role Switcher Toggle

- **Actor:** Dual-Role User (currently `active_role = "CLIENT"`, `client_subtype = "CEO"`)
- **Input:** The user is on the CEO Dashboard. They notice the persistent top-nav toggle (visible only when `roles.length > 1`) that says "Switch to Expert Profile". They click it.
- **Logic:** The frontend captures the click and prepares to switch the application context.
- **Output:** `POST /auth/switch-role` with payload `{ target_role: "EXPERT" }`.
- **State Change:** None.

[2] - Validate Role Switch Request

- **Actor:** System (NestJS - AuthController)
- **Input:** Switch request from [1], current JWT.
- **Logic:** The system verifies the user is allowed to switch to the requested role.
  - Read `roles` array from the current JWT or DB: `["CLIENT_CEO", "EXPERT"]`.
  - Check if `"EXPERT"` exists in the array.
  - If the user tried to switch to `"ADMIN"` or a role they don't hold, reject with `403 FORBIDDEN { code: "INVALID_ROLE_SWITCH" }`.
- **Output:** Validation passes .
- **State Change:** None.

---

[3] - Generate New JWT & Set Cookie

- **Actor:** System (NestJS - AuthService)
- **Input:** Validated target role.
- **Logic:** This is the core of the seamless role switch. The system does NOT invalidate the user's session or require re-authentication. It simply reissues the JWT with updated claims to change the frontend routing and backend guard behavior.
  - **Old JWT:**

```
{
  "sub": "user_id",
  "active_role": "CLIENT",
  "client_subtype": "CEO",
  "roles": ["CLIENT_CEO", "EXPERT"],
  "self_technical_projects": [],
  "project_ids": ["proj_abc"]
}
```

  - **New JWT:**

```
{
  "sub": "user_id",
  "active_role": "EXPERT",
  "client_subtype": null,
  "roles": ["CLIENT_CEO", "EXPERT"],
  "self_technical_projects": [],
  "project_ids": ["proj_abc"]
}
```

  - **Key Changes:** `active_role` changes to `"EXPERT"`, `client_subtype` becomes `null` (Experts don't have a subtype).
  - Set the new JWT as an HTTP-only cookie, overwriting the old one.
- **Output:** New JWT issued.
- **State Change:** None (Stateless token update).

---

[4] - Frontend Context Switch

- **Actor:** Dual-Role User
- **Input:** New JWT from [3].
- **Logic:** The frontend intercepts the new JWT.
  1. Updates the local auth state/context.
  2. Clears the CEO Dashboard UI components from memory (wallet balances, project initiation forms, shortlist views).
  3. Re-renders the application shell for the **Expert Dashboard** (profile status, service listings, active bids, sprint plans, earnings analytics).
- **Output:** User is now operating fully within the Expert context.
- **State Change:** None.

PHASE B - SELF-EXCLUSION GUARD

Context: This phase demonstrates the critical safety net required when a single user operates on both sides of the marketplace. The user created project "proj_abc" as a CEO in MF-4. Now, as an Expert, they might try to view or bid on it.

---

[5] - Browses Project Shortlist as Expert

- **Actor:** Dual-Role User (currently `active_role = "EXPERT"`)
- **Input:** User navigates to the "Find Projects" or shortlist view on their Expert Dashboard. Project "proj_abc" is currently PUBLISHED and seeking experts.
- **Logic:** Frontend requests the list of available projects/shortlists.
- **Output:** `GET /marketplace/shortlists` (or similar endpoint).
- **State Change:** None.

---

[6] - Trigger Matching Engine Query

- **Actor:** System (NestJS)
- **Input:** Request from [5], JWT (`active_role = "EXPERT"`, `user_id = 123`).
- **Logic:** The system queries the matching engine to find projects where this expert is eligible. It passes the `user_id` to the database query layer.
- **Output:** SQL query dispatched.
- **State Change:** None.

---

[7] - SQL Execution (Self-Exclusion Rule)

- **Actor:** Database / Rules Engine
- **Input:** Query parameters from [6].
- **Logic:** Per Section 0.7 ("Self-exclusion guard: Every matching engine query appends `WHERE expert.user_id NOT IN (SELECT user_id FROM project_members WHERE project_id = ?)`"), the database enforces the hard boundary.
  - The query evaluates Project "proj_abc".
  - It checks `project_members` for "proj_abc".
  - It finds `user_id = 123` (the CEO) is a member.
  - Therefore, `expert.user_id = 123` is excluded from the results for "proj_abc".
- **Output:** Filtered result set.
- **State Change:** None.

---

[8] - Result: User Excluded

- **Actor:** Database / Rules Engine
- **Input:** Execution result from [7].
- **Logic:** The expert profile belonging to the CEO of "proj_abc" is stripped from the results. The project itself might still appear in a general browse list, but the Expert will not see a "Bid" option, or if this is a specific shortlist query, the project is entirely absent.
- **Output:** Data returned to NestJS.
- **State Change:** None.

---

[9] - Return Filtered Shortlist

- **Actor:** System (NestJS)
- **Input:** Filtered data from [8].
- **Logic:** NestJS formats the response. It does not explicitly tell the user "You are excluded from your own project"; it simply returns the standard shortlist as if the expert wasn't a match for that specific project.
- **Output:** Response sent to frontend.
- **State Change:** None.

---

[10] - Expert Dashboard Displays Shortlist

- **Actor:** Dual-Role User
- **Input:** Shortlist data from [9].
- **Logic:** The user views their Expert shortlist. Project "proj_abc" (which they created as CEO) is either absent or marked as "Ineligible". They cannot bid on their own project, preventing a massive conflict of interest and escrow manipulation vulnerability.
- **Output:** Self-exclusion enforced successfully.
- **State Change:** None.

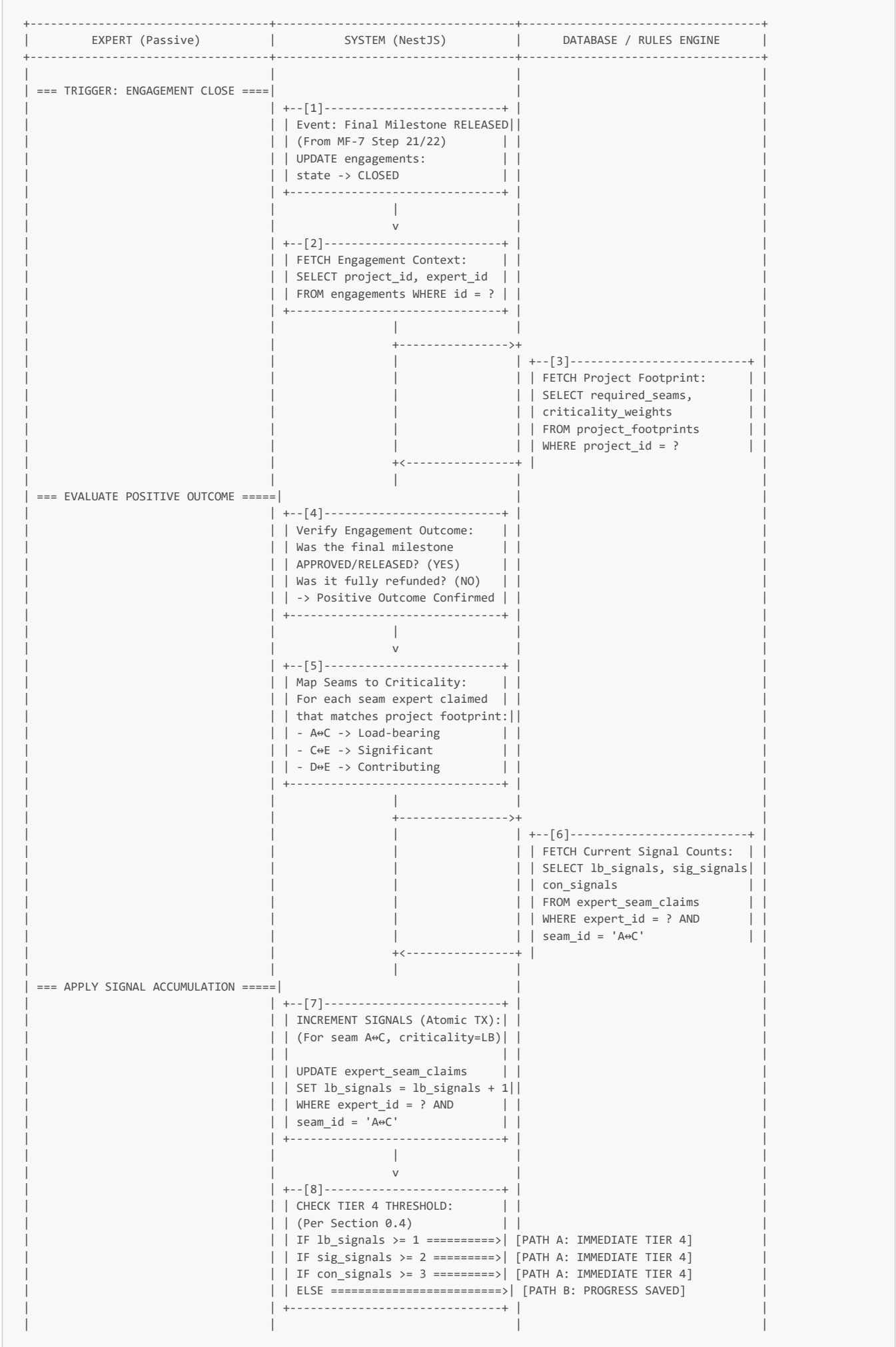
Summary: End-to-End State Trace for MF-15

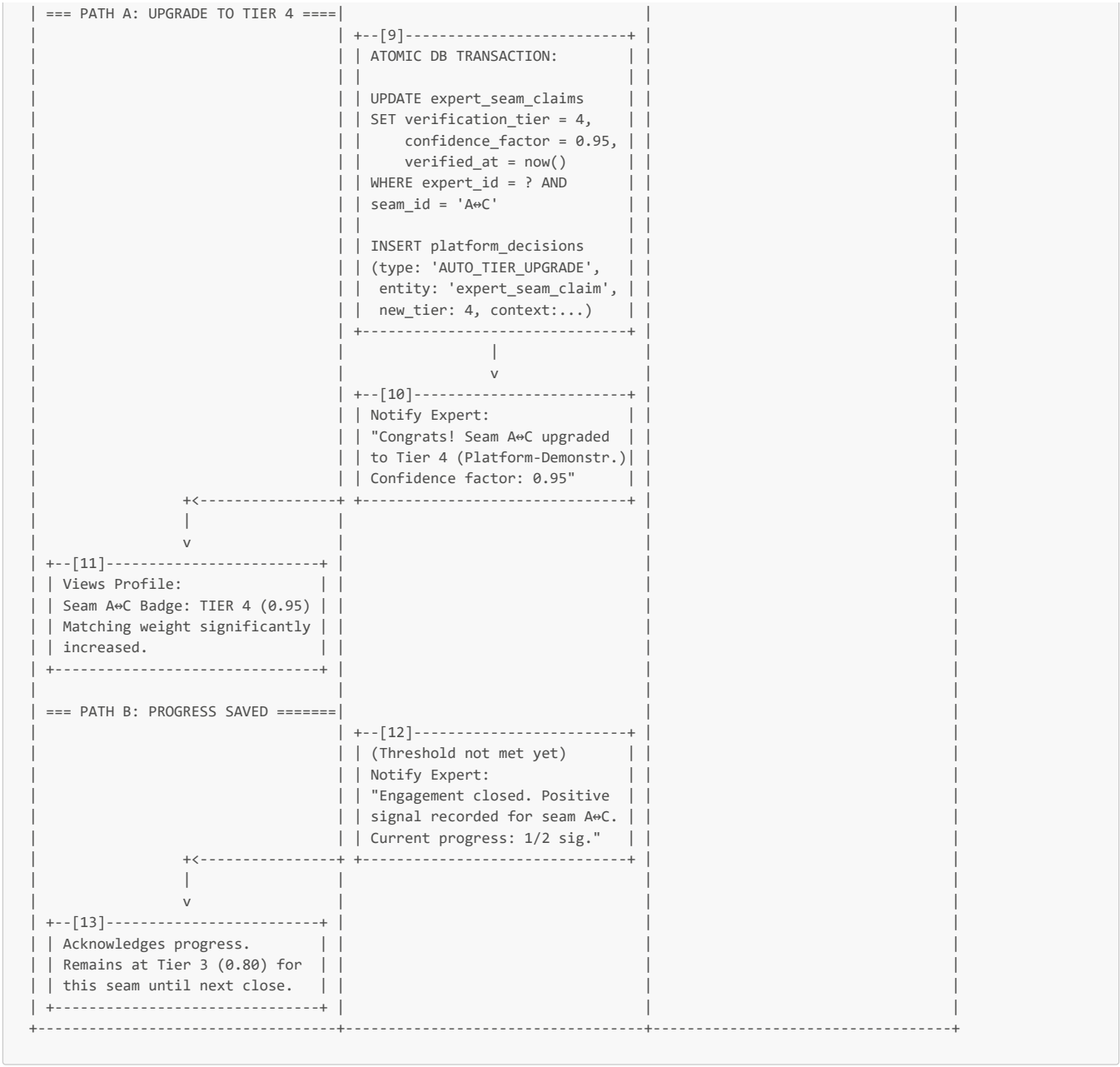
Step	JWT State	UI Context	Matching Engine Constraint
[1] Init	active_role="CLIENT", subtype="CEO"	CEO Dashboard	-
[3] Switch	active_role="EXPERT", subtype=null	-	-
[4] Render	active_role="EXPERT"	Expert Dashboard	-
[7] Guard Check	-	-	`user_id NOT IN project_members` enforced
[10] Result	active_role="EXPERT"	Expert Shortlist View	Cannot bid on own project ("proj_abc")

MF-15 is now complete. The dual-role user has seamlessly switched their application context from CEO to Expert without re-authentication, enabled by a JWT re-issuance strategy. The system's integrity is maintained by the database-level self-exclusion guard, which ensures that switching roles does not allow a user to act as both the buyer and the seller in the same transaction, preserving the trust and escrow models.

MF-16: Expert Verification Auto-Upgrade (Tier 4)

Detailed Swimlane Flow


EXPERT (Passive)
SYSTEM (NestJS)
DATABASE / RULES ENGINE
=== TRIGGER: ENGAGEMENT CLOSE ====
[1]
Event: Final Milestone RELEASED
(From MF-7 Step 21/22)
UPDATE engagements:
state -> CLOSED
[2]
FETCH Engagement Context:
SELECT project_id, expert_id
FROM engagements WHERE id = ?
[3]
FETCH Project Footprint:
SELECT required_seams,
criticality_weights
FROM project_footprints
WHERE project_id = ?
=== EVALUATE POSITIVE OUTCOME =====
[4]
Verify Engagement Outcome:
Was the final milestone
APPROVED/RELEASED? (YES)
Was it fully refunded? (NO)
-> Positive Outcome Confirmed
[5]
Map Seams to Criticality:
For each seam expert claimed
that matches project footprint:
- A↔C -> Load-bearing
- C↔E -> Significant
- D↔E -> Contributing
[6]
FETCH Current Signal Counts:
SELECT lb_signals, sig_signals
con_signals
FROM expert_seam_claims
WHERE expert_id = ? AND
seam_id = 'A↔C'
=== APPLY SIGNAL ACCUMULATION =====
[7]
INCREMENT SIGNALS (Atomic TX):
(For seam A↔C, criticality=LB)
UPDATE expert_seam_claims
SET lb_signals = lb_signals + 1
WHERE expert_id = ? AND
seam_id = 'A↔C'
[8]
CHECK TIER 4 THRESHOLD:
(Per Section 0.4)
IF lb_signals >= 1 ==========>
IF sig_signals >= 2 =========>
IF con_signals >= 3 =========>
ELSE ========================>
[PATH A: IMMEDIATE TIER 4]
[PATH A: IMMEDIATE TIER 4]
[PATH A: IMMEDIATE TIER 4]
[PATH B: PROGRESS SAVED]



Step-by-Step Narration

TRIGGER: ENGAGEMENT CLOSE

---

[1] - Event: Final Milestone RELEASED, Engagement CLOSED

- **Actor:** System (NestJS - Event Listener)
- **Input:** The completion of MF-7, where the final milestone state transitions to `RELEASED` and funds are disbursed.
- **Logic:** The system detects the engagement lifecycle completion and transitions the engagement state.
  - `UPDATE engagements SET state = 'CLOSED' WHERE id = ?`
  - This `CLOSED` event acts as the trigger for the post-engagement evaluation pipeline.
- **Output:** Engagement closed. Evaluation triggered.
- **State Change:** `engagements` updated (`CLOSED`).
- **Tables Affected:** `engagements`

---

[2] - Fetch Engagement Context

- **Actor:** System (NestJS)
- **Input:** `engagement_id` from the closed event.
- **Logic:** The system retrieves the context needed to evaluate the expert's performance.
  - `SELECT project_id, expert_user_id FROM engagements WHERE id = ?`
- **Output:** `project_id` and `expert_user_id` identified.
- **State Change:** None.

---

[3] - Fetch Project Footprint

- **Actor:** Database / Rules Engine
- **Input:** `project_id` from [2].
- **Logic:** To evaluate the expert, the system must know *what* the expert was supposed to deliver. This was defined in MF-4 and stored in `project_footprints`.
  - `SELECT required_seams, criticality_weights FROM project_footprints WHERE project_id = ?`
  - *Example Result:* Seam A↔C is `load-bearing`, Seam C↔E is `significant`, Seam D↔E is `contributing`.
- **Output:** Footprint criticality data returned.
- **State Change:** None.

EVALUATE POSITIVE OUTCOME

---

[4] - Verify Engagement Outcome

- **Actor:** System (NestJS)
- **Input:** Milestone/Engagement history.
- **Logic:** A Tier 4 upgrade requires a "positive outcome signal." The system verifies this by checking the financial/dispute history of the engagement.
  - Did the final milestone reach `APPROVED` or `RELEASED`? (YES)
  - Was the final resolution a full refund to the client via dispute? (NO)
  - *Conclusion:* The expert delivered satisfactory work. Positive outcome confirmed.
- **Output:** Positive outcome boolean = TRUE.
- **State Change:** None.

---

[5] - Map Seams to Criticality

- **Actor:** System (NestJS)
- **Input:** Expert's claimed seams (from `expert_seam_claims`) intersected with the Project Footprint (from [3]).
- **Logic:** The system identifies which of the expert's claimed seams were actually exercised in this project, and what criticality level they operated under.
  - Expert claimed A↔C. Project footprint says A↔C is **load-bearing**.
  - Expert claimed C↔E. Project footprint says C↔E is **significant**.
  - Expert claimed D↔E. Project footprint says D↔E is **contributing**.
- **Output:** Mapping complete.
- **State Change:** None.

---

[6] - Fetch Current Signal Counts

- **Actor:** Database / Rules Engine
- **Input:** `expert_user_id`, `seam_id` (e.g., 'A↔C').
- **Logic:** The system retrieves the expert's current accumulation progress for this specific seam from previous engagements.
  - `SELECT lb_signals, sig_signals, con_signals FROM expert_seam_claims WHERE expert_id = ? AND seam_id = 'A↔C'`
- **Output:** Current counters returned (e.g., `lb_signals = 0`).
- **State Change:** None.

APPLY SIGNAL ACCUMULATION

---

[7] - Increment Signals (Atomic TX)

- **Actor:** System (NestJS)
- **Input:** Positive outcome confirmation [4], Criticality mapping [5].
- **Logic:** The system atomically increments the specific signal counter based on the role the seam played in *this* project. Since A↔C was load-bearing here, `lb_signals` is incremented.
  - `UPDATE expert_seam_claims SET lb_signals = lb_signals + 1 WHERE expert_id = ? AND seam_id = 'A↔C'`
- **Output:** Counter updated.
- **State Change:** `expert_seam_claims` updated (`lb_signals += 1`).
- **Tables Affected:** `expert_seam_claims`

---

[8] - Check Tier 4 Threshold

- **Actor:** System (NestJS)
- **Input:** Updated signal counts from [7].
- **Logic:** The system applies the **Signal Accumulation Rule** defined in Section 0.4.
  - Rule 1: `lb_signals >= 1`? → **YES** (Path A: Immediate Tier 4)
  - Rule 2: `sig_signals >= 2`? → (Check if applicable)
  - Rule 3: `con_signals >= 3`? → (Check if applicable)
  - *Because the expert just earned 1 load-bearing signal for A↔C, they immediately qualify for Tier 4 on that seam.*

- **Output:** Routing decision → **Path A**.
- **State Change:** None.

PATH A: UPGRADE TO TIER 4

---

[9] - Execute Upgrade (Atomic TX)

- **Actor:** System (NestJS)
- **Input:** Threshold met for seam A↔C.
- **Logic:** The system upgrades the seam verification and logs the automated decision for the Admin Integrity Monitor (MF-17).

```
BEGIN TRANSACTION;

-- 1. Upgrade Verification
UPDATE expert_seam_claims
  SET verification_tier = 4,
      confidence_factor = 0.95,
      verified_at = now()
  WHERE expert_id = ? AND seam_id = 'A↔C';

-- 2. Audit Log
INSERT INTO platform_decisions
  (decision_type, entity_type, entity_id, new_value, context, created_at)
  VALUES ('AUTO_TIER_UPGRADE', 'expert_seam_claim', claim_id, 'TIER_4',
          'Engagement closed. 1x load-bearing signal accumulated.', now());

COMMIT;
```

- **Output:** Upgrade executed. Audit logged.
- **State Change:** `expert_seam_claims` updated (`verification_tier = 4`, `confidence_factor = 0.95`). `platform_decisions` gains one row.
- **Tables Affected:** `expert_seam_claims`, `platform_decisions`

---

[10] - Notify Expert of Tier 4 Upgrade

- **Actor:** System (NestJS)
- **Input:** Successful upgrade from [9].
- **Logic:** Send an in-app and email notification celebrating the achievement.
  - "*Congratulations! Your capability in Seam A↔C (Ground truth-driven iteration) has been automatically upgraded to Tier 4 (Platform-Demonstrated). Your confidence factor is now 0.95. This significantly boosts your visibility in the matching engine.*"
- **Output:** Notification dispatched.
- **State Change:** None.

---

[11] - Expert Views Updated Profile

- **Actor:** Expert
- **Input:** Notification from [10].
- **Logic:** The Expert navigates to their profile. The Seam A↔C badge now displays "Tier 4 (0.95)". In future matching (MF-5), the composite score for this seam will be calculated using 0.95 instead of 0.80, making them a "Strong" match for high-tier projects requiring this seam.
- **Output:** None.
- **State Change:** None.

PATH B: PROGRESS SAVED (If Threshold Not Met)

Note: Steps [12]-[13] represent the alternative path if, for example, the seam was only "contributing" and the expert now has 2/3 con_signals, but not yet 3.

---

[12] - Notify Expert of Progress

- **Actor:** System (NestJS)
- **Input:** Threshold not met in [8].
- **Logic:** The system still acknowledges the positive outcome to encourage the expert.
  - "*Engagement closed successfully! A positive outcome signal has been recorded for Seam D↔E (Contributing role). Current progress toward Tier 4: 2/3 contributing signals required.*"
- **Output:** Notification dispatched.
- **State Change:** None.

---

[13] - Expert Acknowledges Progress

- **Actor:** Expert

- **Input:** Notification from [12].
- **Logic:** The expert sees their progress tracker update, but their verification tier for that seam remains at its current level (e.g., Tier 3) until the threshold is crossed.
- **Output:** None.
- **State Change:** None.

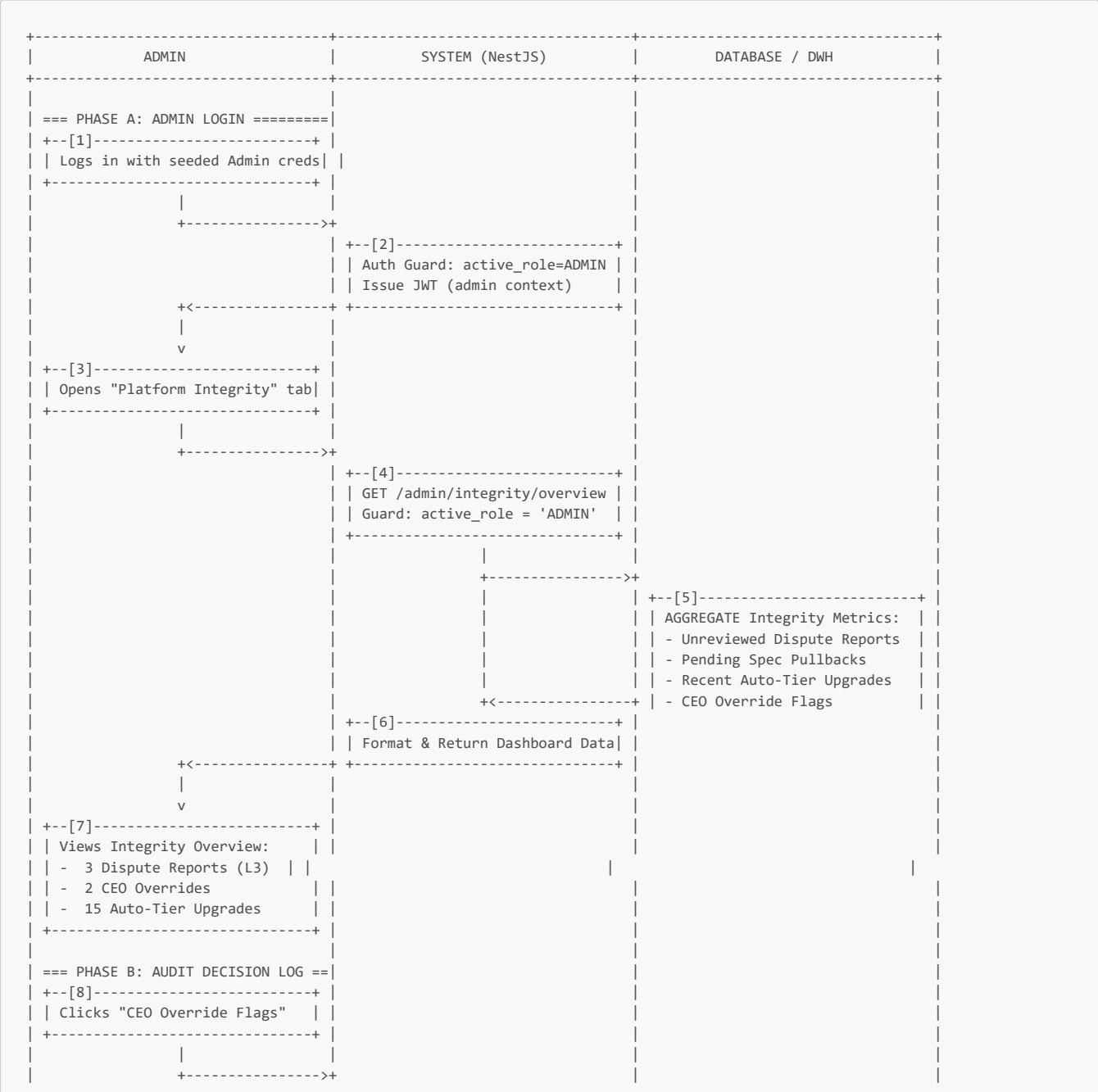
Summary: End-to-End State Trace for MF-16

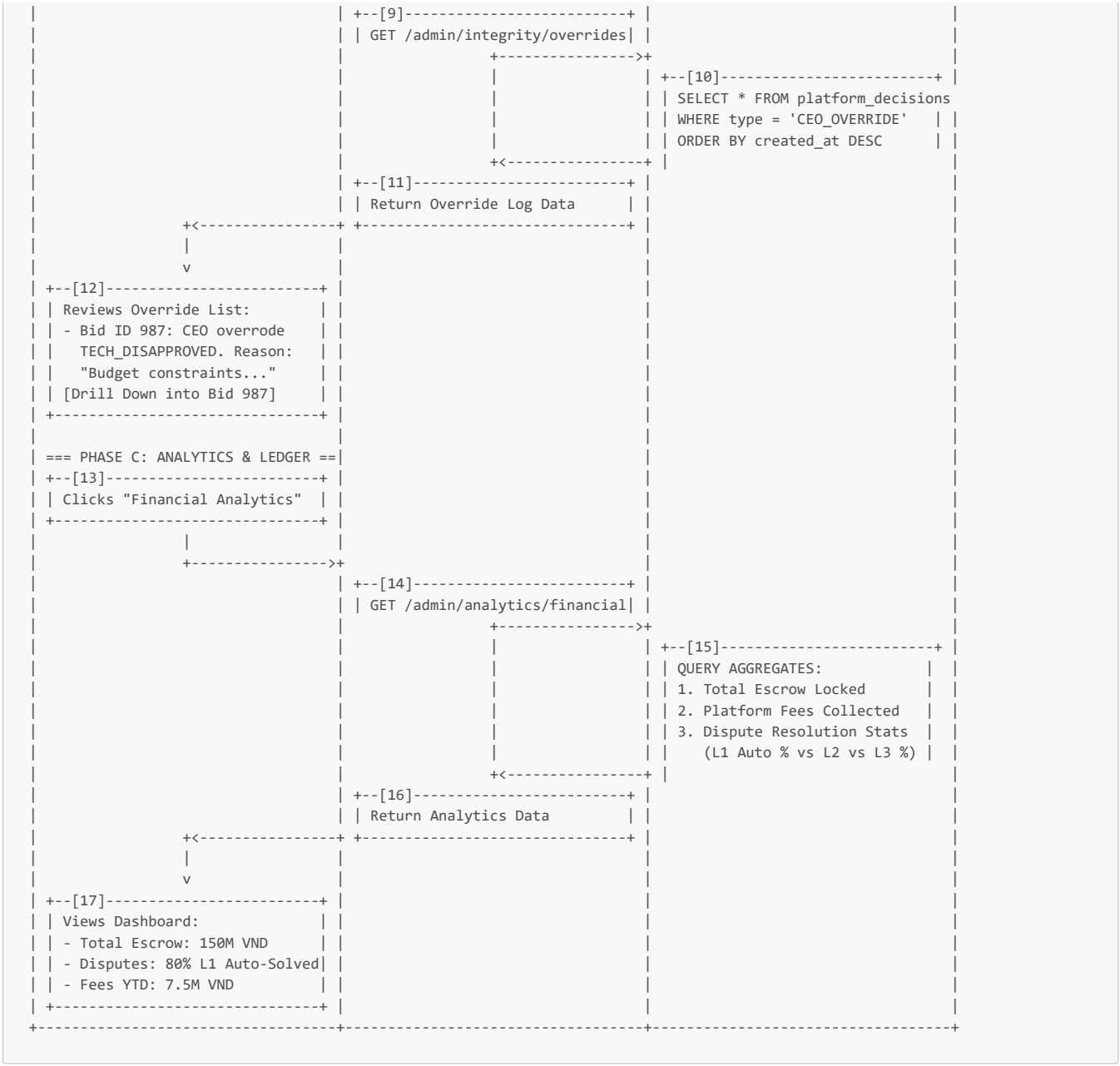
Step	`engagements`	`expert_seam_claims` (A↔C)	`platform_decisions`
[1] Close	state=CLOSED	-	-
[7] Increment	-	lb_signals += 1	-
[9] Upgrade	-	verification_tier=4, confidence_factor=0.95	AUTO_TIER_UPGRADE logged

MF-16 is now complete. The platform has autonomously evaluated the expert's real-world performance upon engagement closure, applied the signal accumulation rule, and upgraded the expert's seam verification to Tier 4 with a 0.95 confidence factor. This zero-admin flow ensures that proven capability is immediately reflected in the matching engine, directly supporting RQ1 (Matching Accuracy) by rewarding verified, practical success over theoretical knowledge.

MF-17: Platform Integrity Monitor & Analytics

Detailed Swimlane Flow





Step-by-Step Narration

PHASE A - ADMIN LOGIN & OVERVIEW

---

[1] - Logs in with Seeded Admin Credentials

- **Actor:** Admin
- **Input:** Admin navigates to `/admin/login` and enters the DB-seeded super-admin credentials.
- **Logic:** Standard authentication flow.
- **Output:** `POST /auth/login` with credentials.
- **State Change:** None.

---

[2] - Auth Guard & Issue JWT

- **Actor:** System (NestJS)
- **Input:** Login request from [1].
- **Logic:** NestJS validates credentials. Upon success, it issues a JWT specifically scoped to the Admin context:

```
{
  "sub": "admin_user_id",
  "active_role": "ADMIN",
  "client_subtype": null,
```

```
  "roles": ["ADMIN"]
}
```

- **Output:** JWT set as HTTP-only cookie. Redirect to Admin Dashboard.
- **State Change:** None.

---

[3] - Opens "Platform Integrity" Tab

- **Actor:** Admin
- **Input:** Admin clicks the "Platform Integrity" or "Monitor" tab in the Admin Dashboard.
- **Logic:** Frontend requests the high-level overview of platform health and automated decisions requiring passive review.
- **Output:** `GET /admin/integrity/overview` request sent.
- **State Change:** None.

---

[4] - Route Guard: Admin Only

- **Actor:** System (NestJS)
- **Input:** Overview request from [3], JWT.
- **Logic:** The NestJS guard strictly enforces `active_role === 'ADMIN'`. No other role can access this endpoint. Per Section 0.7, Admin has read-only monitoring access.
- **Output:** Guard passes .
- **State Change:** None.

---

[5] - Aggregate Integrity Metrics

- **Actor:** Database / Data Warehouse
- **Input:** Query from NestJS.
- **Logic:** The DB executes aggregation queries across key audit and state tables to provide a real-time snapshot of platform integrity:
  1. **Unreviewed Dispute Reports:** `SELECT COUNT(*) FROM platform_decisions WHERE type = 'DISPUTE_REPORT' AND reviewed = false` (Generated from MF-8 Layer 3 "Report this Resolution" clicks).
  2. **Pending Spec Pullbacks:** `SELECT COUNT(*) FROM specs WHERE state = 'SUSPENDED'` (Admin emergency action).
  3. **Recent Auto-Tier Upgrades:** `SELECT COUNT(*) FROM platform_decisions WHERE type = 'AUTO_TIER_UPGRADE' AND created_at > now() - interval '7 days'` (Generated from MF-16).
  4. **CEO Override Flags:** `SELECT COUNT(*) FROM platform_decisions WHERE type = 'CEO_OVERRIDE'` (Generated from MF-6 Surface D).
- **Output:** Aggregated metrics returned.
- **State Change:** None.

---

[6] & [7] - Format & Display Dashboard

- **Actor:** System (NestJS) -> Admin
- **Input:** Metrics from [5].
- **Logic:** NestJS formats the data into a frontend-friendly JSON structure. The Admin Dashboard renders the high-level health cards:
  - **3 Dispute Reports**: Users who felt Layer 3 50/50 splits were unfair.
  - **2 CEO Overrides**: Business decisions made against technical advice.
  - **15 Auto-Tier Upgrades**: Experts who demonstrated real-world capability this week.
- **Output:** Dashboard rendered.
- **State Change:** None.

PHASE B - AUDIT DECISION LOG

---

[8] - Clicks "CEO Override Flags"

- **Actor:** Admin
- **Input:** Admin wants to investigate the 2 CEO Overrides to ensure no platform abuse is occurring.
- **Logic:** Frontend requests the detailed log.
- **Output:** `GET /admin/integrity/overrides` request sent.
- **State Change:** None.

---

[9] & [10] - Fetch Decision Log

- **Actor:** System (NestJS) + Database
- **Input:** Request from [8].
- **Logic:** NestJS queries the `platform_decisions` table, which acts as the immutable, append-only audit trail for all automated or exceptional platform actions.

```

```

```
SELECT * FROM platform_decisions
WHERE decision_type = 'CEO_OVERRIDE'
ORDER BY created_at DESC;
```

- **Output:** Log rows returned.
- **State Change:** None.

---

[11] & [12] - Review Override Details

- **Actor:** System (NestJS) -> Admin
- **Input:** Log data from [10].
- **Logic:** The UI renders a table of overrides. Admin reviews the context:
  - *Bid ID 987:* CEO overrode TECH_DISAPPROVED. Reason: "*Willing to accept technical risk due to budget constraints.*"
  - Admin can click to drill down into the Bid details (reading the bid versions, tech team reasons, etc.) to ensure the override reason is legitimate and not collusion. Admin has **read-only** access here; they cannot reverse the override or the connection.
- **Output:** Log reviewed.
- **State Change:** None.

PHASE C - ANALYTICS & LEDGER

---

[13] - Clicks "Financial Analytics"

- **Actor:** Admin
- **Input:** Admin switches to the financial health view.
- **Logic:** Frontend requests financial aggregates.
- **Output:** `GET /admin/analytics/financial` request sent.
- **State Change:** None.

---

[14] & [15] - Query Financial Aggregates

- **Actor:** System (NestJS) + Database
- **Input:** Request from [13].
- **Logic:** The system queries the internal ledger (`wallet_transactions`, `wallets`, `disputes`) to generate business intelligence.
  1. **Total Escrow Locked:** `SELECT SUM(locked_balance) FROM wallets` - Funds committed to milestones.
  2. **Platform Fees Collected:** `SELECT SUM(amount) FROM wallet_transactions WHERE type = 'PLATFORM_FEE'` - Revenue tracking.
  3. **Dispute Resolution Stats:** `SELECT state, COUNT(*) FROM disputes GROUP BY state` - Evaluates the effectiveness of the AI Layer 1 vs. Mutual Layer 2 vs. Forced Layer 3.
- **Output:** Aggregated financial data returned.
- **State Change:** None.

---

[16] & [17] - Display Analytics

- **Actor:** System (NestJS) -> Admin
- **Input:** Data from [15].
- **Logic:** The Admin Dashboard renders charts and KPIs:
  - **Total Escrow:** 150,000,000 VND (Indicates platform transaction volume).
  - **Dispute Resolution:** 80% solved by Layer 1 (AI Auto-Resolve), 15% Layer 2 (Mutual), 5% Layer 3 (Forced Split). *This validates RQ3 research-automated trust resolution works.*
  - **Fees YTD:** 7,500,000 VND.
- **Output:** Analytics viewed.
- **State Change:** None.

Summary: End-to-End State Trace for MF-17

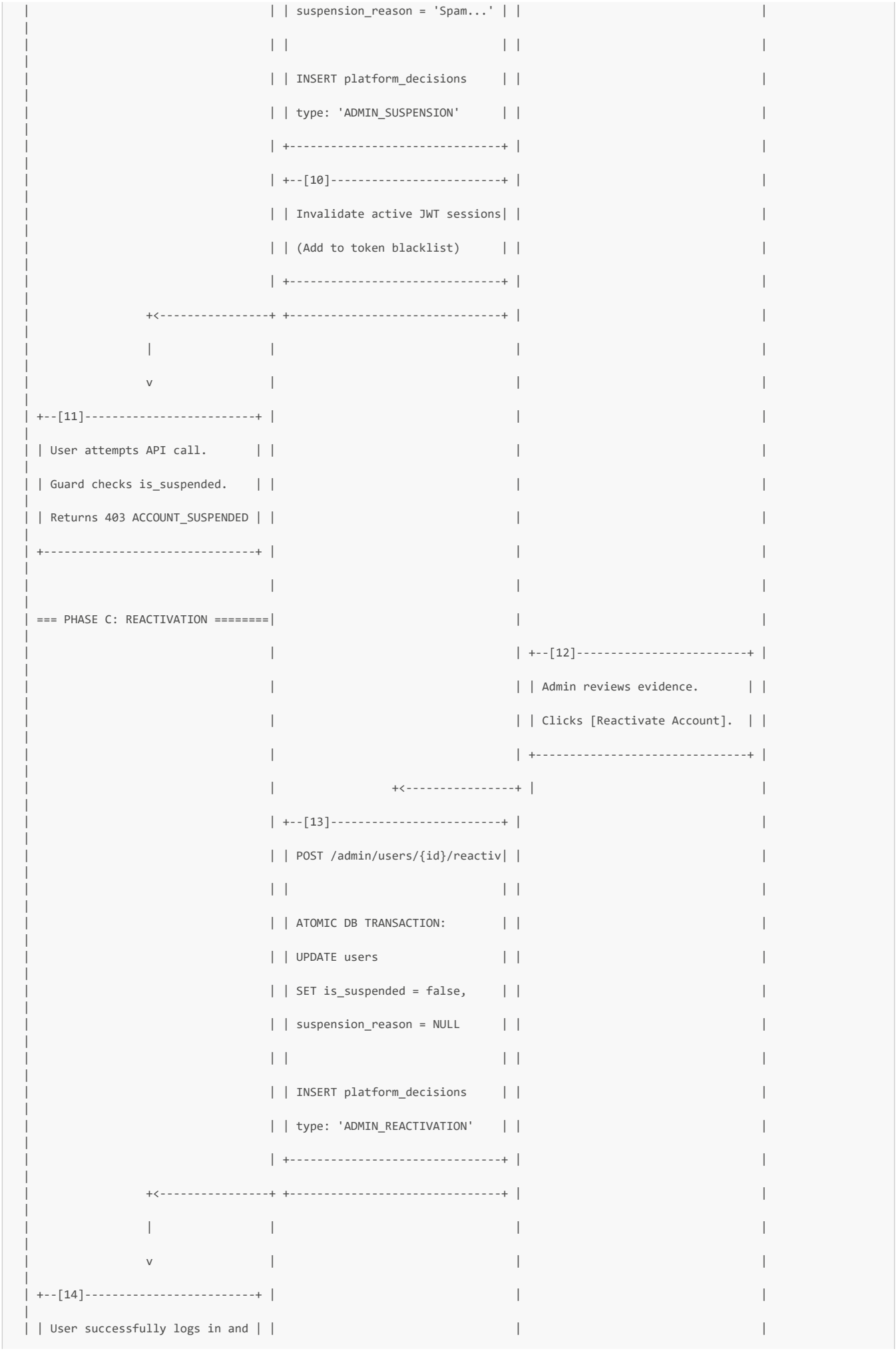
Step	`platform_decisions`	`wallets` / `wallet_transactions`	`disputes`	Admin Action
[5] Overview	Count by type (overrides, upgrades, reports)	-	-	Read
[10] Overrides	Read CEO_OVERRIDE rows	-	-	Read
[15] Analytics	-	SUM(locked_balance), SUM(platform_fees)	Group by resolution state	Read

MF-17 is now complete. The Admin has passively monitored the platform's automated integrity systems. By leveraging the `platform_decisions` audit log and aggregated ledger data, the Admin ensures transparency and tracks the health of the AI-driven matching, verification, and dispute pipelines without manual choke points or routine approvals.

MF-18: Account Management & Fraud Detection

Detailed Swimlane Flow


SET is_suspended = true,
suspension_reason =
'DUPLICATE_BANK_ACCOUNT'
WHERE id = 'expert_B_id'
2. INSERT platform_decisions
type: 'FRAUD_AUTO_BLOCK',
entity_id: 'expert_B_id',
context: 'xid matched A'
[6]
Admin Dashboard Alert:
"FRAUD AUTO-BLOCK: Expert B
blocked. Duplicate bank link."
[7]
Reject Bank Link save.
Notify Expert B:
"Account suspended. Contact
support."
=== PHASE B: MANUAL SUSPENSION ===
[8]
Admin reviews reported user.
Clicks [Suspend Account].
Inputs Reason: "Spam bids"
[9]
POST /admin/users/{id}/suspend
Guard: active_role = 'ADMIN'
ATOMIC DB TRANSACTION:
UPDATE users
SET is_suspended = true,


suspension_reason = 'Spam...'
INSERT platform_decisions
type: 'ADMIN_SUSPENSION'
[10]
Invalidate active JWT sessions
(Add to token blacklist)
[11]
User attempts API call.
Guard checks is_suspended.
Returns 403 ACCOUNT_SUSPENDED
=== PHASE C: REACTIVATION ========
[12]
Admin reviews evidence.
Clicks [Reactivate Account].
[13]
POST /admin/users/{id}/reactiv
ATOMIC DB TRANSACTION:
UPDATE users
SET is_suspended = false,
suspension_reason = NULL
INSERT platform_decisions
type: 'ADMIN_REACTIVATION'
[14]
User successfully logs in and

```
|
| | accesses dashboard.          | |                                |                                |
|
| +------------------------------+ |                                |                                |
|
+----------------------------------+--------------------------------+--------------------------------+---------------
-------------------+
```

Step-by-Step Narration

PHASE A - AUTOMATED FRAUD DETECTION (DEDUPLICATION CHECK)

---

[1] - Expert Attempts to Link Bank Account

- **Actor:** Expert (User B)
- **Input:** Expert B navigates to their Earnings/Wallet section and clicks "Link Bank Account". They go through the Bank Hub Hosted Link flow (MF-2 Phase F), entering their bank details and OTP.
- **Logic:** The flow is identical to a legitimate user until the webhook returns.
- **Output:** Bank Hub processes OTP.
- **State Change:** None.

---

[2] - SePay Fires BANK_ACCOUNT_LINKED Webhook

- **Actor:** SePay / Bank Hub
- **Input:** OTP verification success.
- **Logic:** SePay fires the standard webhook indicating a successful link.
  - *Payload:* `{ event: "BANK_ACCOUNT_LINKED", user_reference: "expert_B_id", bank_account_xid: "bax_suspicious_123" }`
  - *Context:* Unbeknownst to the system, Expert A has *already* linked this exact same bank account (`bax_suspicious_123`) in a previous session.
- **Output:** Webhook sent to NestJS.
- **State Change:** External.

---

[3] - Receive Webhook Payload

- **Actor:** System (NestJS - WebhookController)
- **Input:** Webhook payload from [2].
- **Logic:** NestJS parses the payload to extract the `user_id` and the `bank_account_xid`.
- **Output:** Data prepared for validation.
- **State Change:** None.

---

[4] - Deduplication Guard

- **Actor:** System (NestJS)
- **Input:** `bank_account_xid` from [3].
- **Logic:** Before crediting the xid to Expert B, the system executes a critical fraud prevention check. Per Section 0.8/0.7, a single bank account cannot be used to withdraw funds by multiple experts (prevents a single person from operating multiple shell accounts to bypass self-exclusion, manipulate reviews, or drain funds).
  - `SELECT id FROM users WHERE sepay_bank_account_xid = 'bax_suspicious_123' AND id != 'expert_B_id'`
- **Output:** Query returns 1 row (Expert A holds this xid). **Fraud condition triggered **.
- **State Change:** None.

---

[5] - Fraud Detected: Auto-Block Account

- **Actor:** System (NestJS)
- **Input:** Deduplication hit from [4].
- **Logic:** The system immediately executes damage control. Expert B's account is compromised/fraudulent.

```
BEGIN TRANSACTION;

-- 1. Suspend the fraudulent account
UPDATE users
  SET is_suspended = true,
      suspension_reason = 'DUPLICATE_BANK_ACCOUNT'
  WHERE id = 'expert_B_id';

-- 2. Log the automated decision for the Admin
INSERT INTO platform_decisions
  (decision_type, entity_type, entity_id, context, created_at)
  VALUES ('FRAUD_AUTO_BLOCK', 'USER', 'expert_B_id',
```

```
          'Bank account xid bax_suspicious_123 already linked to expert_A_id', now());

COMMIT;
```

  - *Note:* The system explicitly **does not** save `bax_suspicious_123` to Expert B's record. It leaves `sepay_bank_account_xid` as NULL for Expert B to ensure no withdrawal can ever be routed there, even if the suspension is lifted later without fixing the bank link.
- **Output:** Transaction committed. Account suspended.
- **State Change:** `users` updated (`is_suspended = true`). `platform_decisions` gains one row.
- **Tables Affected:** `users`, `platform_decisions`

---

[6] - Admin Dashboard Alert

- **Actor:** Admin
- **Input:** Real-time update or dashboard refresh.
- **Logic:** The Admin's Integrity Monitor (MF-17) flags the `FRAUD_AUTO_BLOCK` event. The Admin is now aware that Expert B tried to link Expert A's bank account.
- **Output:** Alert displayed.
- **State Change:** None.

---

[7] - Reject Link & Notify Expert B

- **Actor:** System (NestJS)
- **Input:** Fraud detection result.
- **Logic:** The system notifies Expert B that their action was blocked and their account is suspended.
  - "*Account suspended. Activity flagged for security review. Please contact support.*"
- **Output:** Notification sent. Link rejected.
- **State Change:** None.

PHASE B - MANUAL SUSPENSION (ADMIN ACTION)

---

[8] - Admin Reviews & Clicks Suspend

- **Actor:** Admin
- **Input:** Admin investigates a user (e.g., reports of spamming bids or abusive messaging) and decides to suspend them.
- **Logic:** Admin clicks [Suspend Account] on the User Management view and inputs a reason: "*Spam bids / Harassment*".
- **Output:** `POST /admin/users/{id}/suspend` with `{ reason: "Spam bids / Harassment" }`.
- **State Change:** None.

---

[9] - Execute Suspension

- **Actor:** System (NestJS)
- **Input:** Suspension request from [8].
- **Logic:** The system applies the suspension.

```
BEGIN TRANSACTION;

-- 1. Update User Status
UPDATE users
  SET is_suspended = true,
      suspension_reason = 'Spam bids / Harassment'
  WHERE id = ?;

-- 2. Audit Log
INSERT INTO platform_decisions
  (decision_type, entity_type, entity_id, context, created_at)
  VALUES ('ADMIN_SUSPENSION', 'USER', user_id, 'Spam bids / Harassment', now());

COMMIT;
```

- **Output:** Account suspended.
- **State Change:** `users` updated (`is_suspended = true`). `platform_decisions` gains one row.
- **Tables Affected:** `users`, `platform_decisions`

---

[10] - Invalidate Active Sessions

- **Actor:** System (NestJS)
- **Input:** Suspension execution from [9].

- **Logic:** Suspension must be immediate. If the user currently has a valid JWT, they could continue performing actions. The system adds the `user_id` to a Redis/DB token blacklist. The NestJS Auth Guard checks this blacklist on every request.
- **Output:** Active sessions invalidated.
- **State Change:** Token blacklist updated.

---

[11] - User Attempts API Call (Blocked)

- **Actor:** User (Suspended Expert/Client)
- **Input:** The suspended user tries to make an API call (e.g., submit a bid).
- **Logic:** The NestJS Auth Guard intercepts the request. It decodes the JWT, checks the `user_id` against the `users.is_suspended` flag (or the token blacklist), and rejects the request.
- **Output:** `403 FORBIDDEN { code: "ACCOUNT_SUSPENDED", reason: "..." }`.
- **State Change:** None.

PHASE C - REACTIVATION

---

[12] - Admin Clicks Reactivate

- **Actor:** Admin
- **Input:** Admin reviews the suspension context (e.g., the duplicate bank account was a legitimate joint business account, or the spam issue is resolved) and clicks [Reactivate Account].
- **Logic:** Admin submits the reactivation.
- **Output:** `POST /admin/users/{id}/reactivate`.
- **State Change:** None.

---

[13] - Execute Reactivation

- **Actor:** System (NestJS)
- **Input:** Reactivation request from [12].
- **Logic:** The system restores the user's access. *Note: If the suspension was due to a duplicate bank account (Phase A), the Admin must ensure the user re-links a unique bank account before they can withdraw, as the previous xid was not saved.*

```
BEGIN TRANSACTION;

-- 1. Restore User Status
UPDATE users
  SET is_suspended = false,
      suspension_reason = NULL
  WHERE id = ?;

-- 2. Audit Log
INSERT INTO platform_decisions
  (decision_type, entity_type, entity_id, context, created_at)
  VALUES ('ADMIN_REACTIVATION', 'USER', user_id, 'Suspension resolved', now());

COMMIT;
```

- **Output:** Account reactivated.
- **State Change:** `users` updated (`is_suspended = false`). `platform_decisions` gains one row.
- **Tables Affected:** `users`, `platform_decisions`

---

[14] - User Access Restored

- **Actor:** User
- **Input:** Successful reactivation notification.
- **Logic:** The user can now log in again (or their existing JWT will pass the guard check). Their wallet balances and active engagements are intact, as escrow is never touched by account suspension-only access is revoked.
- **Output:** Access restored.
- **State Change:** None.

Summary: End-to-End State Trace for MF-18

Step	`users`	`platform_decisions`	**Expert Bank Link** (`sepay_bank_account_xid`)
[5] Auto-Block	is_suspended=true, reason=DUPLICATE_BANK_ACCOUNT	type=FRAUD_AUTO_BLOCK	Remains NULL (link rejected)

Step	users	platform_decisions	Expert Bank Link (sepay_bank_account_xid)
[9] Manual Suspend	is_suspended=true, reason=Admin input	type=ADMIN_SUSPENSION	Unchanged
[13] Reactivate	is_suspended=false, reason=NULL	type=ADMIN_REACTIVATION	Unchanged (Must re-link if fraud case)

MF-18 is now complete. The platform's integrity is secured by a dual-layer defense: an automated deduplication guard that instantly blocks fraudulent bank account linking (protecting the chi hộ withdrawal pipeline from shell account abuse), and a manual admin suspension tool for handling behavioral violations. All actions are immutably logged in `platform_decisions` for the Integrity Monitor, and escrow funds remain safe and untouched during account freezes.